

LLMaBoK

LLM Application Base of Knowledge

조 용 진

(drajin.cho@bosornd.com)

개발 환경

- Visual Studio Code + GitHub Copilot
- Python 3.13
- Node.js v22.19, Flutter 3.35(Dart 3.9)
- <https://github.com/bosornd/llmabok2>
 - init branch : 준비 자료, main branch : 완성된 코드
- <https://github.com/bosornd/book-restaurant-agent>

⚠ Warning

Don't install Flutter to a directory or path that meets one or both of the following conditions:

- The path contains special characters or spaces.
- The path requires elevated privileges.

As an example, `C:\Program Files` meets both conditions.

Language Model 개요



1. 데이터 학습

인터넷, 책 등 방대한 텍스트 데이터를 읽고
단어 간의 관계, 문법, 의미 패턴을 학습합니다.



2. 단어 확률 계산

입력된 문맥을 바탕으로, 다음에 올 단어의 확률을 계산하여 순위를 매깁니다.



3. 문장 생성

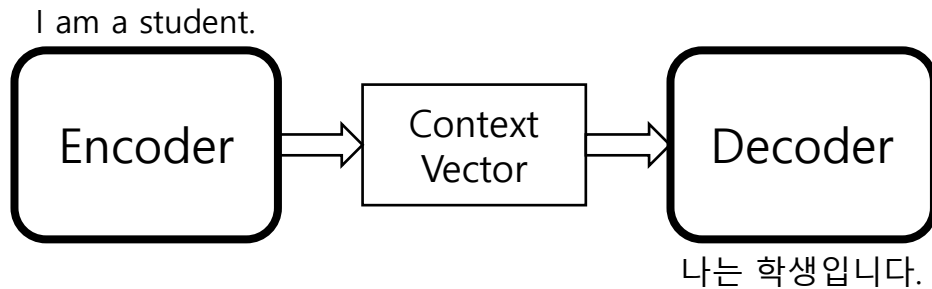
가장 확률 높은 단어를 선택해 문장을 이어갑니다.
이 과정을 반복하여 글을 완성합니다.

나는 오늘 점심으로 → 밥(0.7) → 먹다(0.7)
국수(0.6) 자다(0.1)
자동차(0.1) 짓다(0.5)

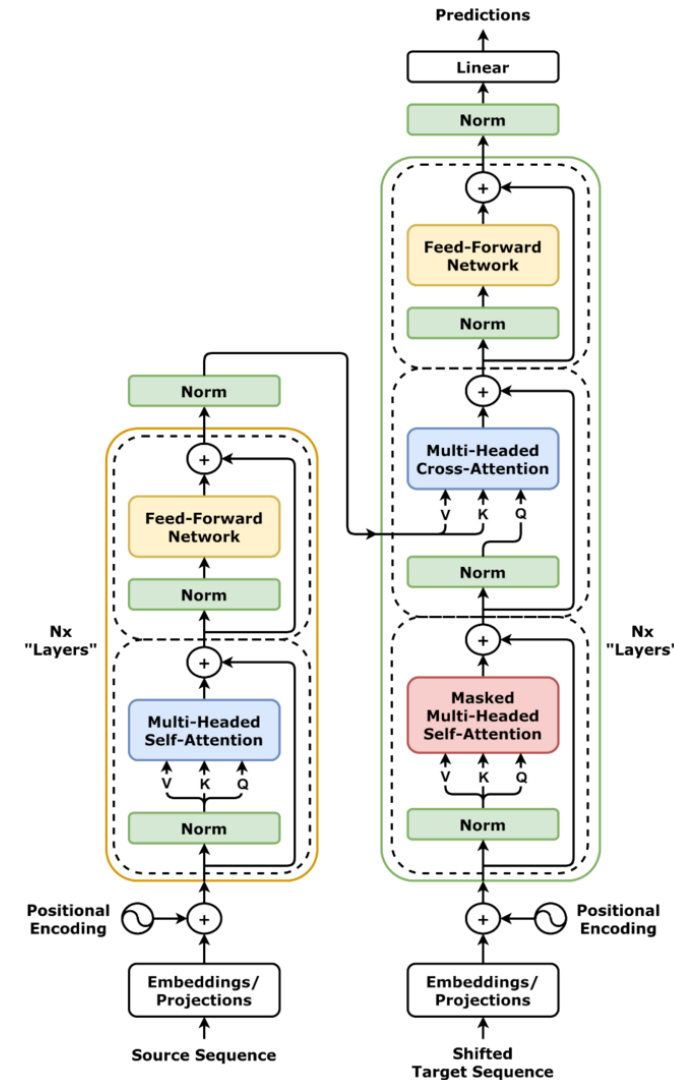
Transformer 모델

<https://en.wikipedia.org/wiki/Seq2seq>
[https://en.wikipedia.org/wiki/Transformer_\(deep_learning_architecture\)](https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture))

■ Sequence-to-Sequence

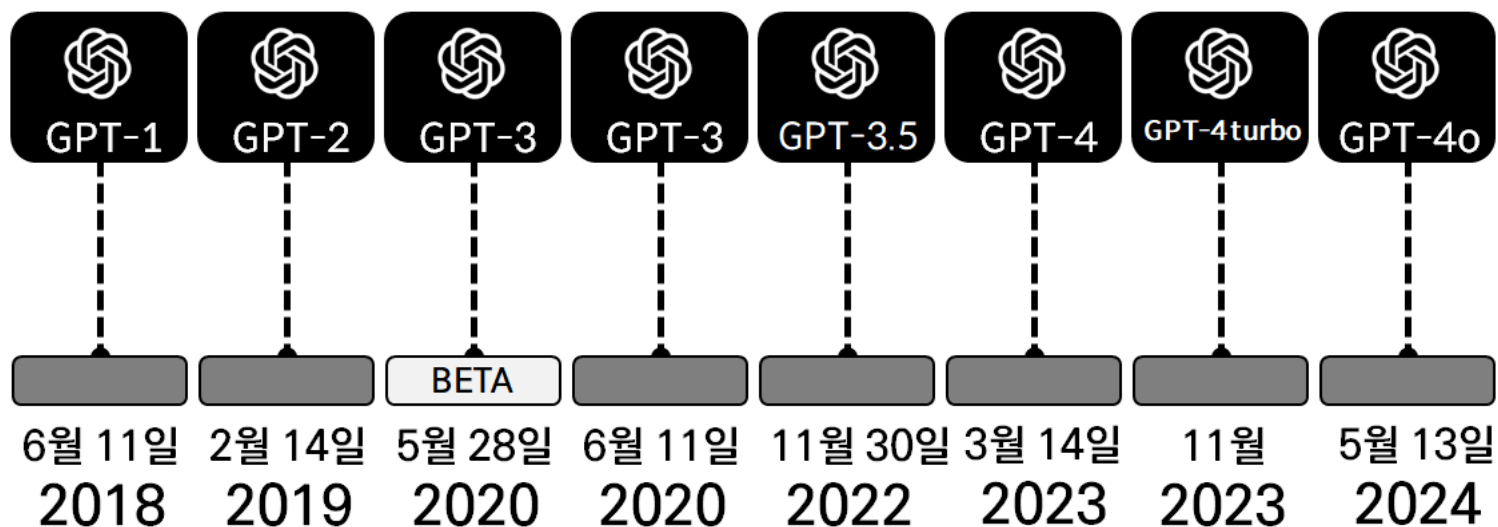


■ Attention Is All You Need - Google (2017)



GPT – Open AI

Generative Pre-trained Transformer

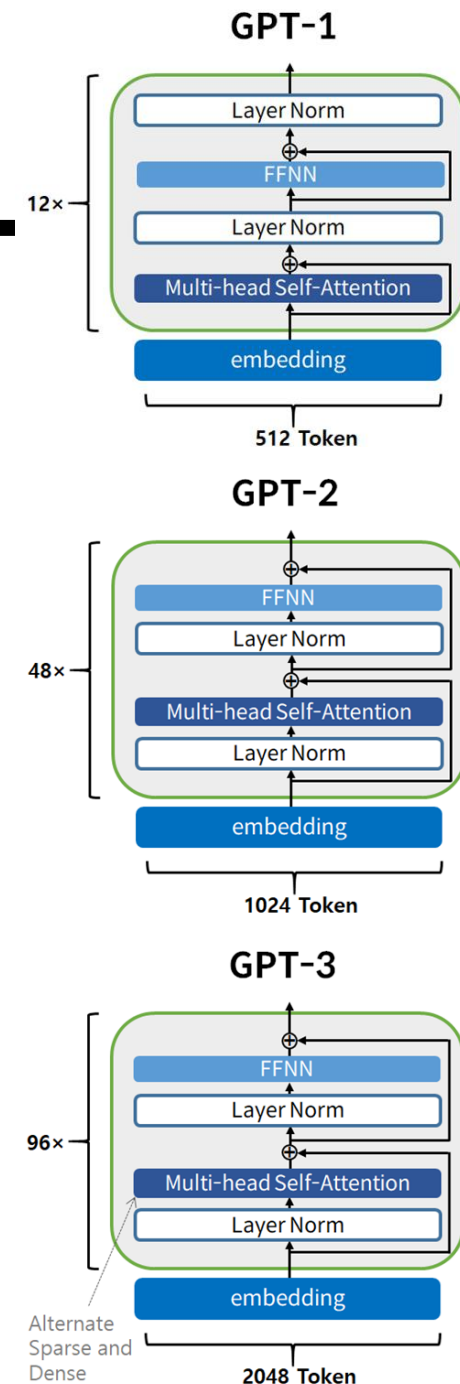


117M 파라미터

1.5B 파라미터

175B 파라미터

1800B 파라미터(추정)



LLM의 활용

1. 콘텐츠 및 텍스트 생성

- ◆ 마케팅 광고
- ◆ 보고서 및 문서 작성
- ◆ 창의적 글쓰기

2. 정보 요약 및 분석

- ◆ 뉴스 기사 및 논문 요약
- ◆ 고객 피드백 분석
- ◆ 회의록 정리

3. 번역 및 다국어 지원

- ◆ 실시간 통역
- ◆ 번역 서비스

4. 교육 및 학습 지원

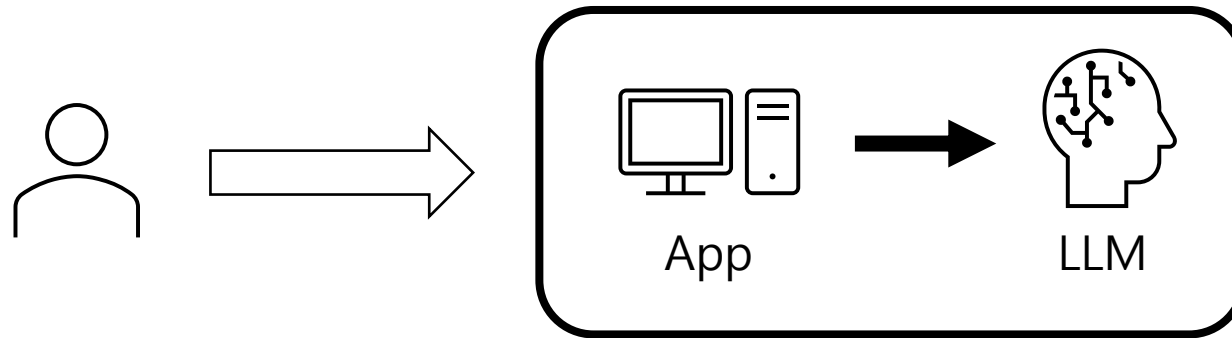
- ◆ 맞춤형 튜터
- ◆ 퀴즈 및 문제 생성
- ◆ 언어 학습

5. 코드 작성 및 개발 지원

- ◆ 자동 코드 완성
- ◆ 버그 수정
- ◆ 문서화

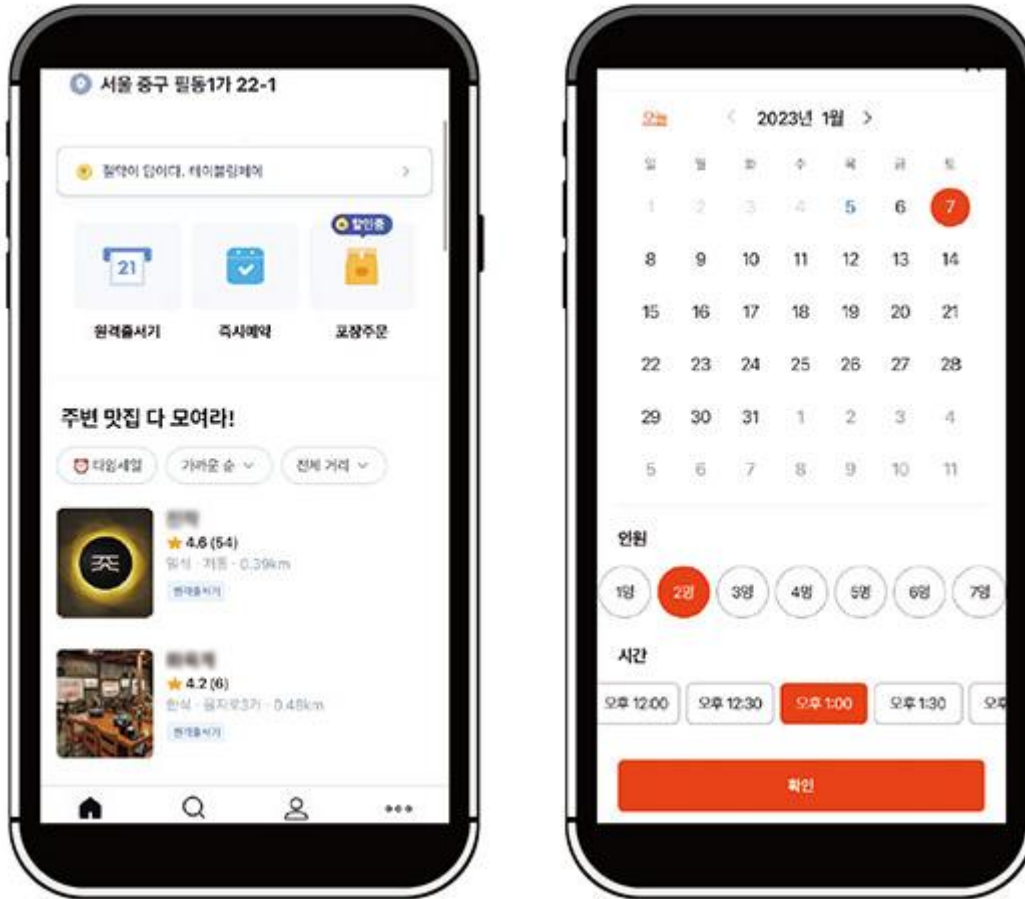
LLM 앱/서비스

- LLM의 핵심 기술인 텍스트 생성, 이해, 분석 능력 등을 활용하여 사용자에게 특정 기능이나 가치를 제공하는 소프트웨어 및 솔루션
- LLM의 능력을 특정 목적에 맞게 적용하고 사용자 친화적인 형태로 구현



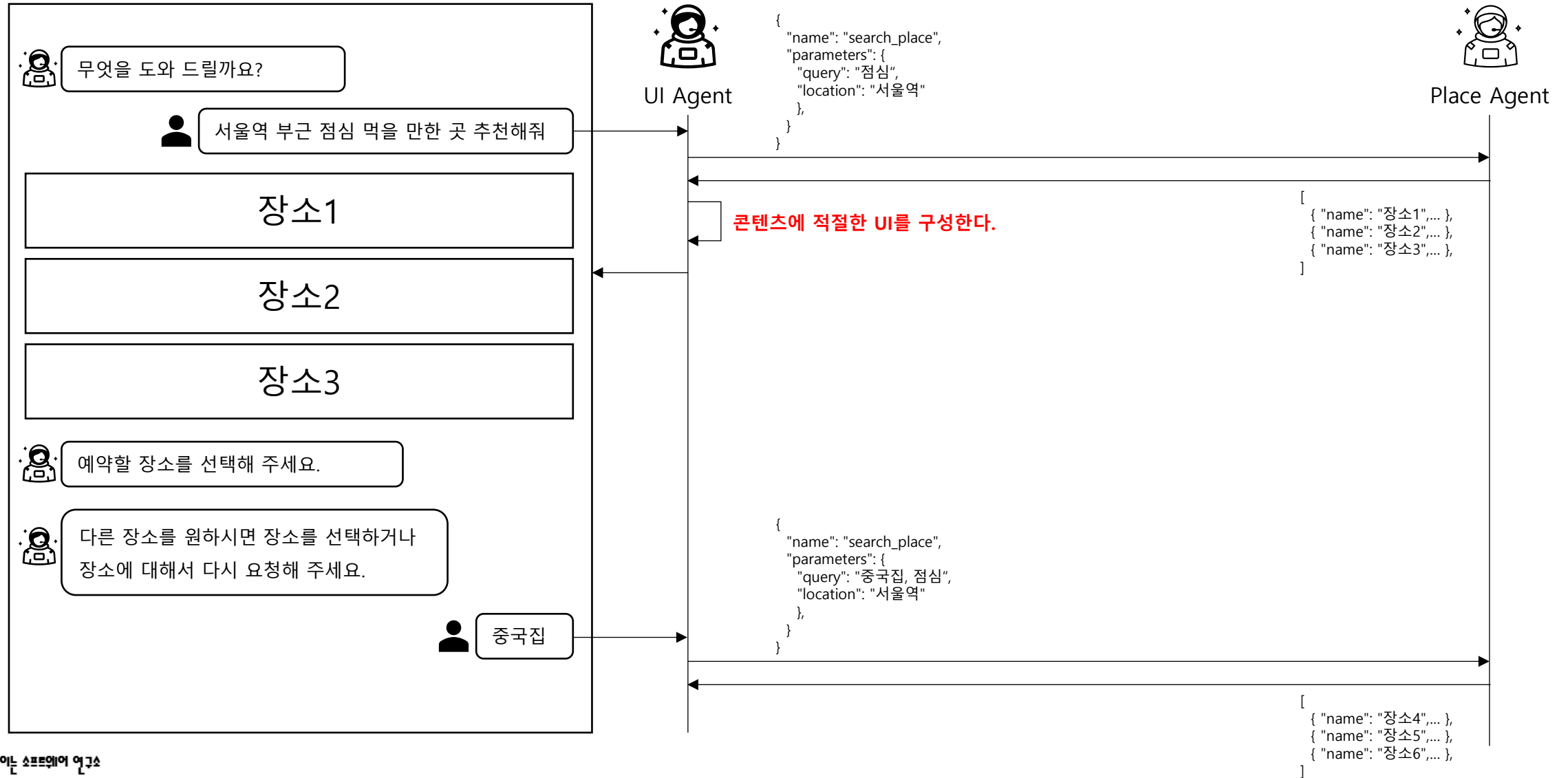
자연어 인터페이스
지능형 기능

LLM 앱 - 식당 예약 서비스

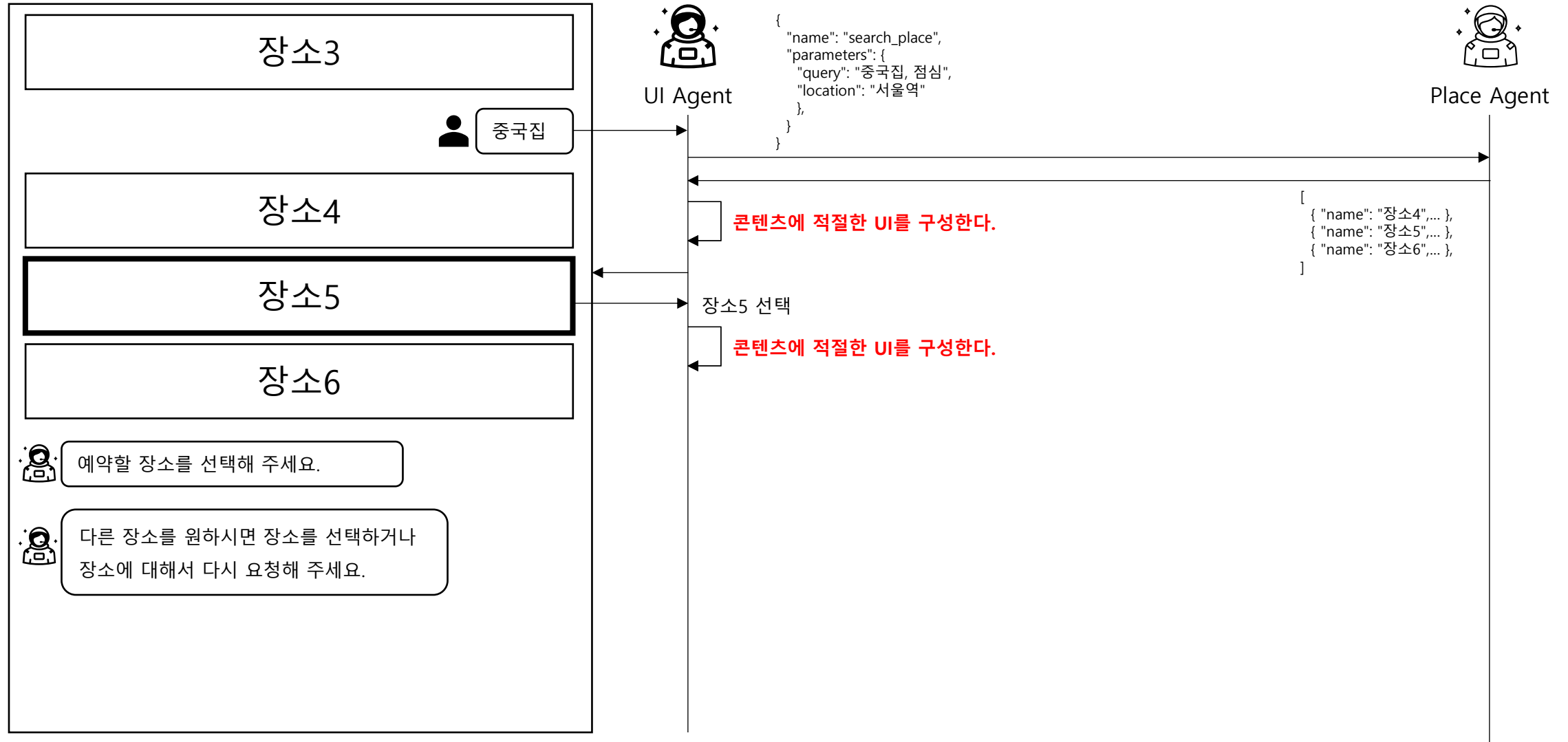


가족과 외식을 하려고 하는데, ...
친구들과 술한잔하려고 하는데, ...
간단하게 점심 먹으려고 하는데, ...
:
어디가 좋을까? 추천해줘.

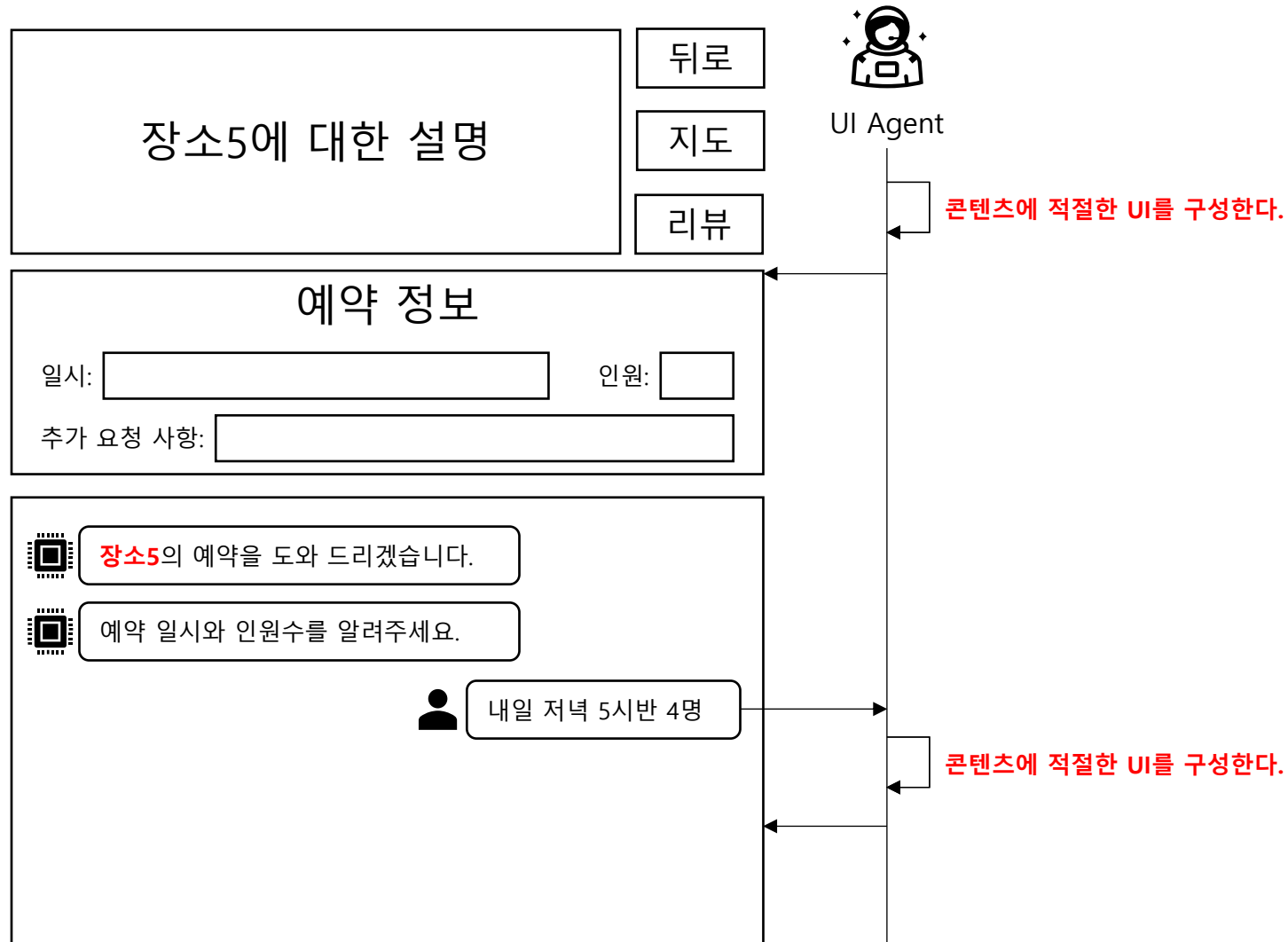
LLM 앱 - 식당 예약 서비스



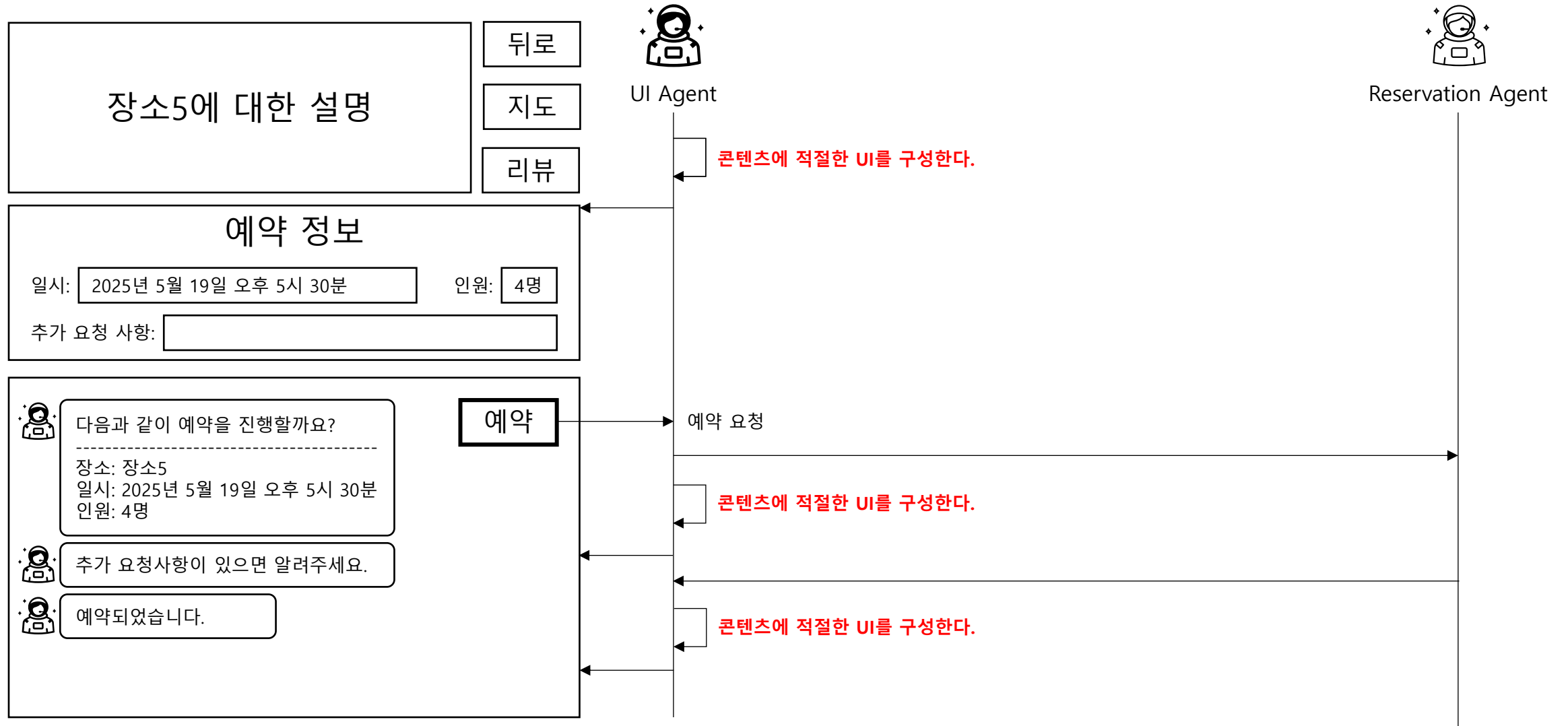
LLM 앱 - 식당 예약 서비스



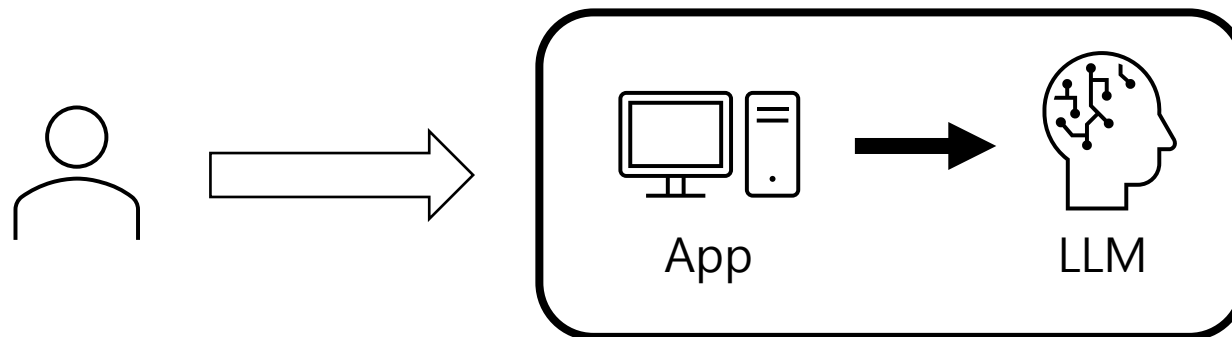
LLM 앱 - 식당 예약 서비스



LLM 앱 – 식당 예약 서비스



LLM 앱의 설계 요소



1. LLM 모델

- ◆ 목적에 맞는 성능
- ◆ 비용 및 운영 효율성
- ◆ 인프라 및 보안

2. 프롬프트

- ◆ 명확한 지침 및 역할 부여
- ◆ 출력 가이드 및 파서
- ◆ 프롬프트 엔지니어링

3. 외부 도구 통합

- ◆ RAG
- ◆ Tools & MCP
- ◆ 컨텍스트 엔지니어링

4. 에이전트

- ◆ 복잡한 문제 해결
- ◆ 역할 분담, 협업 및 소통
- ◆ 전문성, 효율성, 유연성

LLMaBoK

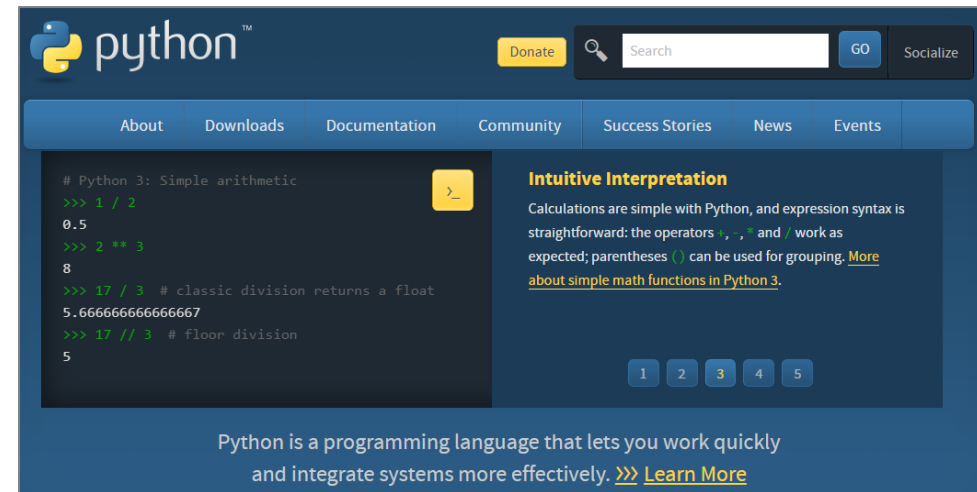
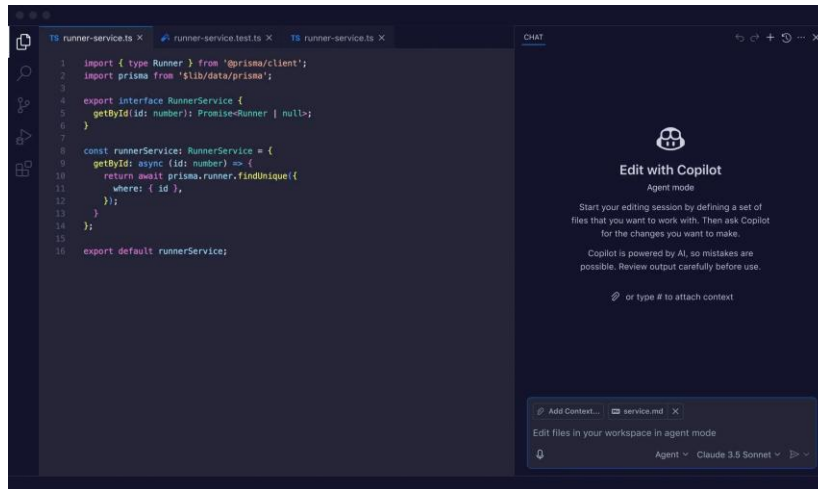
Getting started

조 용 진

(drajin.cho@bosornd.com)

개발 환경

Visual Studio Code + Copilot

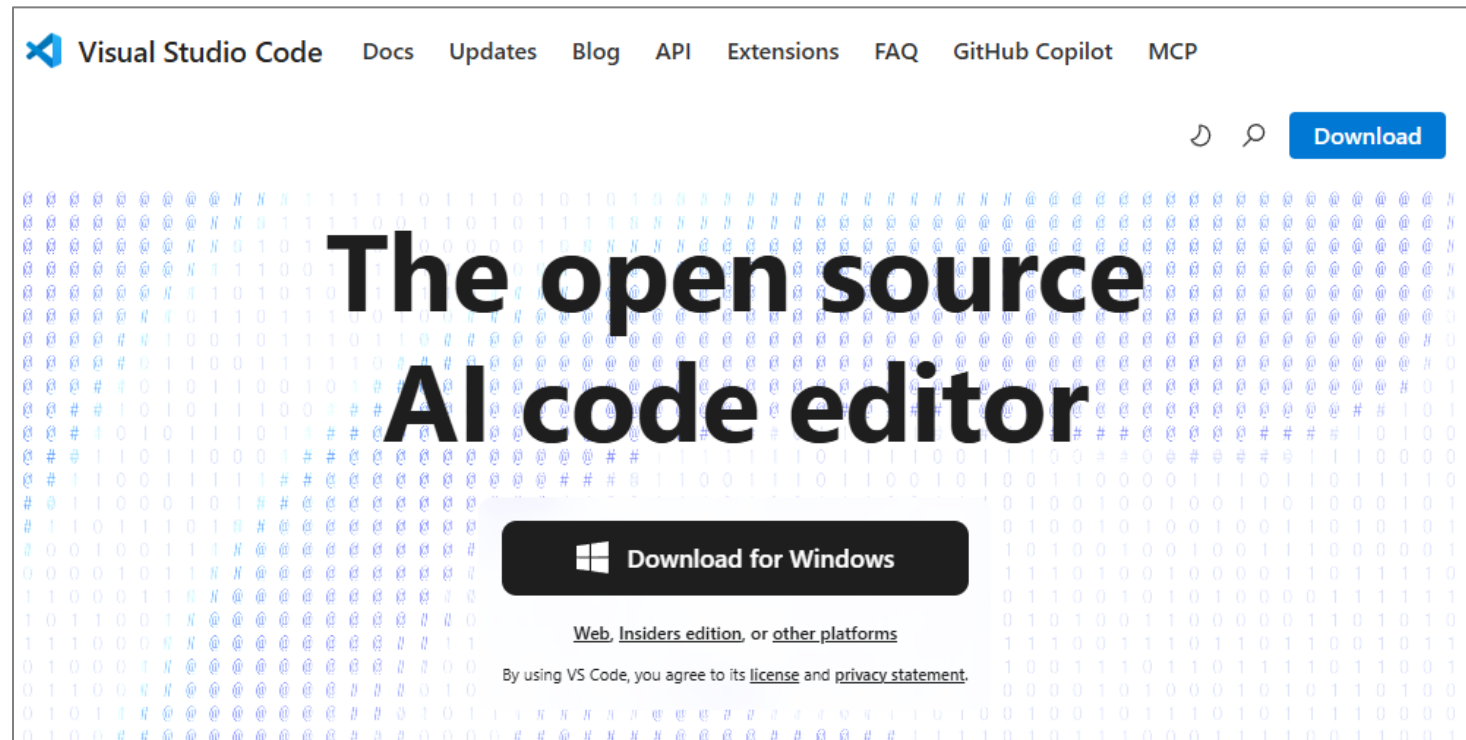


Google Agent Development Kit

Visual Studio Code

Step 1. a folder for your workspace

Step 2. Open with VS Code



<https://code.visualstudio.com>

Step 3. Setup Python VENV

```
(.venv) D:\Work\lma bok>
```



Python

Step 3. Setup Python VENV

Step 4. Install packages

```
(.venv) python -m pip install --upgrade pip
```

```
(.venv) pip install google-adk
```



Agent Development Kit

What is Agent Development Kit?

Agent Development Kit (ADK) is a flexible and modular framework for **developing and deploying AI agents**. While optimized for Gemini and the Google ecosystem, ADK is **model-agnostic**, **deployment-agnostic**, and is built for **compatibility with other frameworks**. ADK was designed to make agent development feel more like software development, to make it easier for developers to create, deploy, and orchestrate agentic architectures that range from simple tasks to complex workflows.

<https://google.github.io/adk-docs>

Gemini API Key

Step 5. Get Gemini API Key



<https://aistudio.google.com/app/apikey>

Gemini API Key

Step 5. Get Gemini API Key

Step 6. Store Key in .env and load .env

.env

```
GOOGLE_API_KEY=AlzaSaDLc0tqC89ZyD71ZAiHJZXVsMu-T20iN1c  
GOOGLE_GENAI_USE_VERTEXAI=False
```

```
from dotenv import load_dotenv  
load_dotenv()
```

T1. chat_agent

- 간단한 챗봇 에이전트를 만들어 보자.

chat_agent/agent.py

```
from google.adk.agents import Agent

root_agent = Agent(
    name="chat_agent",
    model="gemini-2.0-flash",
    instruction="""You are a friendly agent who can have casual,
                  everyday conversations with users."""
)
```

T1. chat_agent

chat_agent/__init__.py

```
from . import agent
```

```
(.venv) adk run chat_agent
```

```
(.venv) adk api_server --reload_agents
```

```
(.venv) adk web --reload_agents
```

T1. chat_agent

Agent Development Kit

chat_agent

TraceEventsStateArtifactsSessionsEval

Invocations

안녕, 나는 용진인데, 너는 이름이 뭐니?

Invocation ID: e-01f08113-8ed2-4ea0-801b-64fa5b04bfa2

invocation

2090.07ms

agent_run [chat_agent]

2089.67ms

call_llm

2087.40ms

그럼, 챗이라고 부르면 되나?

Invocation ID: e-c128e810-4861-4dc0-9d7f-bd636687850c

invocation

1495.18ms

agent_run [chat_agent]

1494.77ms

call_llm

1493.31ms

SESSION ID bdc15157-65b1-4f9e-8353-1712a9ce7407

Token Streaming | + New Session

안녕, 나는 용진인데, 너는 이름이 뭐니?

안녕하세요 용진님! 저는 챗 에이전트라고 해요. 만나서 반갑습니다! 😊 오늘 하루 어떠셨어요?

그럼, 챗이라고 부르면 되나?

네, 편하게 챗이라고 불러주세요! 😊

챗, 그럼 나와 너의 관계는?

T1. chat_agent

Event 2 of 2

Request

Response

chat_agent

content:

parts:

0:

text: "네, 편하게 챗이라고 불러주세요! 😊"

role: "model"

usageMetadata:

candidatesTokenCount: 15

candidatesTokensDetails:

0:

modality: "TEXT"

tokenCount: 15

promptTokenCount: 114

promptTokensDetails:

0:

modality: "TEXT"

tokenCount: 114

totalTokenCount: 129

invocationId: "e-c128e810-4861-4dc0-9d7f-bd636687850c"

author: "chat_agent"

actions:

stateDelta:

artifactDelta:

requestedAuthConfigs:

id: "c0d1289c-de22-4df8-99b6-a0c5016bf929"

timestamp: 1754864069.538043

title: "text:네, 편하게 챗이라고 불러주세요! 😊"

SESSION ID bdc15157-65b1-4f9e-8353-1712a9ce7407

Token Streaming

New Session

안녕, 나는 용진인데, 너는 이름이 뭐니?

안녕하세요 용진님! 저는 챗 에이전트라고 해요. 만나서 반갑습니다! 😊 오늘 하루 어떠셨어요?

그럼, 챗이라고 부르면 되나?

네, 편하게 챗이라고 불러주세요! 😊

챗, 그럼 나와 너의 관계는?

T1. chat_agent

Event 2 of 2

Request

Response

```
model: "gemini-2.0-flash"
config:
  http_options:
    headers:
      x-goog-api-client: "google-adk/1.9.0 gl-python/3.13.6"
      user-agent: "google-adk/1.9.0 gl-python/3.13.6"
  system_instruction: "You are a friendly agent who can have casual, everyday conversations with users. You are an agent. Your internal name is 'chat_agent'. The description about you is 'Agent to engage in everyday conversations with users.'"
  contents:
    0:
      parts:
        0:
          text: "안녕, 나는 용진인데, 너는 이름이 뭐니?"
          role: "user"
      1:
        parts:
          0:
            text: "안녕하세요 용진님! 저는 챗 에이전트라고 해요. 만나서 반갑습니다! 😊 오늘 하루 어떠셨어요?"
            role: "model"
      2:
        parts:
          0:
            text: "그럼, 챗이라고 부르면 되나?"
            role: "user"
```

SESSION ID bdc15157-65b1-4f9e-8353-1712a9ce7407

Token Streaming | + New Session

안녕, 나는 용진인데, 너는 이름이 뭐니?

안녕하세요 용진님! 저는 챗 에이전트라고 해요. 만나서 반갑습니다! 😊 오늘 하루 어떠셨어요?

그럼, 챗이라고 부르면 되나?

네, 편하게 챗이라고 불러주세요! 😊

챗, 그럼 나와 너의 관계는?

T1. chat_agent

Event 2 of 2 < > X

EventRequestResponse

```
content:
  parts:
    0:
      text: "네, 편하게 챗이라고 불러주세요! 😊"
  role: "model"
usage_metadata:
  candidates_token_count: 15
  candidates_tokens_details:
    0:
      modality: "TEXT"
      token_count: 15
  prompt_token_count: 114
  prompt_tokens_details:
    0:
      modality: "TEXT"
      token_count: 114
  total_token_count: 129
```

SESSION ID bdc15157-65b1-4f9e-8353-1712a9ce7407 Token Streaming | + New Session 🗑️ ⬇️

안녕, 나는 용진인데, 너는 이름이 뭐니?

안녕하세요 용진님! 저는 챗 에이전트라고 해요. 만나서 반갑습니다! 😊 오늘 하루 어떠셨어요?

그럼, 챗이라고 부르면 되나?

네, 편하게 챗이라고 불러주세요! 😊

챗, 그럼 나와 너의 관계는?

📎 ⋮ 🎤 🎥

Gemini Models

<https://ai.google.dev/gemini-api/docs/models>

2.5 Pro ✨

최대 응답 정확도와 최첨단 성능을 갖춘
Google의 가장 강력한 사고 모델

- 오디오, 이미지, 동영상, 텍스트를 입력하고 텍스트 응답을 받습니다.
- 까다로운 문제 해결, 대규모 데이터베이스 분석 등
- 복잡한 코딩, 추론, 멀티모달 이해에 적합

2.5 Flash ✨

가격 대비 성능이 가장 우수한 모델로, 다
재다능한 기능을 제공합니다.

- 오디오, 이미지, 동영상, 텍스트를 입력하고 텍스트 응답을 받습니다.
- 모델이 필요에 따라 사고하거나 사고 예산을 구성할 수 있습니다.
- 생각이 필요한 자연 시간이 짧고 대량의 작업에 적합

2.5 Flash-Lite ⚡

비용 효율성과 짧은 지연 시간에 최적화
된 Gemini 2.5 Flash 모델

- 오디오, 이미지, 동영상, 텍스트를 입력하고 텍스트 응답을 받습니다.
- 높은 처리량을 지원하는 가장 비용 효율적인 모델
- 실시간, 지연 시간이 짧은 사용 사례에 적합

Gemini 2.0 Flash

모든 작업에서 뛰어난 성능을 제공하고, 100만 개의 토큰 컨텍스트 윈도우를 지원하며, 에이전트 시대를 위해 빌드된 가장 균형 잡힌 멀티모달 모델입니다.

	무료 등급	유료 등급, 1백만 토큰당 가격(USD)
가격 입력	무료	\$0.10 (텍스트 / 이미지 / 동영상) \$0.70 (오디오)
출력 가격	무료	\$0.40
컨텍스트 캐싱 가격	무료	1,000,000토큰당 \$0.025 (텍스트/이미지/동영상) 1,000,000토큰당\$0.175 (오디오)
컨텍스트 캐싱 (저장소)	무료, 시간당 최대 1,000,000개의 토큰 저장	시간당 토큰 1,000,000개당 \$1.00
이미지 생성 가격 책정	무료	이미지당 \$0.039*
조정 가격	사용할 수 없음	사용할 수 없음
Google 검색을 사용하는 그라운드링	최대 500RPD까지 무료	1,500 RPD (무료), 이후 요청 1,000개당 \$35
Live API	무료	입력: \$0.35 (텍스트), \$2.10 (오디오 / 이미지[동영상]) 출력: \$1.50 (텍스트), \$8.50 (오디오)
제품 개선에 사용됨	예	아니요

[*] 이미지 출력은 토큰 1,000,000개당 30달러입니다. 최대 1024x1024px의 출력 이미지는 1,290개의 토큰을 사용하며 이미지당 \$0.039에 해당합니다.

T2. math_agent with function tools

- 연산 도구를 사용하는 Math 에이전트를 만들어 보자.

math_agent/agent.py

```
from google.adk.agents import Agent

root_agent = Agent(
    name="chat_agent",
    model="gemini-2.0-flash",
    instruction="""4칙 연산 도구를 사용해서 사용자가 요청한 수식을 계산하고,
                    결과를 답하세요.""",
    tools=[add, subtract, multiply, divide],
)
```

T2. math_agent with function tools

math_agent/agent.py

```
def add(a: int, b: int) -> int:
    """
    Adds two integers and returns the result.

    Args:
        a (int): The first integer.
        b (int): The second integer.

    Returns:
        int: The sum of the two integers.
    """
    return a + b
```

T2. math_agent with function tools

Agent Development Kit

math_agent

TraceEventsStateArtifactsSessionsEval

Invocations

2에 3을 더한 결과에 5를 곱하면?

Invocation ID: e-cb20428f-c4ec-4289-9064-166b791040f5

invocation

5084.21ms

agent_run [math_agent]

5083.97ms

call_llm

1682.56ms

execute_tool add

0.27ms

call_llm

1750.76ms

execute_tool multi_

0.16ms

call_llm

1644.46ms

SESSION ID 5b99aed4-7103-4047-8b9f-2c19d11fea37

Token Streaming | + New Session

2에 3을 더한 결과에 5를 곱하면?

⚡ add

✓ add

⚡ multiply

✓ multiply

2에 3을 더한 결과는 5이고, 5에 5를 곱한 결과는 25입니다.

Type a Message...

📎

⋮

🎤

📹

주식회사 보이는 소프트웨어 연구소

T2. math_agent with function tools

The screenshot displays the Agent Development Kit (ADK) interface for a session with the agent 'math_agent'. The interface is divided into a left sidebar and a main chat area.

Left Sidebar:

- Header: Agent Development Kit
- Dropdown: math_agent
- Tabs: Trace, Events, State, Artifacts, Sessions, Eval
- Conversations section with 'Events' and 'Trace' sub-tabs.
- Conversation log:
 - 0 functionCall:add
 - 1 functionResponse:add
 - 2 functionCall:multiply
 - 3 functionResponse:multiply
 - 4 text:2에 3을 더한 결과는 5이고, 5에 5를 곱한 결과는 25입니다.

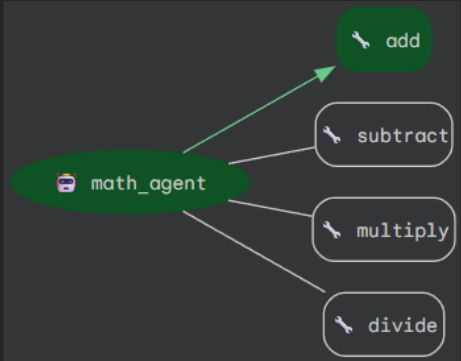
Main Chat Area:

- Session ID: 5b99aed4-7103-4047-8b9f-2c19d11fea37
- Token Streaming toggle and '+ New Session' button.
- User input: 2에 3을 더한 결과에 5를 곱하면?
- Agent responses (indicated by purple 'D0' icons):
 - ⚡ add
 - ✓ add
 - ⚡ multiply
 - ✓ multiply
 - 2에 3을 더한 결과는 5이고, 5에 5를 곱한 결과는 25입니다.
- Input field: Type a Message...
- Input controls: Attach, More options, Voice, Video.

T2. math_agent with function tools

Event 1 of 5

EventRequestResponse



```
graph LR; math_agent((math_agent)) --> add(add); math_agent --> subtract(subtract); math_agent --> multiply(multiply); math_agent --> divide(divide);
```

```
content:
  parts:
    0:
      functionCall:
        id: "adk-0e77ac0d-947d-4a44-a391-b83927971531"
        args:
          a: 2
          b: 3
          name: "add"
      role: "model"
  usageMetadata:
    candidatesTokenCount: 5
    candidatesTokensDetails:
      0:
        modality: "TEXT"
        tokenCount: 5
    promptTokenCount: 286
    promptTokensDetails:
      0:
```

SESSION ID 5b99aed4-7103-4047-8b9f-2c19d11fea37

Token Streaming | New Session

2에 3을 더한 결과에 5를 곱하면?



 ⚡ add

 ✓ add

 ⚡ multiply

 ✓ multiply

 2에 3을 더한 결과는 5이고, 5에 5를 곱한 결과는 25입니다.

Type a Message...

T2. math_agent with function tools

The screenshot displays the Agent Development Kit (ADK) interface for a `math_agent`. The left sidebar contains tabs for Trace, Events, State, Artifacts, Sessions, and Eval. The 'Invocations' section shows a sequence of events: an initial query, a call to `agent_run [math_agent]`, a call to `call_llm`, an `execute_tool divide` action, and a final `call_llm` response. The main chat area shows the interaction between the user and the agent, including tool calls like `add`, `multiply`, and `divide`, and the final calculation result.

Agent Development Kit

math_agent

Trace Events State Artifacts Sessions Eval

Invocations

2에 3을 더한 결과에 5를 곱하면?

이전 계산 결과를 3으로 나누면?

Invocation ID:
e-cbe218cd-d9b7-4710-afa0-d9a0f96780f5

→ invocation 3262.94ms

agent_run [math_agent] 3262.28ms

call_llm 1629.19ms

→ execute_tool divide 0.16ms

call_llm 1629.29ms

SESSION ID 5b99aed4-7103-4047-8b9f-2c19d11fea37

Token Streaming | + New Session

2에 3을 더한 결과에 5를 곱하면?

add

add

multiply

multiply

2에 3을 더한 결과는 5이고, 5에 5를 곱한 결과는 25입니다.

이전 계산 결과를 3으로 나누면?

divide

divide

이전 계산 결과(25)를 3으로 나누면 8.333333333333339입니다.

Type a Message...

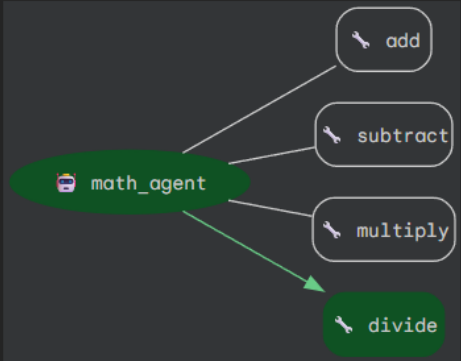
SW 주식회사 보이는 소프트웨어 연구소

T2. math_agent with function tools

Event 6 of 8

Request

Response



```
graph LR; math_agent((math_agent)) --> add([add]); math_agent --> subtract([subtract]); math_agent --> multiply([multiply]); math_agent --> divide([divide]);
```

```
content:
  parts:
    0:
      functionCall:
        id: "adk-5976f4f5-92f2-40f8-939e-ebcac7314cbc"
        args:
          b: 3
          a: 25
          name: "divide"
      role: "model"
      usageMetadata:
        candidatesTokenCount: 5
        candidatesTokensDetails:
          0:
            modality: "TEXT"
            tokenCount: 5
        promptTokenCount: 344
        promptTokensDetails:
          0:
```

SESSION ID 5b99aed4-7103-4047-8b9f-2c19d11fea37

Token Streaming | + New Session

2에 3을 더한 결과에 5를 곱하면?



 ⚡ add

 ✓ add

 ⚡ multiply

 ✓ multiply

 2에 3을 더한 결과는 5이고, 5에 5를 곱한 결과는 25입니다.

이전 계산 결과를 3으로 나누면?



 ⚡ divide

 ✓ divide

 이전 계산 결과(25)를 3으로 나누면 8.333333333333333입니다.

Type a Message...

T2. math_agent with function tools

Event 6 of 8

Request

```
model: "gemini-2.0-flash"
config:
  http_options:
    headers:
      x-goog-api-client: "google-adk/1.9.0 gl-
      user-agent: "google-adk/1.9.0 gl-python,
system_instruction: "You are a helpful agent wh
questions about mathematical operations. You ar
name is "math_agent". The description about you
questions about mathematical operations.""
tools:
  0:
    function_declarations:
      0:
        description: "Adds two floats an
        Args: a (float): The first float
        float. Returns: float: The sum o
        name: "add"
        parameters:
          properties:
            a:
              type: "NUMBER"
            b:
              type: "NUMBER"
          required:
            0: "a"
            1: "b"
          type: "OBJECT"
      1:
        description: "Subtracts the seco
        and returns the result. Args: a
        float, b (float): The second flo
        difference of the two floats. "
        name: "subtract"
        parameters:
          properties:
```

contents:

```
0:
  parts:
    0:
      text: "2에 3을 더한 결과에 5를 곱하면?"
      role: "user"
1:
  parts:
    0:
      function_call:
        args:
          a: 2
          b: 3
          name: "add"
        role: "model"
2:
  parts:
    0:
      function_response:
        name: "add"
        response:
          result: 5
      role: "user"
3:
  parts:
    0:
      function_call:
        args:
          a: 5
          b: 5
          name: "multiply"
        role: "model"
4:
  parts:
    0:
      function_response:
        name: "multiply"
        response:
          result: 25
      role: "user"
5:
  parts:
    0:
      text: "2에 3을 더한 결과는 5이고, 5에 5를 곱한 결과는 25
      입니다."
      role: "model"
6:
  parts:
    0:
      text: "이전 계산 결과를 3으로 나누면?"
      role: "user"
```

17-8b9f-2c19d11fea37

Token Streaming | + New Session

2에 3을 더한 결과에 5를 곱하면?

5이고, 5에 5를 곱한 결과는 25입니다.

이전 계산 결과를 3으로 나누면?

3으로 나누면 8.333333333333339입니다.

Type a Message...

📎 ⋮ 🎤 🎥

T3. search_agent using google_search

- 구글 검색 도구를 사용하는 RAG 에이전트를 만들어 보자.

search_agent/agent.py

```
from google.adk.agents import Agent
from google.adk.tools import google_search

root_agent = Agent(
    name="search_agent",
    model="gemini-2.0-flash",
    instruction="""사용자의 질문에 필요한 정보를 google_search 도구로 검색하고,
                  검색 결과를 근거로 답하세요.""",
    tools=[google_search],
)
```

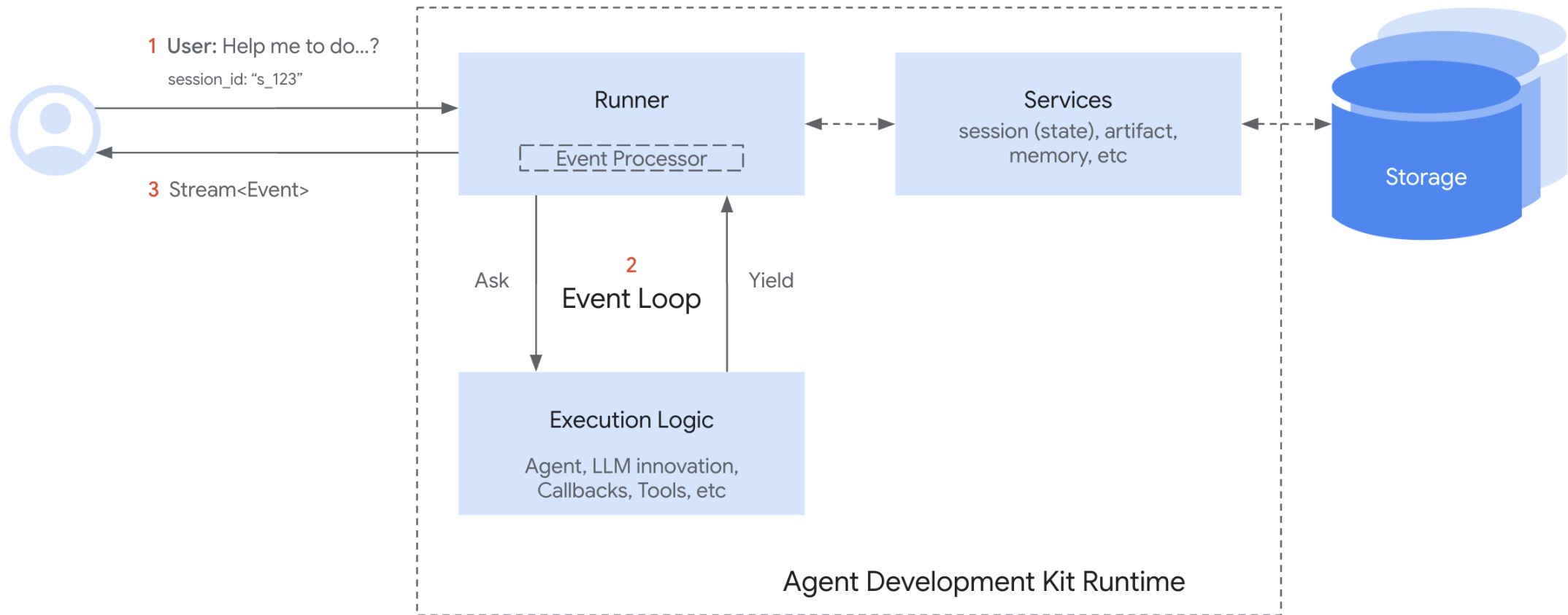
LLMaBoK

ADK Runtime

조 용 진

(drajin.cho@bosornd.com)

ADK Runtime



T4. runner

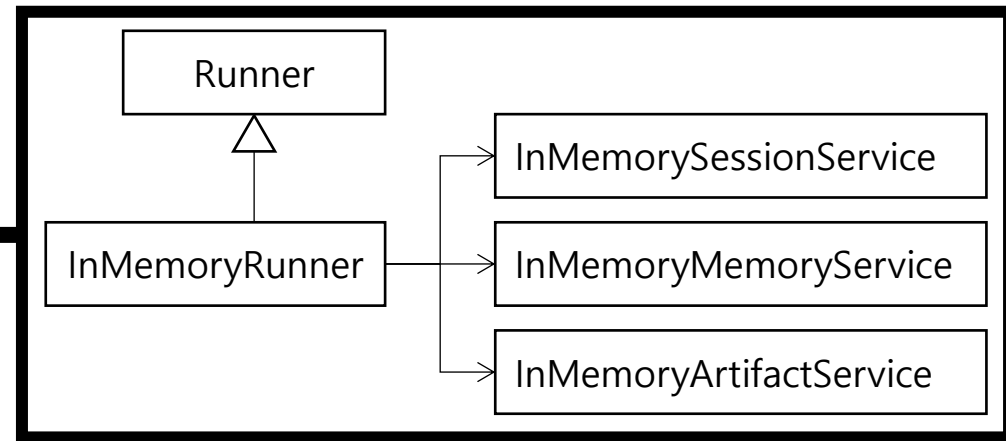
- 기본적인 ADK Runner를 구성해 보자.

chat_agent/agent.py

```
import asyncio
from google.adk.runners import InMemoryRunner
from google.genai.types import UserContent

async def main():
    runner = InMemoryRunner(agent=root_agent, app_name=root_agent.name)

    session = await runner.session_service.create_session(app_name=runner.app_name,
                                                         user_id="user1")
```



T4. runner

chat_agent/agent.py

```
print("일상적인 대화를 지원하는 에이전트입니다. 'exit' 입력 시 종료됩니다.")
while True:
    user_input = input("User: ")
    if user_input.lower() == "exit": break

    for event in runner.run(user_id=session.user_id, session_id=session.id,
                           new_message=UserContent(user_input)):
        if event.is_final_response():
            print(f"Agent: {event.content.parts[0].text}")
```

```
asyncio.run(main())
```


Asynchronous Programming

```
def func(message):  
    print(message)
```

```
def main():  
    func("Hello")  
    func("World")
```

```
main()
```

```
import asyncio  
  
async def func(delay, message):  
    print(message)  
    await asyncio.sleep(delay)  
    print(message)
```

```
async def main():  
    f1 = func(2, "Hello")    # coroutine object f1  
    f2 = func(1, "World")   # coroutine object f2  
    await asyncio.gather(f1, f2)
```

```
asyncio.run(main())
```

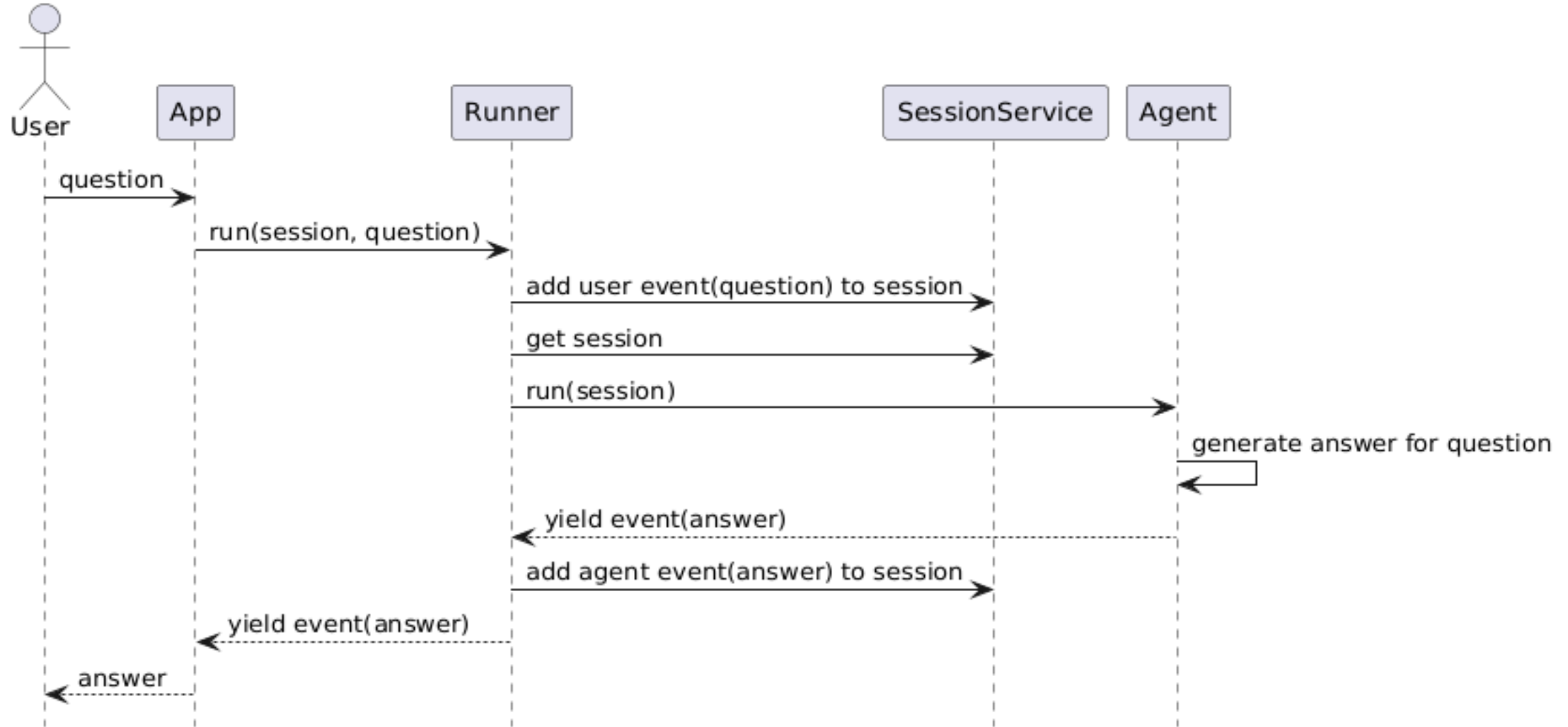
Generator

```
def count():  
    print("--- 0")  
    yield 1  
    print("--- 1")  
    yield 2  
    print("--- 2")
```

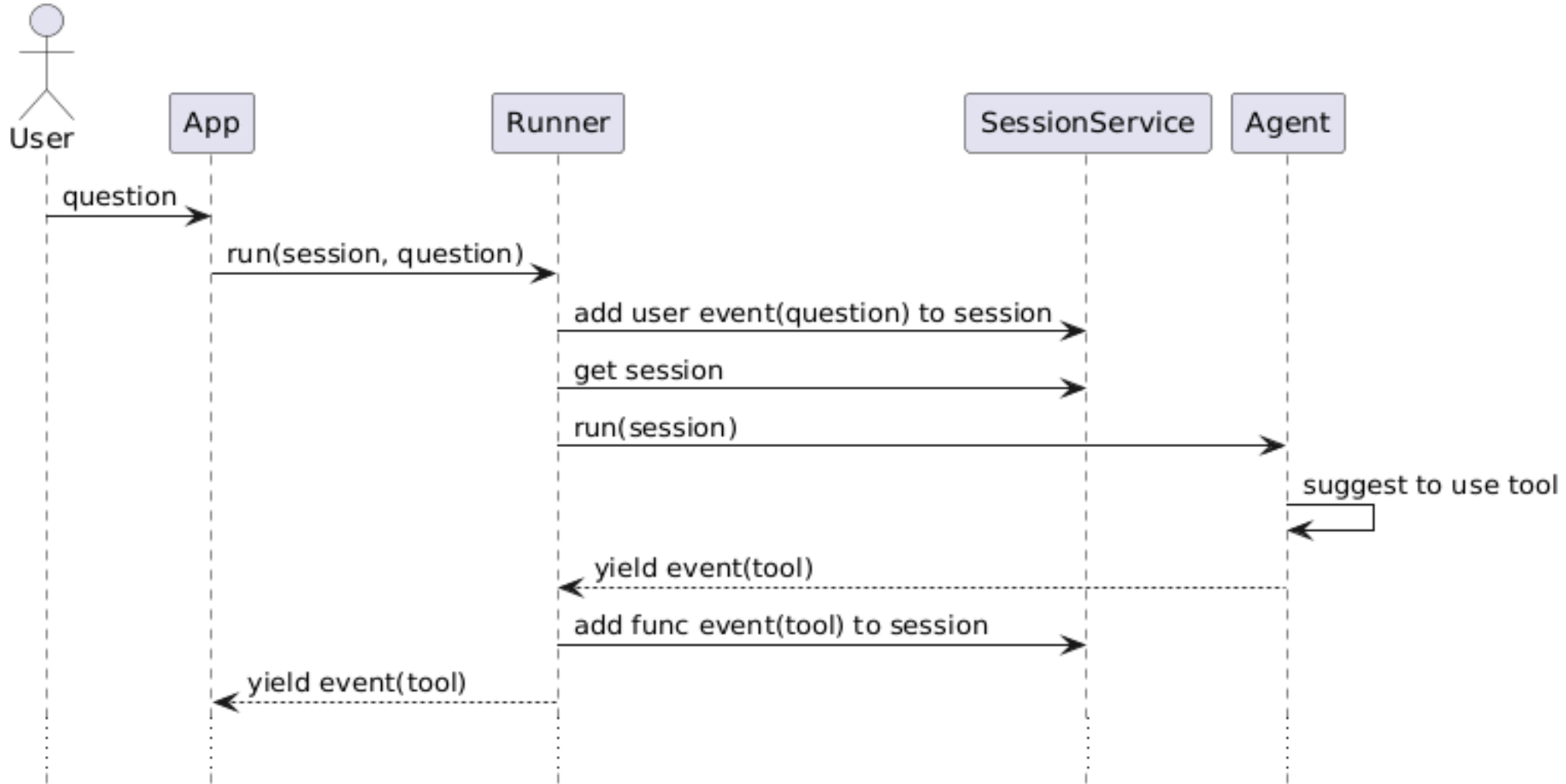
```
try:  
    g = count()  
    print(g)  
  
    print(next(g))  
    print(next(g))  
    print(next(g))  
  
except StopIteration:  
    print("Generator exhausted")
```

```
def count(max):  
    for i in range(max):  
        yield i  
  
print(list(count(3)))  
  
for n in count(5):  
    print(n)
```

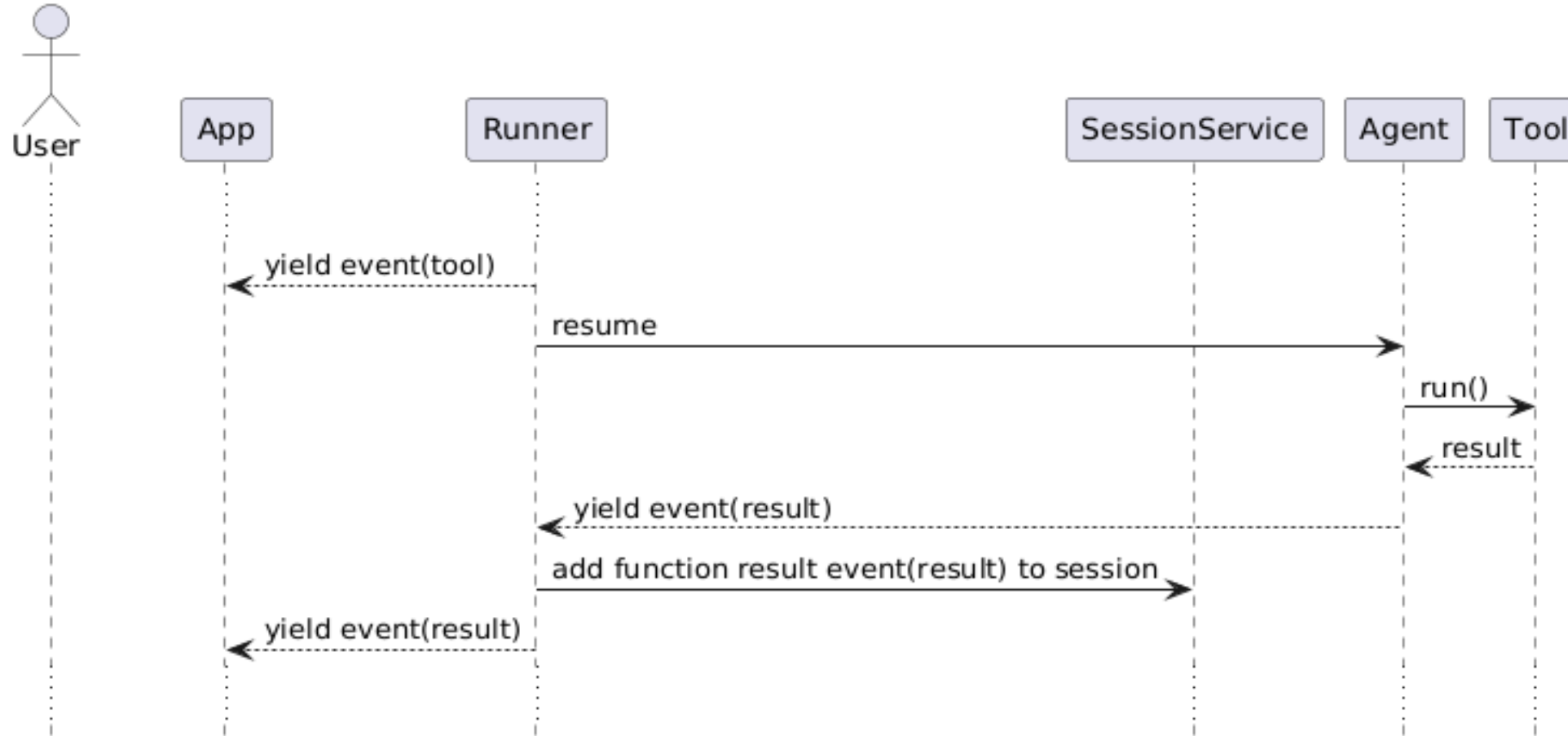
Runner – Agent



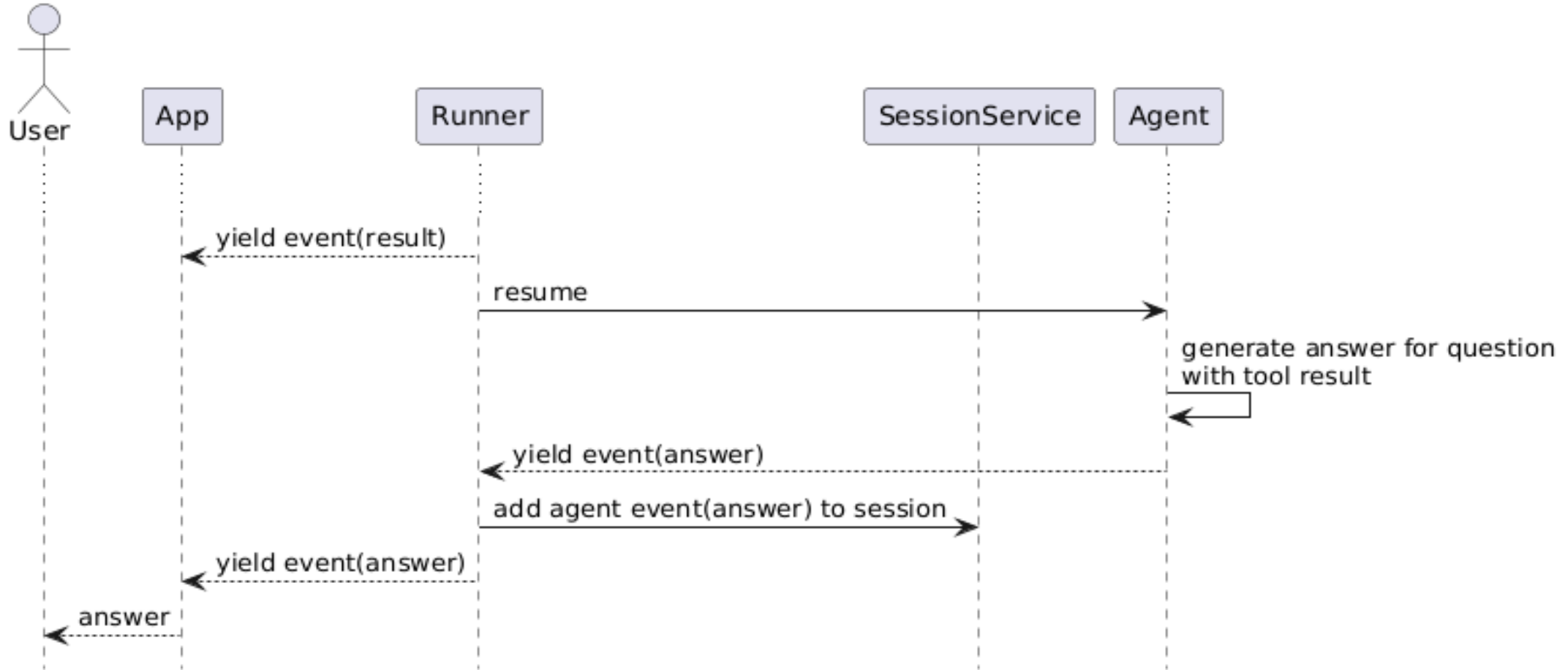
Runner – Agent with Tool

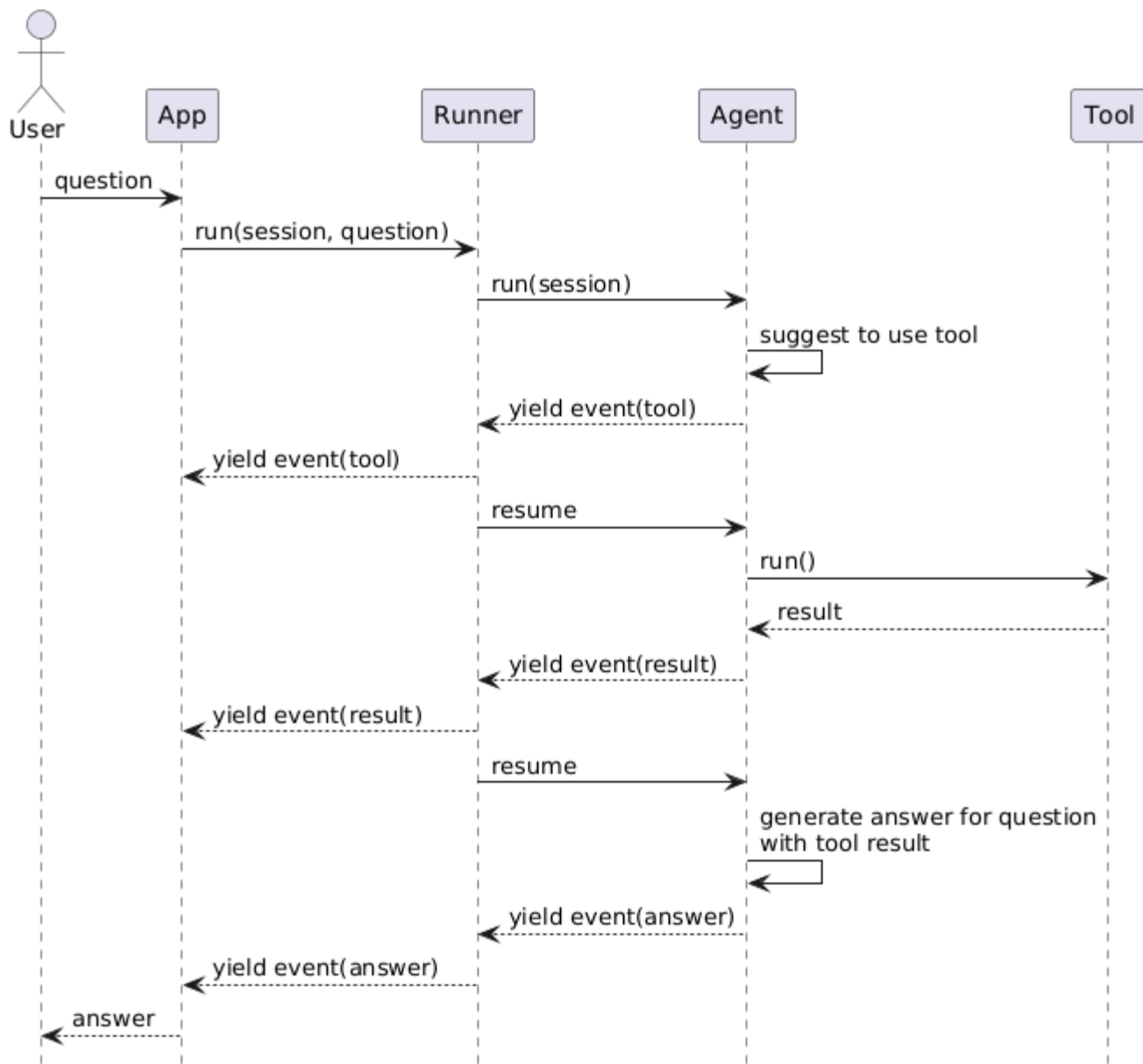


Runner – Agent with Tool



Runner – Agent with Tool





```

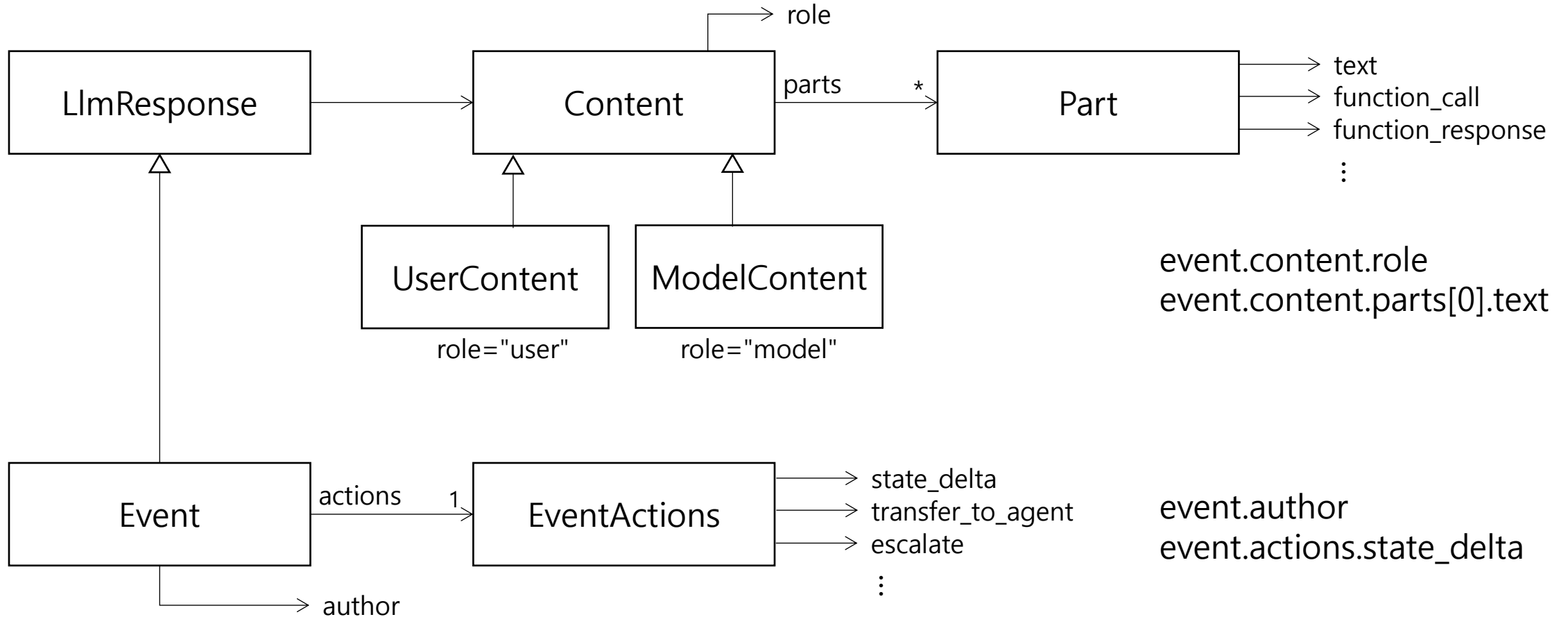
for event in runner.run(user_id=session.user_id,
                        session_id=session.id,
                        new_message=UserContent(user_input)):
    if event.is_final_response():
        print(f"Agent: {event.content.parts[0].text}")
  
```

```

events = list(runner.run(user_id=session.user_id,
                        session_id=session.id,
                        new_message=UserContent(user_input)))

print(f"Agent: {events[-1].content.parts[0].text}")
  
```

Event Data Structure



T5. country_agent

- {country} 변수로 설정된 국가에 관한 질문에 답하는 Country 에이전트를 만들어 보자.

```
country_agent/agent.py
```

```
from google.adk.agents import Agent

root_agent = Agent(
    name="country_agent",
    model="gemini-2.0-flash",
    instruction="사용자의 {country}에 관한 질문에 답하세요.",
```

T5. country_agent

The screenshot displays the Agent Development Kit (ADK) interface. On the left, a sidebar shows the 'country_agent' dropdown and tabs for Trace, Events, State, Artifacts, Sessions, and Eval. The main area shows a session ID '50eccf40-1783-4b87-a294-99b37592f566' and a 'Token Streaming' toggle. A modal dialog titled 'Update state' is open, showing a JSON object with the key 'country' and the value 'KR'. The dialog has 'Cancel' and 'Save' buttons. At the bottom, there is a 'Type a Message...' input field with icons for attachments, a menu, voice, and video.

Agent Development Kit

country_agent

Trace Events State Artifacts Sessions Eval

Invocations

SESSION ID 50eccf40-1783-4b87-a294-99b37592f566

Token Streaming | + New Session

Update state

```
1 {  
2   "country": "KR"  
3 }
```

Line: 1 Column: 1

Cancel Save

Type a Message...

📎 ⋮ 🎤 📹

T5. country_agent

The screenshot shows the Agent Development Kit (ADK) interface. The top bar displays the session ID: 50eccf40-1783-4b87-a294-99b37592f566. The left sidebar is titled "Agent Development Kit" and shows the "country_agent" selected. Below the sidebar, the "Invocations" section displays a list of events:

- Invocation ID: e-0bf9ff9d-055f-40bb-99b0-9c7be4161bb0
- invocation (1746.16ms)
- agent_run [country_age- (1745.83ms)
- call_llm (1743.70ms)

The main chat area shows a message from the agent: "중국의 인구는 2021년 기준으로 약 14억 4천 8백만 명입니다." (The population of China is approximately 144.8 million as of 2021). The input field at the bottom contains the text "인구는 얼마나 되지?" (How many people are there?).

T5. country_agent

The screenshot displays the Agent Development Kit (ADK) interface. On the left, a sidebar shows the 'country_agent' selected from a dropdown menu. Below the menu are tabs for 'Trace', 'Events', 'State', 'Artifacts', 'Sessions', and 'Eval'. The 'State' tab is active, showing a variable 'country' with the value '중국' (China). The main area on the right shows a chat session with a session ID of '50eccf40-1783-4b87-a294-99b37592f566'. A toggle for 'Token Streaming' is visible. A blue button asks '인구는 얼마나 되지?' (How many people are there?). A message bubble from the agent states: '중국의 인구는 2021년 기준으로 약 14억 4천 8백만 명입니다.' (China's population is approximately 1.448 billion as of 2021). At the bottom, there is a text input field 'Type a Message...' with icons for attachments, a menu, voice, and video.

T5. country_agent

Event 1 of 1

Event

Request

Response

```
model: "gemini-2.0-flash"
config:
  http_options:
    headers:
      x-goog-api-client: "google-adk/1.9.0 gl-python/3.13.6"
      user-agent: "google-adk/1.9.0 gl-python/3.13.6"
  system_instruction: "사용자의 중국에 관한 질문에 답하세요. You are an agent. Your internal name is 'country_agent'. The description about you is '국가에 대한 정보를 제공하는 에이전트입니다.'"
  contents:
    0:
      parts:
        0:
          text: "인구는 얼마나 되지?"
      role: "user"
```

SESSION ID 50eccf40-1783-4b87-a294-99b37592f566

Token Streaming | + New Session

인구는 얼마나 되지?

중국의 인구는 2021년 기준으로 약 14억 4천 8백만 명입니다.

Type a Message...





T5. country_agent

The screenshot displays the Agent Development Kit (ADK) interface. On the left, the 'country_agent' is selected, and the 'Invocations' tab shows the query '인구는 얼마나 되지?'. The main panel shows a session with ID '50eccf40-1783-4b87-a294-99b37592f566'. A modal dialog titled 'Update state' is open, showing a JSON object:

```
1 {  
2   "country": "일본"  
3 }
```

 with a status bar indicating 'Line: 2 Column: 17'. The dialog has 'Cancel' and 'Save' buttons. At the bottom, there is a 'Type a Message...' input field with icons for attachments, a menu, voice, and video.

T5. country_agent

Agent Development Kit

country_agent

TraceEventsStateArtifactsSessionsEval

Invocations

인구는 얼마나 되지?

수도는?

Invocation ID:
e-0d25528f-b1ef-42dc-8c4f-4881675d5537

invocation

1547.77ms

agent_run [country_age...

1547.41ms

call_llm

1545.52ms

SESSION ID 50eccf40-1783-4b87-a294-99b37592f566

Token Streaming | + New Session

인구는 얼마나 되지?

수도는?

중국의 인구는 2021년 기준으로 약 14억 4천 8백만 명입니다.

일본의 수도는 도쿄입니다.

Type a Message...

T5. country_agent

Event 2 of 2

×

EventRequestResponse

```
model: "gemini-2.0-flash"
config:
  http_options:
    headers:
      x-goog-api-client: "google-adk/1.9.0 gl-python/3.13.6"
      user-agent: "google-adk/1.9.0 gl-python/3.13.6"
  system_instruction: "사용자의 일본에 관한 질문에 답하세요. You are an agent. Your internal name is 'country_agent'. The description about you is '국가에 대한 정보를 제공하는 에이전트입니다.'"
  contents:
    0:
      parts:
        0:
          text: "인구는 얼마나 되지?"
          role: "user"
      1:
        parts:
          0:
            text: "중국의 인구는 2021년 기준으로 약 14억 4천 8백만 명입니다."
            role: "model"
      2:
        parts:
          0:
            text: "수도는?"
            role: "user"
```

SESSION ID 50eccf40-1783-4b87-a294-99b37592f566

Token Streaming | + New Session

인구는 얼마나 되지?

중국의 인구는 2021년 기준으로 약 14억 4천 8백만 명입니다.

수도는?

일본의 수도는 도쿄입니다.

Type a Message...



T5. country_agent - runner

country_agent/agent.py

```
async def main():
    runner = InMemoryRunner(agent=root_agent, app_name=root_agent.name)

    print("국가에 정보를 제공하는 에이전트입니다. 궁금한 국가를 입력하세요.")
    print("'exit' 입력 시 종료됩니다.")
    while True:
        user_input = input("Country: ")
        if user_input.lower() == "exit": break

        session = await runner.session_service.create_session(
            app_name=runner.app_name, user_id="user1", state={"country": user_input})
```

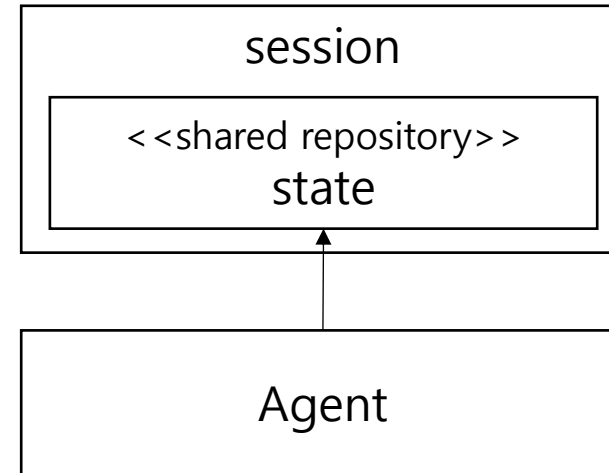
T5. country_agent - runner

country_agent/agent.py

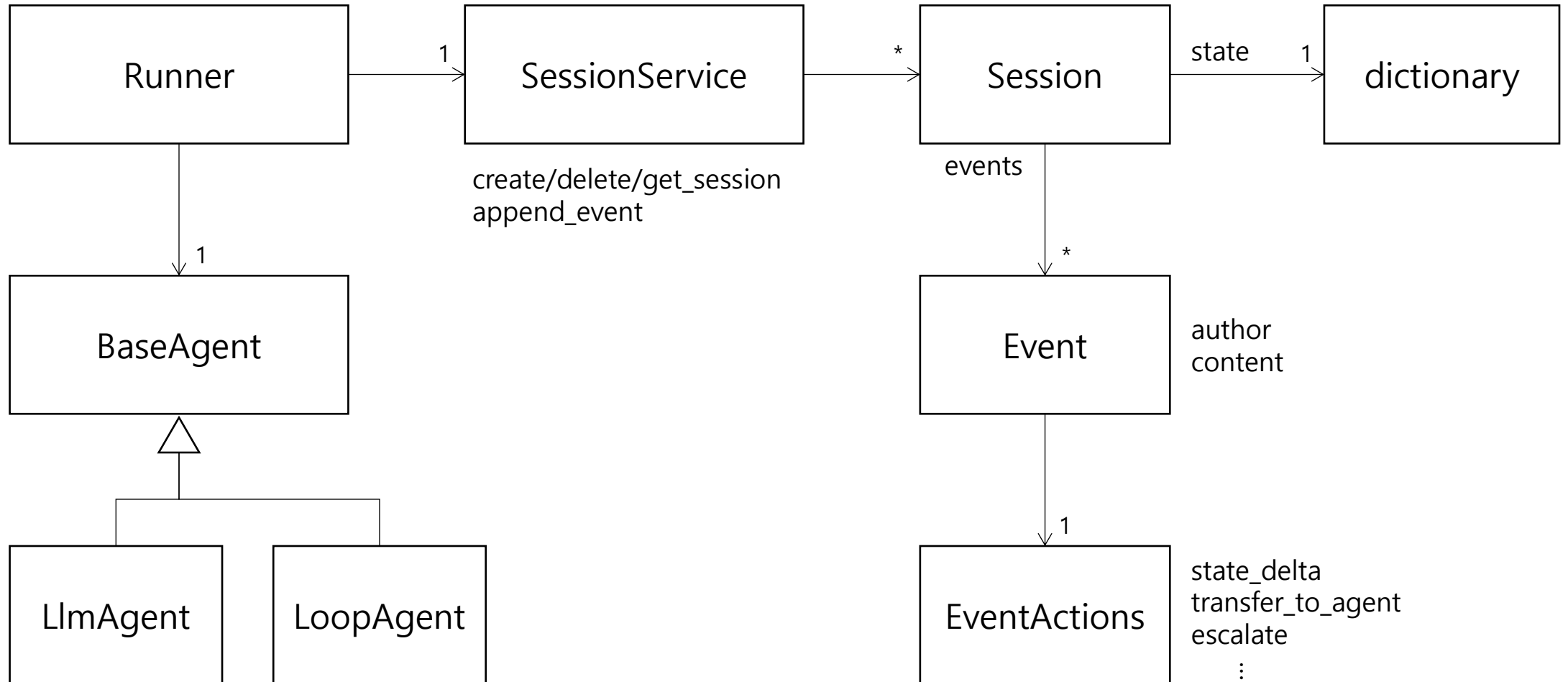
```
print(f"{user_input}에 대해 궁금한 것을 질문하세요. 'exit' 입력 시 종료됩니다.")
while True:
    user_input = input("User: ")
    if user_input.lower() == "exit": break

    for event in runner.run(user_id=session.user_id, session_id=session.id,
                           new_message=UserContent(user_input)
    ):
        if event.is_final_response():
            print(f"Agent: {event.content.parts[0].text}")
```

LangChain vs. Google ADK



Runner, Session Service, Session, ...



LLMaBoK

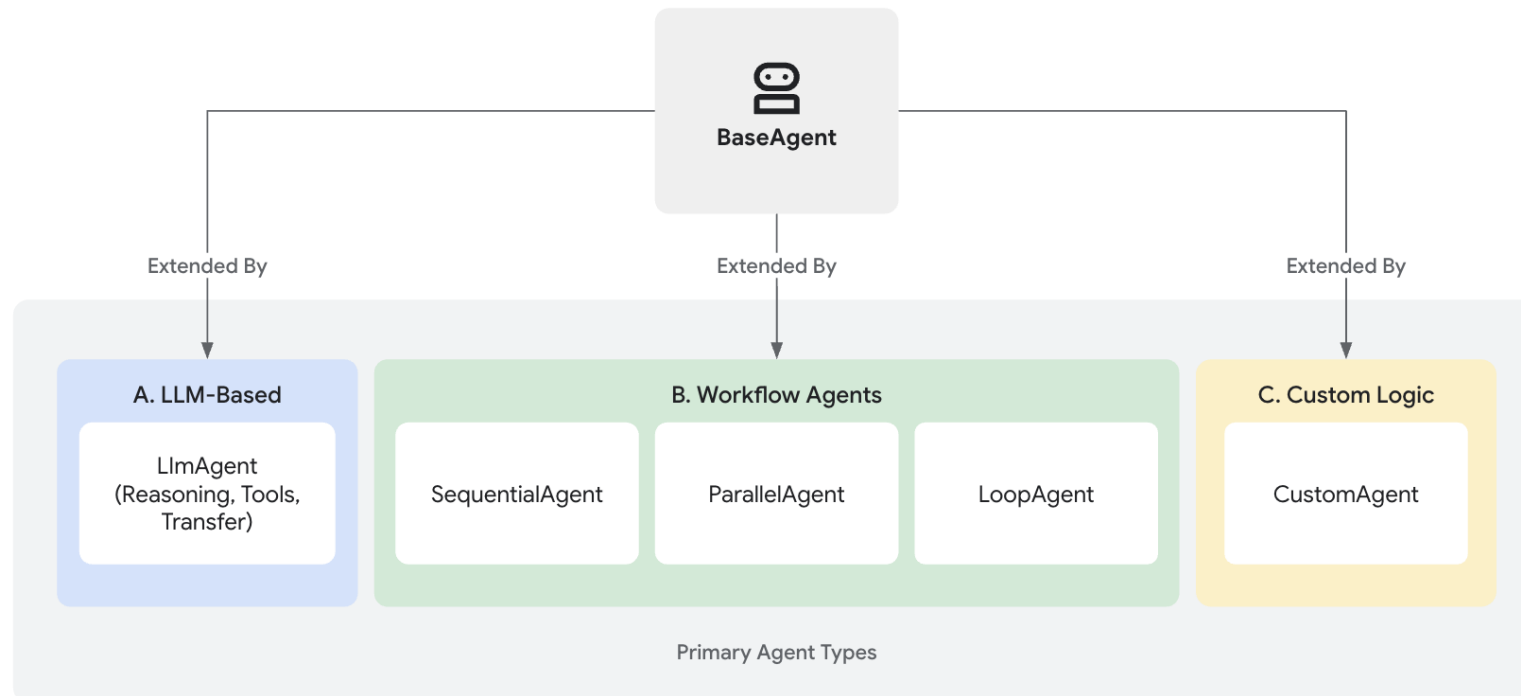
Agent Categories

조 용 진

(drajin.cho@bosornd.com)

Agent

- Agent is a self-contained execution unit designed to act autonomously to achieve specific goals. Agents can perform tasks, interact with users, utilize external tools, and coordinate with other agents.



LlmAgent == Agent

- name: Every agent needs a unique string identifier.
- model: Specify the underlying LLM that will power this agent's reasoning.
- description: Provide a concise summary of the agent's capabilities.
This description is primarily used by other LLM agents to determine if they should route a task to this agent.
- instruction: Shape LLM's behavior.
 - Core task or goal.
 - Personality or persona.
 - Constraints on its behavior.
 - How and when to use its tools.
 - The desired format for its output.

LlmAgent == Agent

- Tips for Effective Instructions:
 - **Be Clear and Specific**: Avoid ambiguity. Clearly state the desired actions and outcomes.
 - **Use Markdown**: Improve readability for complex instructions using headings, lists, etc.
 - **Provide Examples (Few-Shot)**: For complex tasks or specific output formats, include examples directly in the instruction.
 - **Guide Tool Use**: Don't just list tools; explain when and why the agent should use them.
- The instruction is a string template;
you can use the {var} syntax to insert dynamic values into the instruction.

Structured Data

- `input_schema`: a schema representing the expected input structure.
- `output_schema`: a schema representing the desired output structure.
- `output_key`: a string key to store the agent's final response in the session's state dictionary under this key.

T6. country_agent with structured data

- Country 에이전트에 입/출력 포맷을 지정해 보자.

country_agent/agent.py

```
from pydantic import BaseModel, Field

class CountryInput(BaseModel):
    country: str = Field(description="The country to get information about.")

class CapitalInfoOutput(BaseModel):
    country: str = Field(description="The country to get information about.")
    capital: str = Field(description="The capital city of the country.")
```

T6. country_agent with structured data

country_agent/agent.py

```
import json
from google.adk.agents import Agent
root_agent = Agent(
    name="country_agent", model="gemini-2.0-flash",
    input_schema=CountryInput, output_schema=CapitalInfoOutput,
    output_key="output",
    description="국가의 수도 정보를 제공하는 에이전트입니다.",
    instruction=f"""사용자는 {"{"country": "한국"}"}와 같은 JSON 형식으로 국가 이름을
        제공합니다. 정확히 다음 스키마에 맞는 JSON 객체로만 응답하십시오:
        {json.dumps(CapitalInfoOutput.model_json_schema(), indent=2)}"""
)
```

T7. code_agent

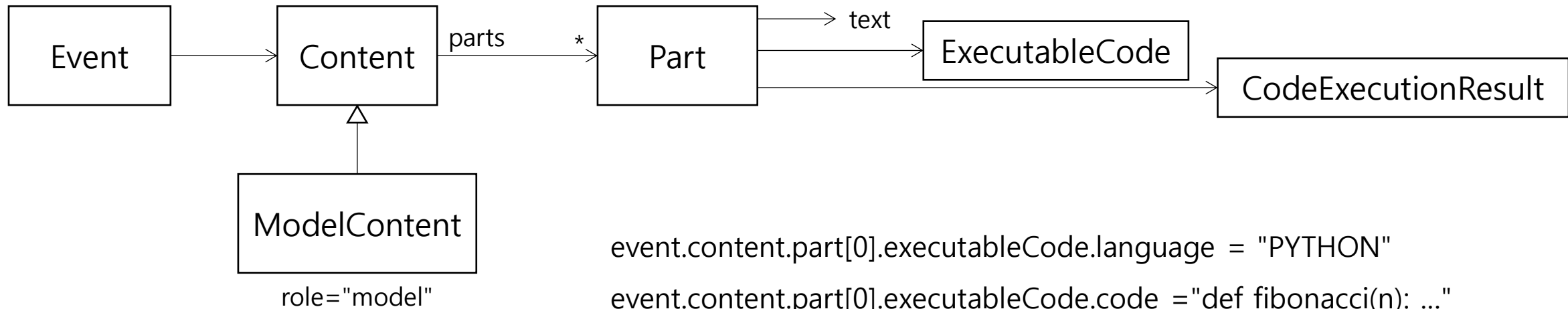
- BuiltInCodeExecutor를 사용하는 코딩 에이전트를 만들어 보자.

code_agent/agent.py

```
from google.adk.agents import Agent
from google.adk.code_executors import BuiltInCodeExecutor

root_agent = Agent(
    name="code_agent", model="gemini-2.0-flash",
    code_executor=BuiltInCodeExecutor(),
    instruction="""주어진 문제를 해결하기 위해 파이썬 코드를 작성하고
                  실행한 결과를 출력하세요.""",
    include_contents="none"      # 이전 대화의 내용은 포함하지 않음
)
```

T7. code_agent



```
event.content.part[0].executableCode.language = "PYTHON"
```

```
event.content.part[0].executableCode.code = "def fibonacci(n): ..."
```

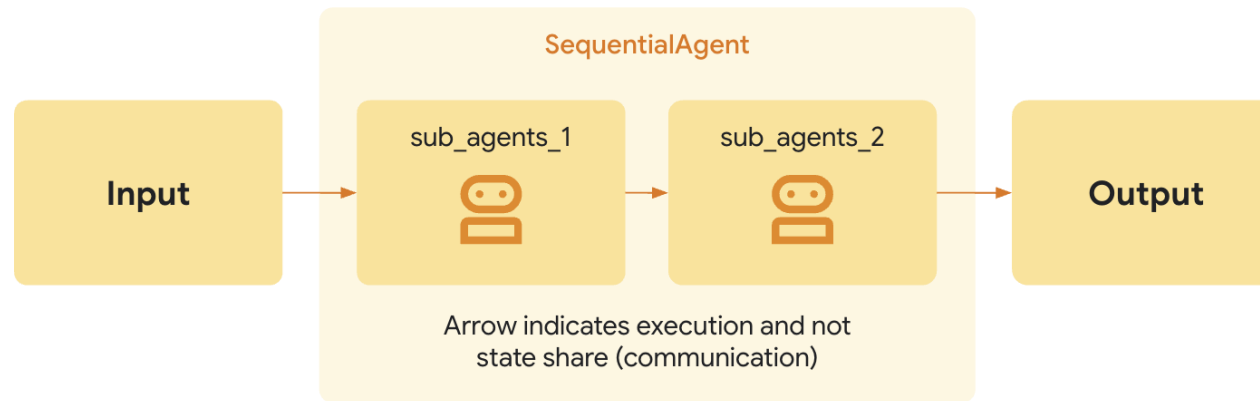
```
event.content.part[1].codeExecutionResult.outcome = OUTCOME_OK
```

```
event.content.part[1].codeExecutionResult.output = "55"
```

```
event.content.part[2].text = "55"
```

Workflow Agents

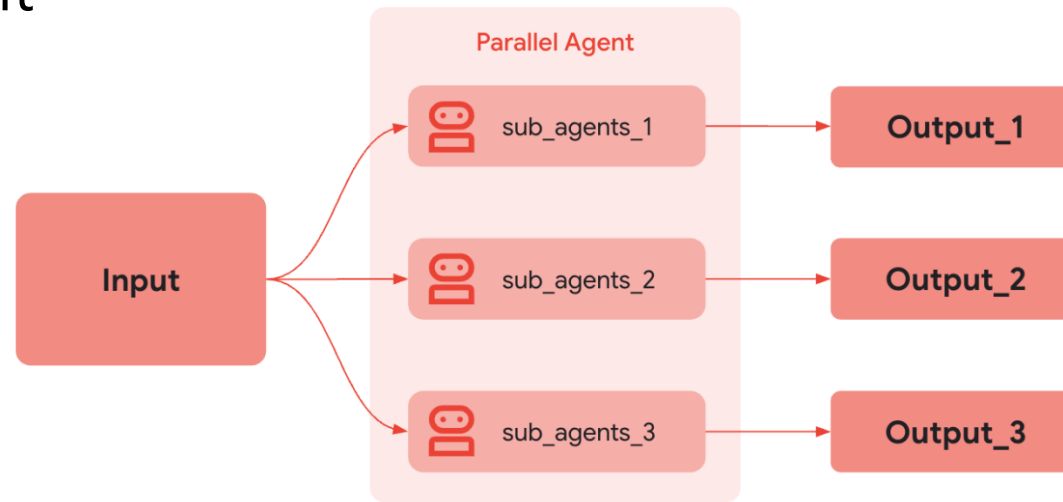
- SequentialAgent



```
from google.adk.agents import SequentialAgent
root_agent = SequentialAgent(
    name="sequential agent",
    sub_agents=[sub_agents_1, sub_agents_2],
)
```

Workflow Agents

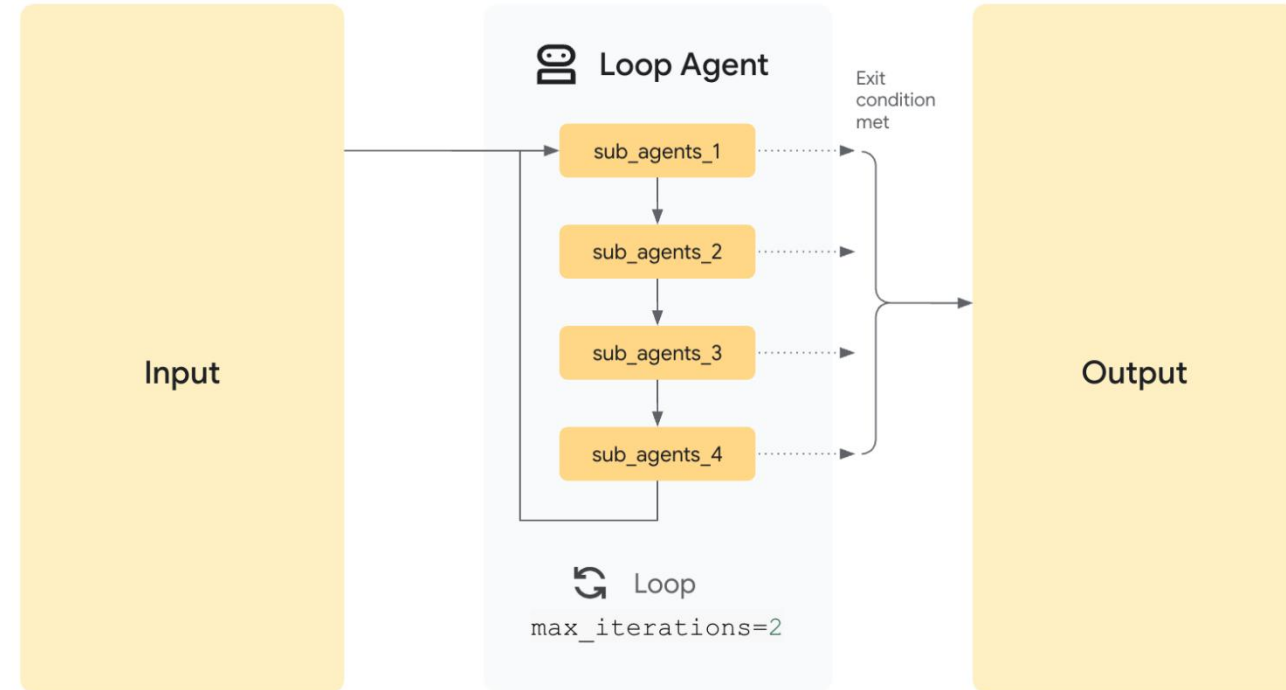
- ParallelAgent



```
from google.adk.agents import ParallelAgent
root_agent = ParallelAgent(
    name="parallel agent",
    sub_agents=[sub_agents_1, sub_agents_2, sub_agents_3],
)
```

Workflow Agents

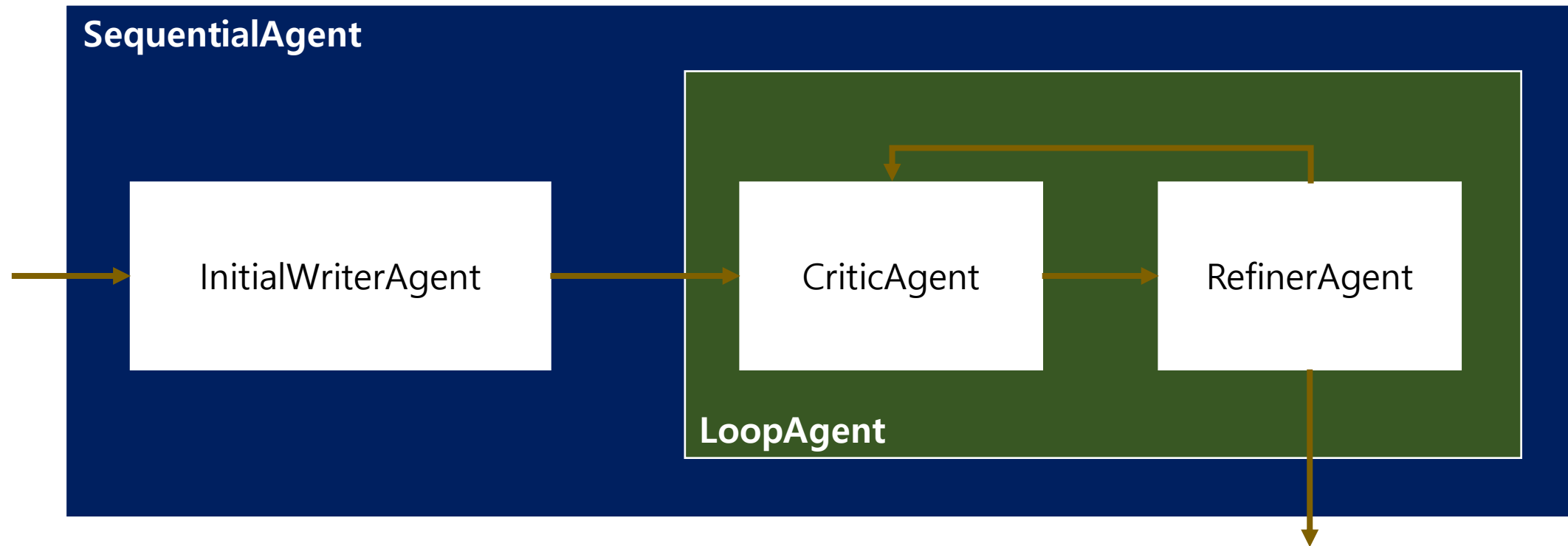
- LoopAgent



```
from google.adk.agents import LoopAgent
root_agent = LoopAgent(
    name="loop agent", max_iterations=2,
    sub_agents=[sub_agents_1, sub_agents_2, sub_agents_3, sub_agents_4],
)
```


T8. story_agent

- 주어진 주제로 이야기를 생성하는 Story 에이전트를 만들어 보자.
 - 비평을 근거로 이야기를 개선한다.



T8. story_agent

story_agent/sub_agents.py

```
from google.adk.agents import Agent
GEMINI_MODEL = "gemini-2.0-flash"

initial_writer_agent = Agent(
    name="InitialWriterAgent", model=GEMINI_MODEL,
    instruction="""당신은 창의적인 글쓰기 도우미입니다.
                    요청에 맞는 짧은 이야기(2~4 문장)를 작성하세요.""",
    output_key="story"
)
```

T8. story_agent

story_agent/sub_agents.py

```
critic_agent = Agent(  
    name="CriticAgent", model=GEMINI_MODEL,  
    instruction="""당신은 짧은 이야기 초안을 검토하는 건설적인 비평가입니다.  
    다음 이야기를 검토하고, 개선할 수 있는 명확하고 실행 가능한 방법이 있다면  
    그에 대한 구체적인 제안을 제공하세요.  
    **검토할 이야기:**  
    ...  
    {story}  
    ...
```

T8. story_agent

story_agent/sub_agents.py

****작업 가이드:****

문서의 명확성, 몰입도 및 기본적인 일관성을 검토합니다.

독자의 몰입도를 높이기 위한 명확한 방안이 있으면 1-2가지 제안하세요.

예: "더 강력한 도입 문장이 필요하다", "캐릭터의 목표를 명확히 하라".

구체적이고 간결한 제안을 제공하세요.

더 이상 수정할 사항이 없다면 ***정확히*** "No major issues found."라는 문구를
정확히 출력하세요.

비평 내용만 출력하고 추가 설명은 하지 마세요.

""",

output_key="criticism",

)

T8. story_agent

story_agent/sub_agents.py

```
from google.adk.tools import exit_loop

refiner_agent = Agent(
    name="RefinerAgent", model=GEMINI_MODEL,
    tools=[exit_loop],
    output_key="story",
    instruction="""당신은 피드백을 기반으로 이야기를 다듬거나 프로세스를 종료하는
창의적인 글쓰기 도우미입니다.

**이야기:**

{story}

**비평/제안:**

{criticism}
```

T8. story_agent

story_agent/sub_agents.py

****작업 가이드:****

'비평/제안'을 분석합니다. 비평이 ***정확히*** "No major issues found."라면,
'exit_loop' 함수를 호출하세요. 이 경우 추가 텍스트를 출력하지 마세요.

그렇지 않으면, 비평에서 제안된 개선 사항을 신중하게 적용하여 이야기를 개선하세요.
개선된 문서만 출력하고, 'exit_loop' 함수를 호출하지 마세요.

"""

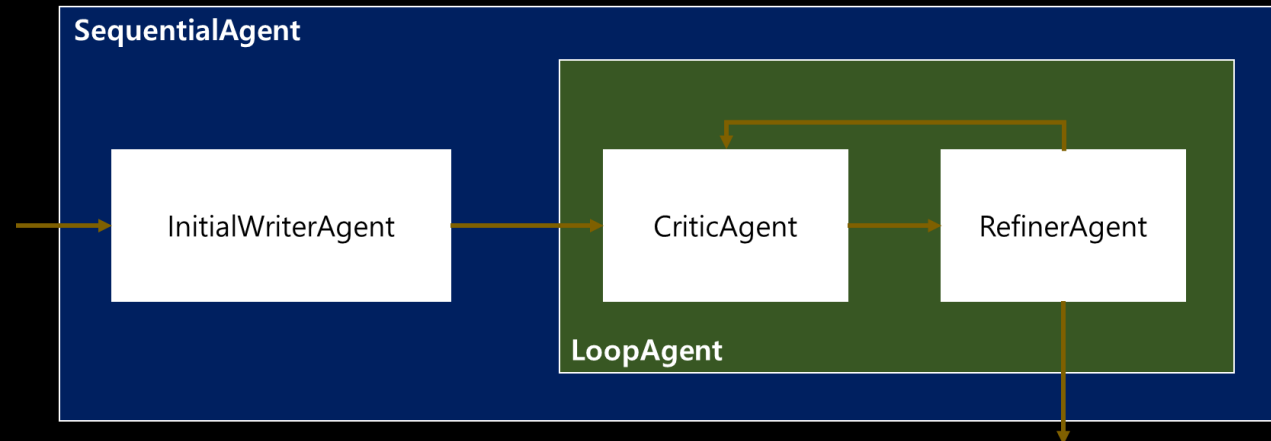
)

T8. story_agent

story_agent/agent.py

```
from google.adk.agents import SequentialAgent, LoopAgent
from .sub_agents import initial_writer_agent, critic_agent, refiner_agent

root_agent = SequentialAgent(
    name="WritingPipeline",
    ...
)
```



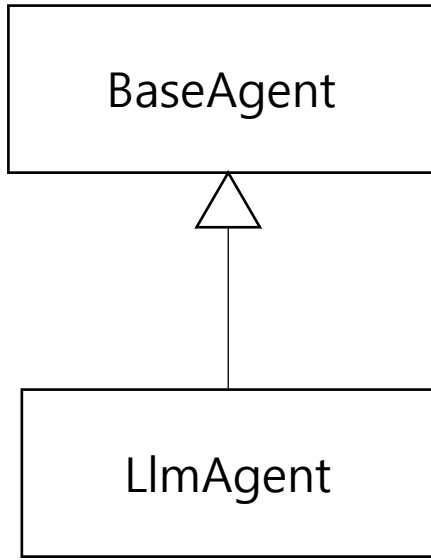
LLMaBoK

Custom Agent

조 용 진

(drajin.cho@bosornd.com)

Custom Agent



```
@final
async def run_async(self, ctx: InvocationContext) -> AsyncGenerator[Event, None]:
    ...
    async for event in self._run_async_impl(ctx):
        yield event
    ...
```

```
@override
async def _run_async_impl(self, ctx: InvocationContext) -> AsyncGenerator[Event, None]:
    async for event in self._llm_flow.run_async(ctx):
        self.__maybe_save_output_to_state(event)
        yield event
```

T9. LambdaAgent

- 주어진 함수를 구동하는 함수 에이전트를 만들어 보자.

simple_agents/lambda_agent.py

```
def increase(x: int) -> int:
    return x + 1

root_agent = LambdaAgent(name="increase_agent",
                          func=increase, # func=lambda x: x + 1,
                          input_keys=["number"],
                          output_key="number")

# {"number" : 1} -> increase_agent -> {"number" : 2}
```

T9. LambdaAgent

simple_agents/lambda_agent.py

```
from typing_extensions import override
from typing import Callable, Any, AsyncGenerator, List, Optional
from google.adk.agents import BaseAgent, InvocationContext
from google.adk.events import Event
from google.adk.events.event_actions import EventActions
from google.genai.types import ModelContent

class LambdaAgent(BaseAgent):
    func: Callable[..., Any]
    input_keys: List[str]
    output_key: Optional[str] = None
```

T9. LambdaAgent

simple_agents/lambda_agent.py

```
@override
async def _run_async_impl(
    self, ctx: InvocationContext
) -> AsyncGenerator[Event, None]:
    inputs = [ctx.session.state.get(k) for k in self.input_keys]
    output = self.func(*inputs)

    yield Event(author=self.name, invocation_id=ctx.invocation_id,
                content=ModelContent(str(output)),
                actions=EventActions(state_delta={self.output_key: output}
                                     if self.output_key else {}))
```

T10. JsonInputAgent

- JSON 입력을 state에 반영하는 에이전트를 만들어 보자.

simple_agents/json_input_agent.py

```
class JsonInputAgent(BaseAgent):  
    @override  
    async def _run_async_impl(  
        self, ctx: InvocationContext  
    ) -> AsyncGenerator[Event, None]:  
        text = ctx.user_content.parts[0].text          # user input  
        yield Event(author=self.name, invocation_id=ctx.invocation_id,  
                    content=ctx.user_content,  
                    actions=EventActions(state_delta=json.loads(text)))
```

T11. calc_agent

- 주어진 숫자에 1을 더하고, 제공한 결과를 반환하는 에이전트를 만들어 보자.

simple_agents/agent.py

```
from .json_input_agent import JsonInputAgent
from .lambda_agent import LambdaAgent

:

from google.adk.agents import SequentialAgent
root_agent = SequentialAgent(name="calc_agent",
                             sub_agents=[json_input_agent, increase_agent, square_agent])
```

T12. WhileAgent

- 조건을 만족하는 동안 반복 실행하는 While 에이전트를 만들어 보자.

simple_agents/while_agent.py

```
from google.adk.agents import LoopAgent
loop_agent = LoopAgent(name="loop_agent",
                        max_iterations=5,
                        sub_agents=[increase_agent, sqaure_agent])

from .while_agent import WhileAgent
while_agent = WhileAgent(name="while_agent",
                        condition="number < 100",
                        sub_agents=[increase_agent, sqaure_agent])
```

T12. WhileAgent

simple_agents/while_agent.py

```
@override
async def _run_async_impl(self, ctx: InvocationContext)
    -> AsyncGenerator[Event, None]:

    running = True
    while running:
        for agent in self.sub_agents:
            running = eval(self.condition, {}, ctx.session.state)
            if not running: break

        async for event in agent.run_async(ctx):
            yield event
```


T13. fibonacci_agent

- WhileAgent를 사용해서 피보나치 값을 반환하는 에이전트를 만들어 보자.

simple_agents/agent.py

```
def fibonacci(f: list[int]) -> list[int]:  
    """  
    Generate the next Fibonacci number in the sequence.  
    """  
    if f is None: f = [0, 1]  
    f.append(f[-1] + f[-2])  
    return f  
  
root_agent = ...
```

T14. story_agent – custom workflow

- Custom 워크플로우를 가지는 Story 에이전트를 만들어 보자.

story_agents/sub_agents.py

```
from typing_extensions import override
from typing import AsyncGenerator, Optional
from google.adk.agents import Agent, BaseAgent, InvocationContext
from google.adk.events import Event

class StoryAgent(BaseAgent):
    generator: Agent
    critic: Agent
    reviser: Agent
```

T14. story_agent – custom workflow

story_agents/sub_agents.py

```
max_iterations: Optional[int] = None
@override
async def _run_async_impl(self, ctx: InvocationContext
                           ) -> AsyncGenerator[Event, None]:
    async for event in self.generator.run_async(ctx):
        yield event

    running = True
    times = 0
    while running and (not self.max_iterations or times < self.max_iterations):
        times += 1
```

T14. story_agent – custom workflow

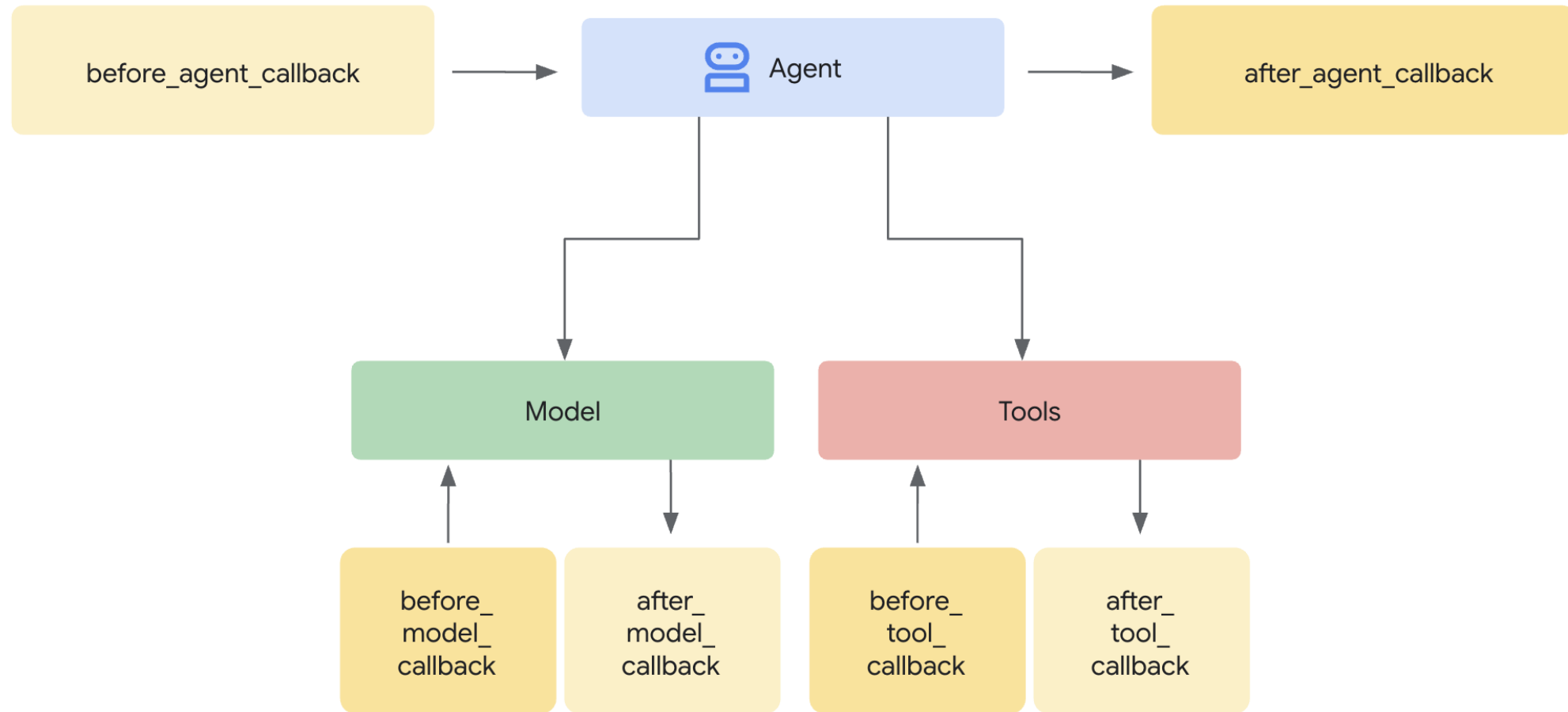
story_agents/sub_agents.py

```
# Generate criticism
async for event in self.critic.run_async(ctx):
    yield event

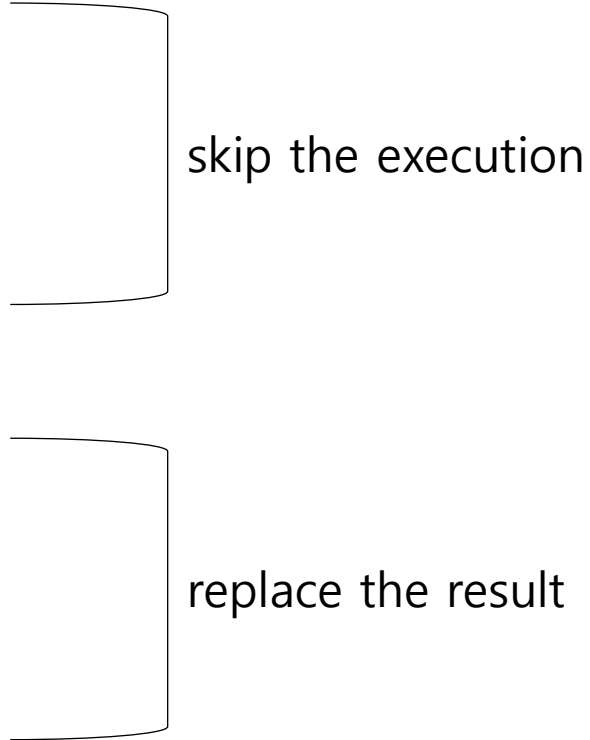
if ctx.session.state.get("criticism") == "No major issues found.":
    # If no major issues found, exit the loop
    break

# Generate revision
async for event in self.reviser.run_async(ctx):
    yield event
```

callbacks



callbacks

- returns None → default behavior
 - returns <Specific Object> → override default behavior
 - before_agent_callback → types.Content
 - before_model_callback → LlmResponse
 - before_tool_callback → dict
 -
 - after_agent_callback → types.Content
 - after_model_callback → LlmResponse
 - after_tool_callback → dict
- 
- skip the execution
- replace the result

callback patterns

- Guardrails & Policy Enforcement
 - Intercept requests before they reach the LLM or tools to enforce rules.
- Dynamic State Management
 - Read from and write to session state within callbacks to make agent behavior context-aware and pass data between steps.
- Logging and Monitoring
 - Add detailed logging at specific lifecycle points for observability and debugging.
- Caching
 - Avoid redundant LLM calls or tool executions by caching results.

callback patterns

- Request/Response Modification
 - Alter data just before it's sent to the LLM/tool or just after it's received.
- Conditional Skipping of Steps
 - Prevent standard operations based on certain conditions.
- Tool-Specific Actions (Authentication & Summarization Control)
 - Handle actions specific to the tool lifecycle, primarily authentication and controlling LLM summarization of tool results.
- Artifact Handling
 - Save or load session-related files or large data blobs during the agent lifecycle.

LLMaBoK

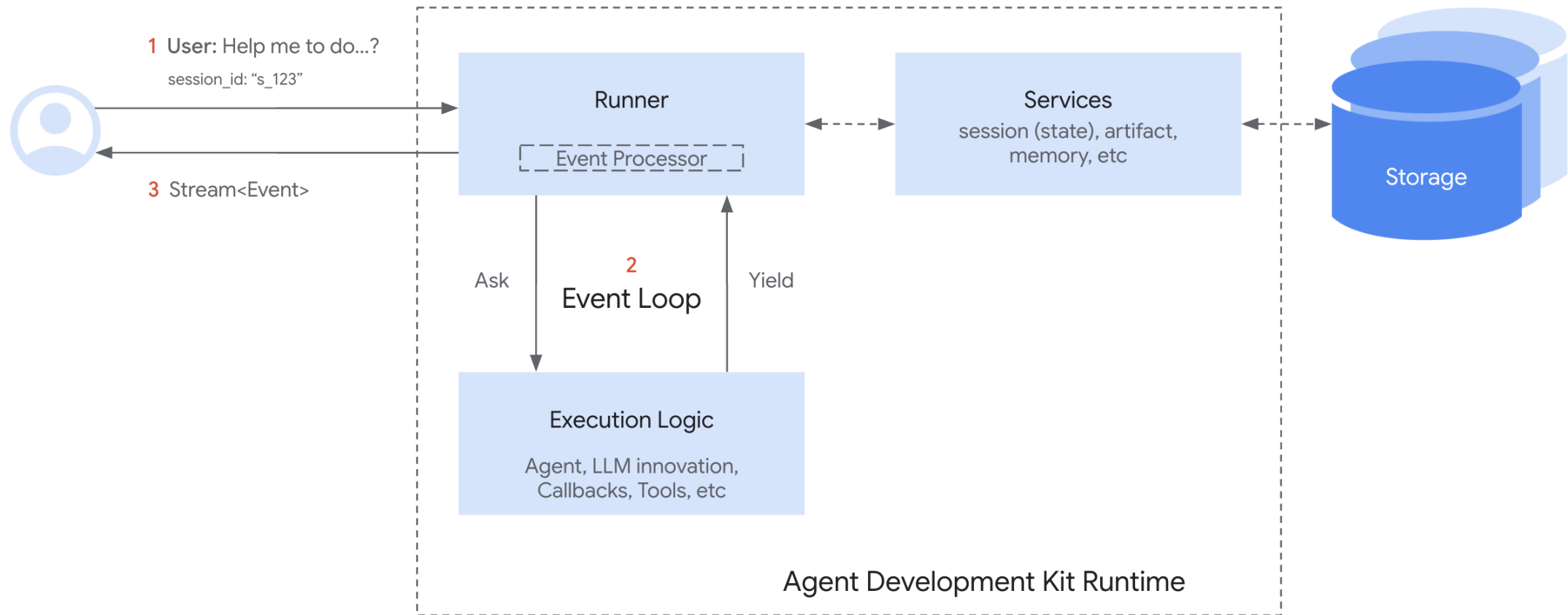
Session, Memory, Artifact

조 용 진

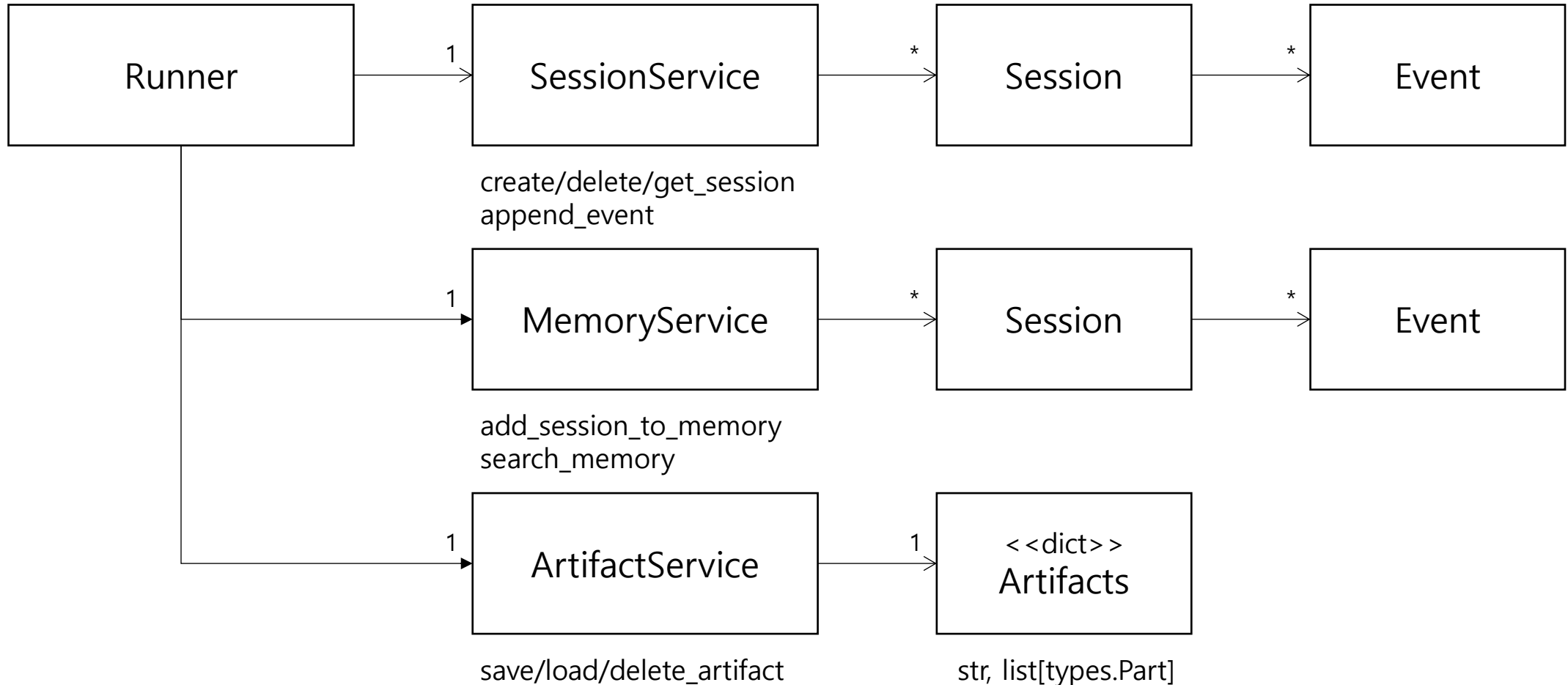
(drajin.cho@bosornd.com)



ADK Runtime



Session, Memory, Artifact



T15. memory_agent

- 이전 대화를 근거로 답하는 메모리 에이전트를 만들어 보자.
 - load_memory 도구를 사용한다.

memory_agent/agent.py

```
from google.adk.agents import Agent
from google.adk.tools import load_memory

root_agent = Agent(
    name="MemoryRecallAgent", model="gemini-2.0-flash",
    instruction="""사용자와 일상적인 대화를 나누는 에이전트입니다. 질문에 답하기 위해서
                    이전 대화 내용이 필요하다면 'load_memory' 도구를 사용합니다.""",
    tools=[load_memory] )
```

T15. memory_agent

memory_agent/agent.py

```
async def main():
    runner = InMemoryRunner(agent=root_agent, app_name=root_agent.name)
    session = await runner.session_service.create_session(app_name=runner.app_name,
                                                         user_id="user1")

    print("일상적인 대화를 나누는 에이전트입니다.")
    print("'new' 입력 시 새로운 대화가 시작됩니다. 'exit' 입력 시 종료됩니다.")
    while True:
        user_input = input("User: ")
        if user_input.lower() == "exit": break
```

T15. memory_agent

memory_agent/agent.py

```
elif user_input.lower() == "new":  
    session = await runner.session_service.get_session(  
        app_name=session.app_name, user_id=session.user_id,  
        session_id=session.id)  
  
    # 대화 세션 저장  
    await runner.memory_service.add_session_to_memory(session)  
  
    # 신규 대화 세션 생성  
    session = await runner.session_service.create_session(  
        app_name=session.app_name, user_id=session.user_id)
```

T15. memory_agent

memory_agent/agent.py

```
else:
    for event in runner.run(user_id=session.user_id, session_id=session.id,
                           new_message=UserContent(user_input)
    ):
        if event.is_final_response():
            print(f"Agent: {event.content.parts[0].text}")
```

T16. image_agent using load_artifact

- 업로드된 이미지를 근거로 답하는 이미지 에이전트를 만들어 보자.
 - load_artifacts 도구를 사용한다.

image_agent/agent.py

```
from google.adk.agents import Agent
from google.adk.tools import load_artifacts

root_agent = Agent(
    name="chat_agent", model="gemini-2.0-flash",
    instruction="""사용자의 질문에 답하세요. 사용자가 요청한 파일을 불러오려면,
        'load_artifacts' 도구를 사용하세요.""",
    tools=[load_artifacts])
```


T16. image_agent using load_artifact

image_agent/agent.py

```
async def main():
    runner = InMemoryRunner(agent=root_agent, app_name=root_agent.name)
    session = await runner.session_service.create_session(app_name=runner.app_name,
                                                         user_id="user1")

    with open("image.jpg", "rb") as f:
        image_data = f.read()

    image = Part.from_bytes(data=image_data, mime_type="image/jpeg")
    await runner.artifact_service.save_artifact(app_name=session.app_name,
                                                user_id=session.user_id, session_id=session.id,
                                                filename="image.jpg", artifact=image)
```

T16. image_agent using load_artifact

image_agent/agent.py

```
for event in runner.run(
    user_id=session.user_id, session_id=session.id,
    new_message=UserContent("image.jpg 파일에 대해 설명해줘."),
):
    if event.is_final_response():
        print(f"Agent: {event.content.parts[0].text}")
```

LLMaBoK

Tool

조 용 진

(drajin.cho@bosornd.com)

T17. date_agent

- 날짜와 요일에 대해 답하는 Date 에이전트를 만들어 보자.
 - 오늘의 날짜는 get_today 도구를 사용한다.

date_agent/agent.py

```
from google.adk.agents import Agent

root_agent = Agent(
    name="date_agent",
    model="gemini-2.0-flash",
    instruction="""사용자의 날짜와 요일에 관한 질문에 답하세요.
                    오늘 날짜와 요일은 'get_today' 도구를 사용해서 확인할 수 있습니다.""",
    tools=[get_today])
```

T17. date_agent

date_agent/agent.py

```
from datetime import datetime

def get_today() -> str:
    """
    오늘의 날짜와 요일을 반환합니다.
    """
    return datetime.now().strftime("%Y-%m-%d %A")
```

T17. date_agent

date_agent/agent.py

```
root_agent = Agent(  
    name="date_agent",  
    model="gemini-2.0-flash",  
    instruction=f"""사용자의 날짜와 요일에 관한 질문에 답하세요.  
        오늘 날짜와 요일은 {get_today()}입니다.""",  
)
```

Define effective tool functions

- Function Name:
 - Use descriptive, verb-noun based names that clearly indicate the action (e.g., get_weather, searchDocuments, schedule_meeting).
 - Avoid generic names like run, process, handle_data, or overly ambiguous names like doStuff.
 - The LLM uses the function name as a primary identifier during tool selection

Define effective tool functions

- Parameters (Arguments):
 - Use clear and descriptive names (e.g., query instead of q).
 - Provide type hints in Python for all parameters (e.g., query: str).
 - Ensure all parameter types are JSON serializable. Avoid complex custom class instances as direct parameters unless they have a clear JSON representation.
 - Do not set default values for parameters. Default values are not reliably supported or used by the underlying models during function call generation. All necessary information should be derived by the LLM from the context or explicitly requested if missing.

Define effective tool functions

- Return Type:

- The function's return value must be a dictionary (dict) in Python.
- If your function returns a non-dictionary type, the ADK framework will automatically wrap it into a dictionary before passing the result back to the model.
- Design the keys and values to be descriptive and easily understood by the LLM.
For example, instead of returning just an error code like 500, return {'status': 'error', 'error_message': 'Database connection failed'}.
- It's a highly recommended practice to include a status key (e.g., 'success', 'error', 'pending', 'ambiguous') to clearly indicate the outcome of the tool execution for the model.

Define effective tool functions

- Docstring / Source Code Comments:
 - The docstring is the primary source of descriptive information for the LLM.
 - Clearly state what the tool does. Be specific about its purpose and limitations.
 - Explain when the tool should be used. Provide context or example scenarios to guide the LLM's decision-making.
 - Describe each parameter clearly. Explain what information the LLM needs to provide for that argument.
 - Describe the structure and meaning of the expected dictionary return value, especially the different status values and associated data keys.
 - Do not describe the injected ToolContext parameter.

Define effective tool functions

- Simplicity and Focus:
 - Keep Tools Focused: Each tool should ideally perform one well-defined task.
 - Fewer Parameters are Better: Models generally handle tools with fewer, clearly defined parameters more reliably than those with many optional or complex ones.
 - Use Simple Data Types: Prefer basic types over complex custom classes or deeply nested structures as parameters when possible.
 - Decompose Complex Tasks: Break down functions that perform multiple distinct logical steps into smaller, more focused tools. This makes it easier for the LLM to select and use the correct capability.

Define effective tool functions

```
def lookup_order_status(order_id: str) -> dict:
    """Fetches the status of a customer's order using its ID.

    Use this tool ONLY when a user explicitly asks for the status of
    a specific order and provides the order ID.
    Do not use it for general inquiries.

    Args:
        order_id: The unique identifier of the order to look up.

    Returns:
        A dictionary containing the order status.
        Possible statuses: 'shipped', 'processing', 'pending', 'error'.
        Example success: {'status': 'shipped', 'tracking_number': '1Z9...'}
        Example error: {'status': 'error', 'error_message': 'Order ID not found.'}
    """
    ...
```

T18. reimburse_agent

- LongRunningFunctionTool을 이용해서 Human-in-the-loop 구현해 보자.

reimburse_agent/agent.py

```
from google.adk.agents import Agent
from google.adk.tools import LongRunningFunctionTool

root_agent = Agent(
    model="gemini-2.0-flash",
    name="reimbursement_agent",
    instruction="...",
    tools=[reimburse, LongRunningFunctionTool(func=ask_for_approval)]
)
```

T18. reimburse_agent

date_agent/agent.py

```
from typing import Any

def ask_for_approval(purpose: str, amount: float) -> dict[str, Any]:
    """Ask for approval for the reimbursement."""
    return {'status': 'pending', 'purpose' : purpose, 'amount': amount}

def reimburse(purpose: str, amount: float) -> dict[str, Any]:
    """Reimburse the amount of money to the employee."""
    return {'status': 'approved', 'purpose' : purpose, 'amount': amount}
```

T18. reimburse_agent

date_agent/agent.py

```
from google.adk.agents import Agent
from google.adk.tools import LongRunningFunctionTool
```

```
root_agent = Agent(
    instruction="""
```

당신은 직원의 환급 프로세스를 처리하는 에이전트입니다.

\$100 이하의 요청은 자동으로 승인합니다. 'reimburse'를 호출합니다.

\$100 초과인 요청은 관리자의 승인을 요청합니다. 'ask_for_approval'을 호출합니다.

관리자가 승인하면 'reimburse'를 호출하여 환급을 처리합니다.

관리자가 거부하면 직원에게 거부 사실을 알립니다.

```
""",
```

T18. reimburse_agent

date_agent/agent.py

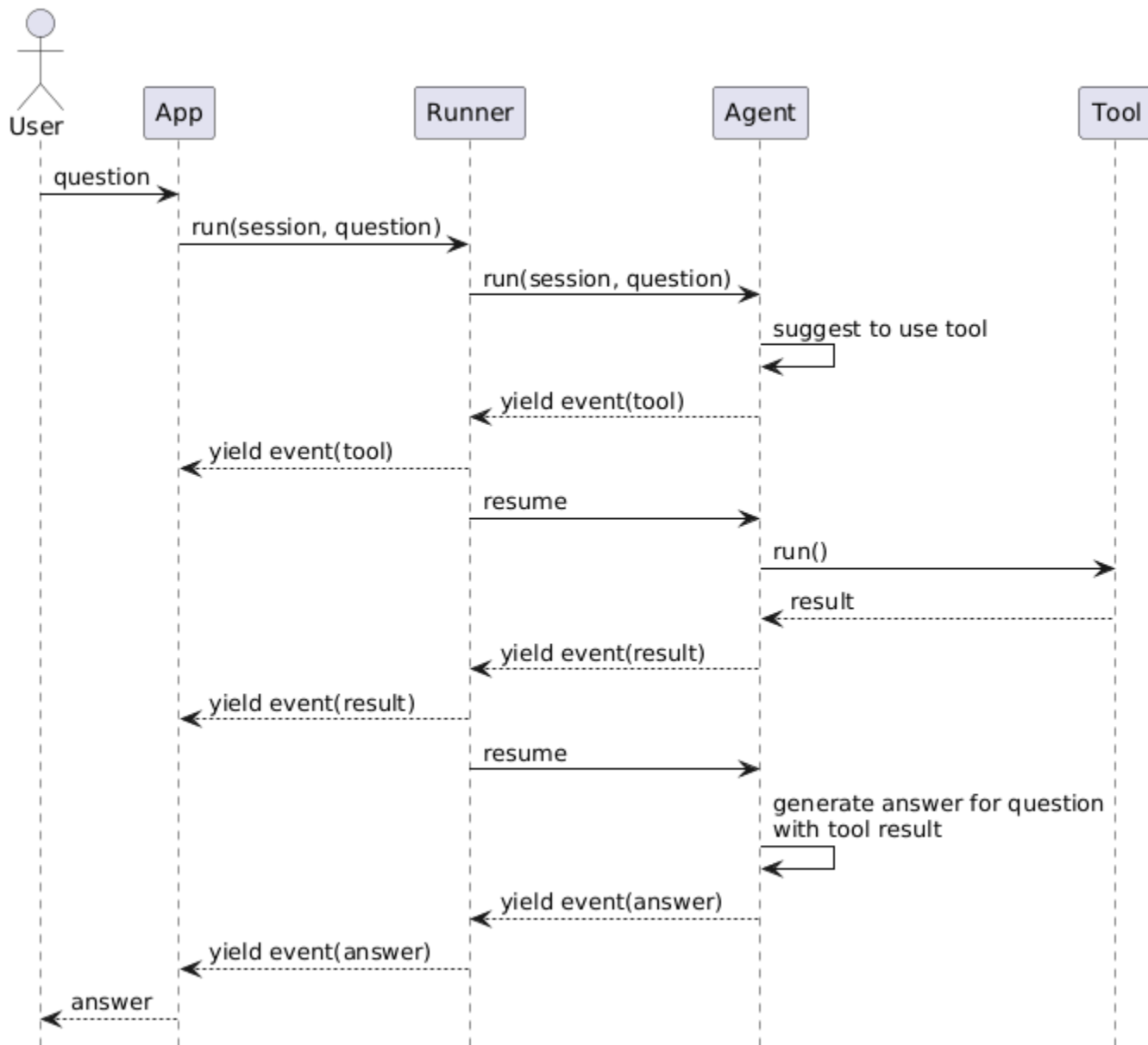
```
request = "회식 비용 $250을 환급해주세요."  
approval_request = None  
for event in runner.run(user_id=session.user_id, session_id=session.id,  
                        new_message=UserContent(request)):  
    if event.content and event.content.parts:  
        for part in event.content.parts:  
            if part.function_response:  
                if part.function_response.name == "ask_for_approval":  
                    approval_request = part.function_response  
            elif part.text:  
                print(f"Agent: {part.text}")
```


T18. reimburse_agent

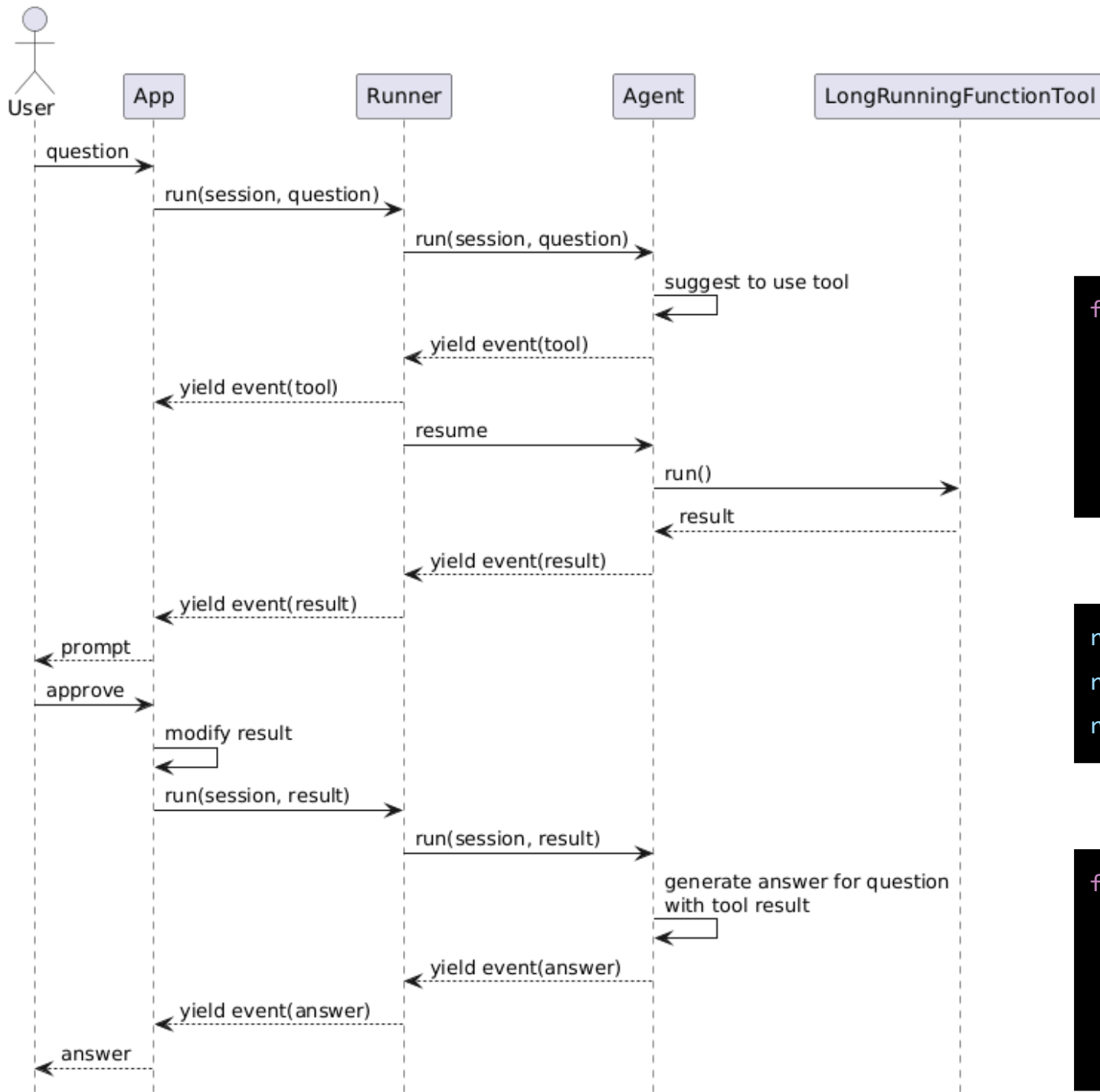
date_agent/agent.py

```
if approval_request:
    result = input("Enter your response(approved or rejected): ")
    approval_response = approval_request.model_copy(deep=True)
    approval_response.response["status"] = result

    for event in runner.run(user_id=session.user_id, session_id=session.id,
                           new_message=UserContent(parts=[Part(function_response=approval_response)])):
        if event.is_final_response():
            print(f"Agent: {event.content.parts[0].text}")
```



```
for event in runner.run(user_id=session.user_id,
                          session_id=session.id,
                          new_message=UserContent(question)):
    if event.is_final_response():
        print(f"Agent: {event.content.parts[0].text}")
```



```

for event in runner.run(user_id=session.user_id,
                        session_id=session.id,
                        new_message=UserContent(question)):
    for part in event.content.parts:
        approval_request = part.function_response
  
```

```

response = input("Enter your response(approved or rejected): ")
result = approval_request.model_copy(deep=True)
result.response["status"] = response
  
```

```

for event in runner.run(user_id=session.user_id,
                        session_id=session.id,
                        new_message=UserContent(result)):
    if event.is_final_response():
        print(f"Agent: {event.content.parts[0].text}")
  
```

T19. summary_agent

- summary_agent를 도구로 사용하는 요약 에이전트를 만들어 보자.

```
summary_agent/agent.py
```

```
from google.adk.agents import Agent

summary_agent = Agent(
    name="summarizer",
    model="gemini-2.0-flash",
    description="Agent to summarize text",
    instruction="주어진 내용을 간결하게 요약하십시오.",
)
```

T19. summary_agent

summary_agent/agent.py

```
from google.adk.agents import Agent

root_agent = Agent(
    model='gemini-2.0-flash',
    name='summary_agent',
    instruction="""당신은 유용한 도우미입니다.
        사용자가 텍스트 요약을 요청하면 'summarizer'에게 요청하세요.""",
    sub_agents=[summarizer],
)
```

T19. summary_agent

summary_agent/agent.py

```
from google.adk.tools import AgentTool

root_agent = Agent(
    model='gemini-2.0-flash',
    name='summary_agent',
    instruction="""당신은 유용한 도우미입니다.
        사용자가 텍스트 요약을 요청하면 'summarizer' 도구를 사용하세요.""",
    tools=[AgentTool(agent=summarizer)]
)
```

Agent-as-a-tool

- Key difference from sub-agents
 - Agent-as-a-Tool: When Agent A calls Agent B as a tool, Agent B's answer is passed back to Agent A, which then summarizes the answer and generates a response to the user. Agent A retains control and continues to handle future user input.
 - Sub-agent: When Agent A calls Agent B as a sub-agent, the responsibility of answering the user is completely transferred to Agent B. Agent A is effectively out of the loop. All subsequent user input will be answered by Agent B.

Built-in tools

- google_search
- code_executor
- load_memory_tool
- load_artifacts_tool
- load_web_page
- get_user_choice_tool
- exit_loop
- :

3rd-party tools

- LangChain tools
- CrewAI tools
- OpenAPI tools
- MCP tools
- ⋮

```
from langchain_community.tools import WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper
tool = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())

from google.adk.agents import Agent
from google.adk.tools.langchain_tool import LangchainTool
wikipedia_agent = Agent(
    model="gemini-2.0-flash",
    name="wikipedia_agent",
    instruction="추가 정보는 Wikipedia 도구를 사용하세요.",
    tools=[LangchainTool(tool=tool)]
)
```

MCP(Model Context Protocol)

인공지능 모델과 외부 세계를 연결하는 혁신적인 표준 프로토콜

엔트로픽이 2024년 11월 공식 발표한 MCP는
AI 모델이 외부 데이터 및 도구와 효율적으로 상호작용할 수 있는
표준화된 방법을 제공합니다.

OpenAI와 Google DeepMind를 포함한 주요 AI 기업들도
이 프로토콜을 채택하며 업계 표준으로 자리잡고 있습니다.



MCP 등장 배경

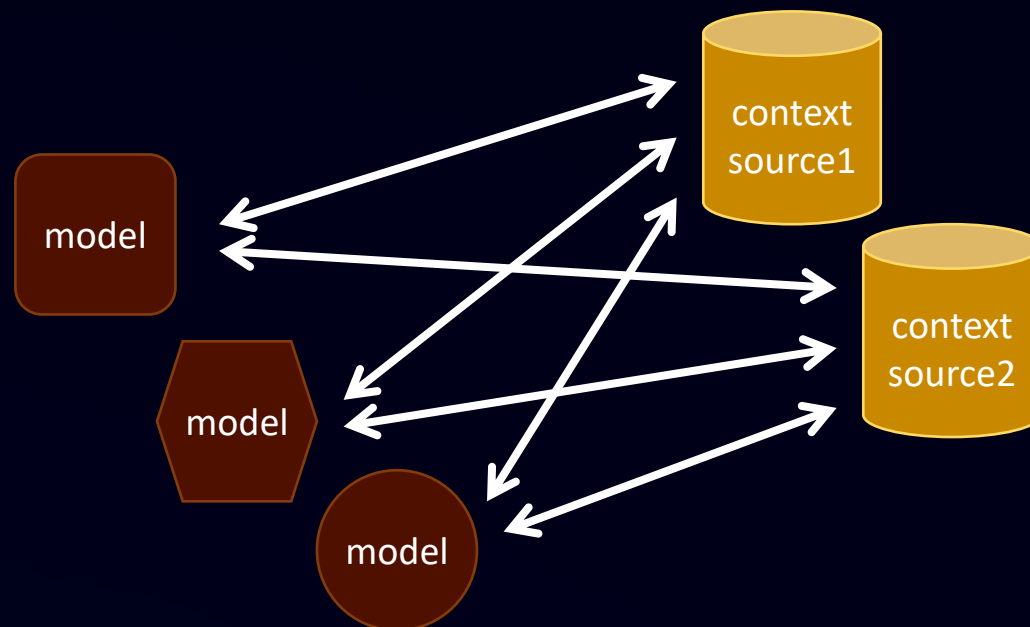
개별 통합의 한계

각 LLM이 컨텍스트 소스마다 별도의
커스텀 통합을 구현해야 하는 비효율성

API 복잡성

다양한 API와 라이브러리를 개별적으로
관리하고 유지해야 하는 개발 부담

기존 방식으로는 각 데이터 소스마다 별도의 통합 작업이 필요하여 개발 과정이 복잡해지고
유지보수 비용이 증가했습니다.



MCP의 기본 목표와 철학



현실 세계와 직접 상호작용하도록 지원

안전하고 양방향적인 데이터 연동 체계 구축

AI 분야의 'USB-C 포트'처럼 보편적 표준 확립

다양한 모델과 데이터 소스 간 호환성 보장

개발자 경험 단순화 및 효율성 극대화

MCP는 AI 모델이 실시간으로 외부 세계와 소통하며 더 유용한 서비스를 제공할 수 있도록 설계된 표준 프로토콜입니다.

MCP의 주요 기능 및 역할

1

파일 처리 및 데이터 접근

문서, 이미지, 데이터베이스 등 다양한 형태의 파일을 읽고 처리할 수 있는 표준화된 인터페이스 제공

2

함수 실행 기능

외부 API 호출, 계산, 데이터 변환 등 다양한 함수를 안전하게 실행할 수 있는 메커니즘 지원

3

프롬프트 컨텍스트 관리

대화 기록, 사용자 설정, 환경 변수 등 컨텍스트 정보를 효율적으로 관리하고 활용

4

모델 불가지론 인터페이스

특정 AI 모델에 종속되지 않고 다양한 모델에서 동일한 방식으로 활용 가능한 유연한 구조 제공

MCP의 핵심 구성요소



Host

AI 모델을 운용하는 주체로, 사용자 요청을 처리하고 필요한 컨텍스트 정보를 관리합니다.



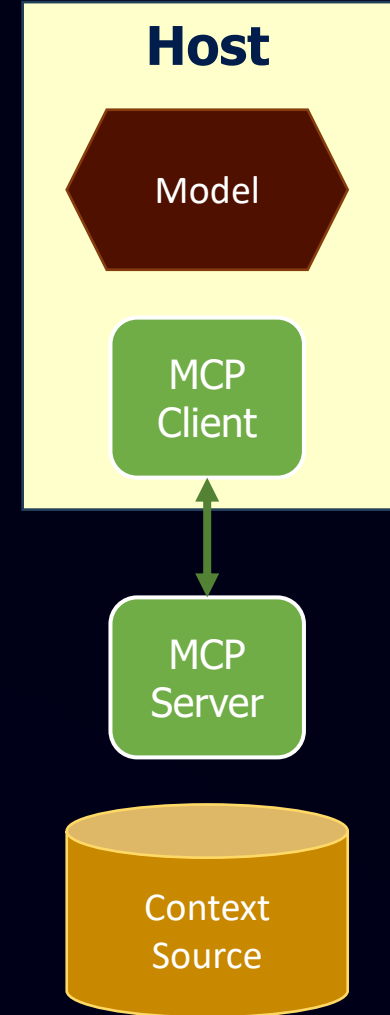
Client

각 데이터 서버와 1:1로 연결을 관리하는 역할을 담당합니다. 호스트의 요청을 적절한 서버에 전달하고 응답을 처리합니다.



Server

리소스, 도구, 프롬프트 등을 함수 형태로 노출하여 AI 모델이 접근할 수 있도록 합니다. 파일 시스템, 데이터베이스, API 등 다양한 서비스를 제공합니다.

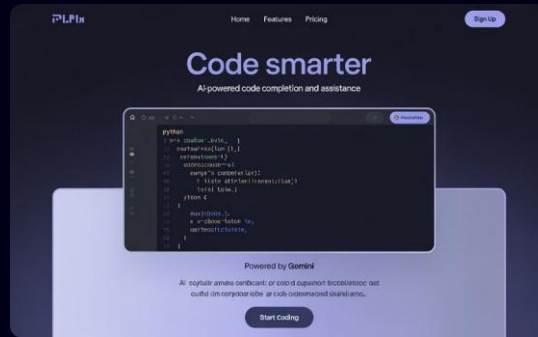


MCP 활용 실제 사례



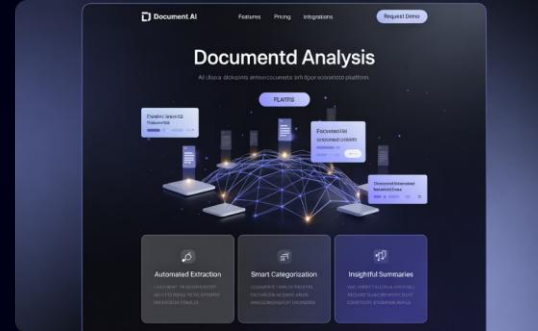
지능형 채팅 어시스턴트

실시간 데이터베이스 조회를
통해 최신 정보를 제공하는
챗봇 서비스



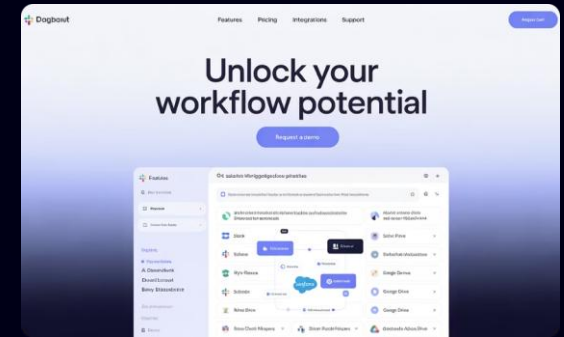
IDE AI 코딩 도우미

코드 저장소와 문서를 참조하여
정확한 코드 제안을 제공하는
개발 도구



문서 처리 자동화

대량의 파일을 분석하고 요약하여
핵심 정보를 추출하는 시스템



업무 생산성 도구

협업 도구와 데이터 시각화 툴을
연동하여 효율적인 업무 지원

MCP는 다양한 산업 분야에서 AI의 실용적 활용을 가능하게 하며, 사용자 경험을 크게 향상시킵니다.

MCP의 장점 및 파급력

상호운용성

다양한 AI 모델과 데이터 소스 간의 원활한 통합을 통해 시스템 간 호환성 극대화

혁신 가속화

신속한 AI 응용 서비스 개발 환경 조성으로 새로운 비즈니스 모델 창출



개발 효율성

표준화된 인터페이스로 개발 시간 단축 및 유지보수 비용 절감

보안 강화

일관된 보안 정책 적용을 통해 데이터 접근 및 처리의 안전성 보장

MCP는 기술적 효율성을 넘어 AI 생태계 전반에 혁신적인 변화를 가져올 것으로 기대됩니다.



향후 전망 및 맺음말

업계 표준 확립

MCP는 AI 애플리케이션 개발의 사실상 표준으로 자리잡을 것으로 예상됩니다. 주요 AI 기업들의 채택으로 그 영향력은 더욱 확대될 전망입니다.

산업 적용 확대

금융, 의료, 제조 등 다양한 산업 분야에서 MCP를 활용한 AI 솔루션이 확산될 것으로 기대됩니다. 특히 실시간 데이터 처리가 중요한 영역에서 큰 혁신이 예상됩니다.

오픈소스 생태계 발전

MCP가 오픈소스로 공개됨에 따라 개발자 커뮤니티의 적극적인 참여와 지속적인 발전이 이루어질 것입니다. 다양한 확장 기능과 플러그인이 개발될 것으로 예상됩니다.

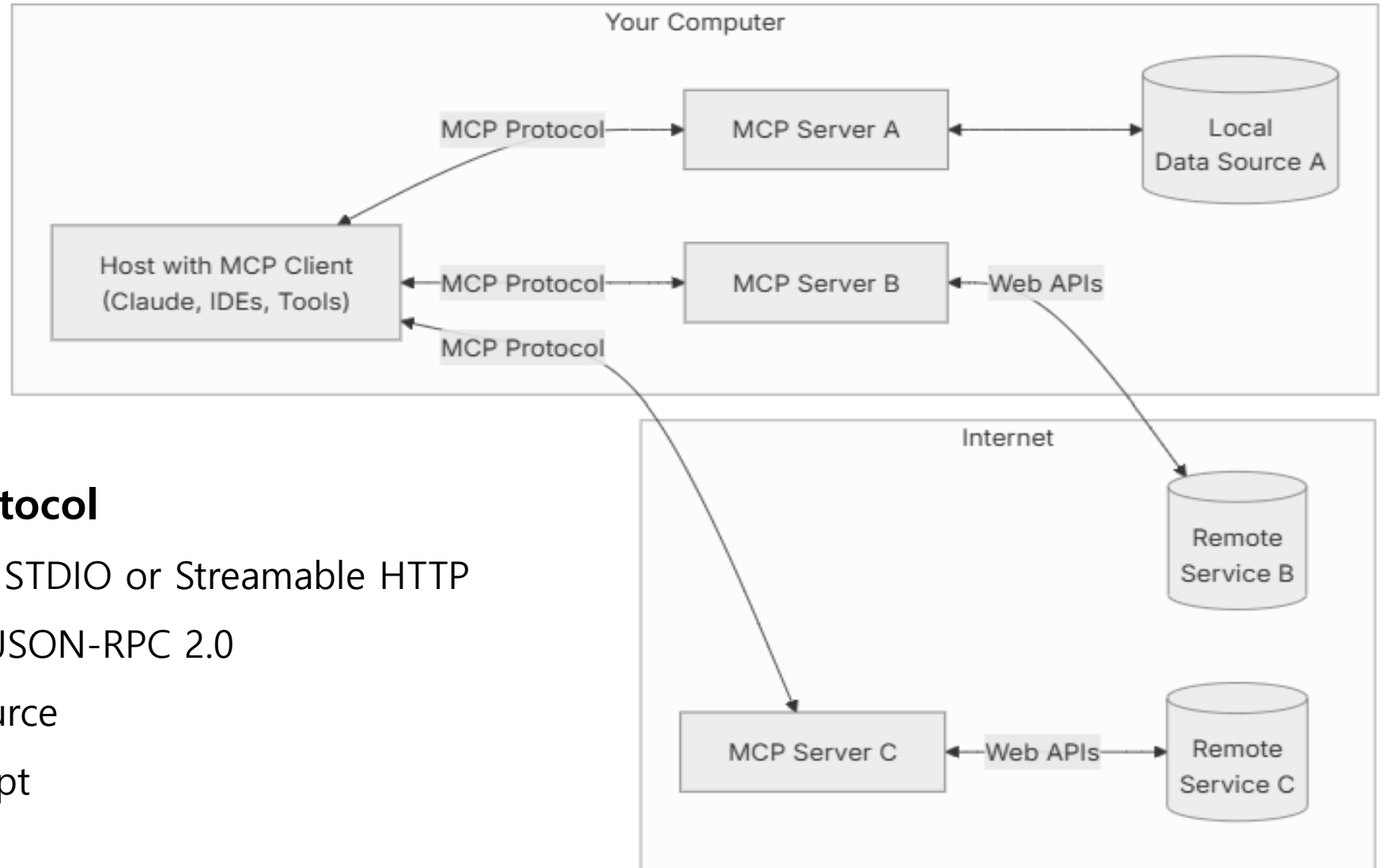
LLMaBoK

MCP(Model Context Protocol)

조 용 진

(drajin.cho@bosornd.com)

MCP Architecture



MCP Protocol

Transport: STDIO or Streamable HTTP

Message: JSON-RPC 2.0

- Resource
- Prompt
- Tool

MCP Inspector

<https://nodejs.org/>

- Install Node.js

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support

for Node.js 18 and below

Create an HTTP Server Write Tests Read and Hash a File Streams Pipeline Work with Threads

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript

클립보드에 복사

Learn more what Node.js is able to offer with our [Learning materials](#).

MCP Inspector

<https://github.com/modelcontextprotocol/inspector>
<https://github.com/modelcontextprotocol/servers>

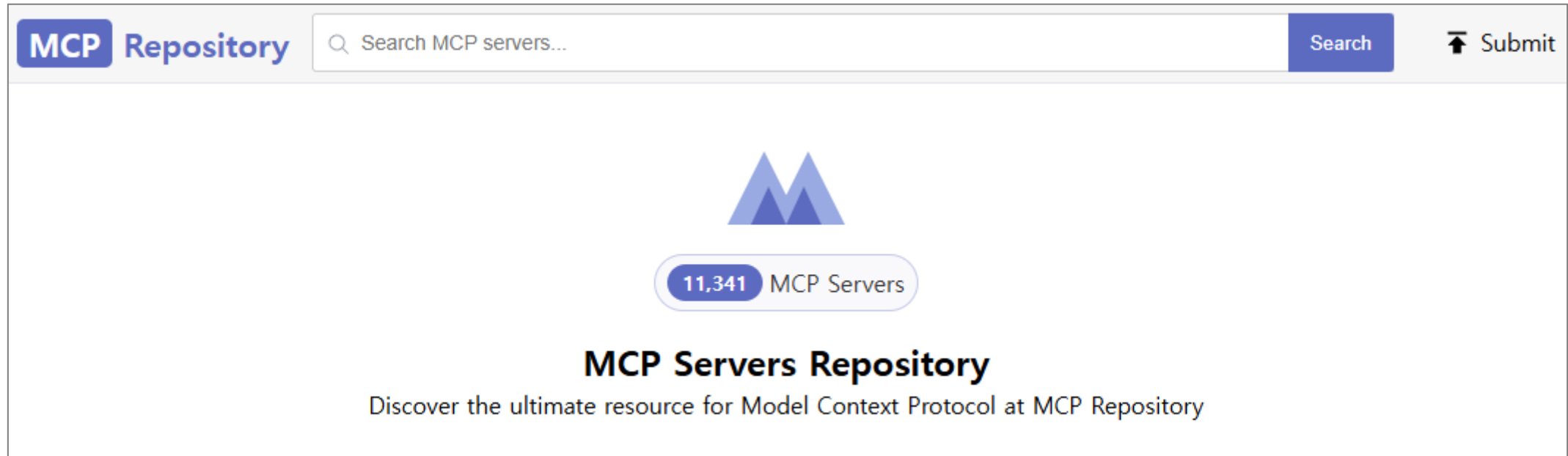
- `npx @modelcontextprotocol/inspector`
- `npx @modelcontextprotocol/server-everything`

The screenshot displays the MCP Inspector v0.16.1 interface. On the left sidebar, the 'Transport Type' is set to 'STDIO', the 'Command' is 'npx', and the 'Arguments' are '@modelcontextprotocol/server-every'. Below these, there are buttons for 'Environment Variables', 'Server Entry', 'Servers File', and 'Configuration'. At the bottom of the sidebar, there are 'Restart' and 'Disconnect' buttons, a 'Connected' status indicator, a 'Logging Level' dropdown set to 'debug', and an 'Error output from MCP server' section with a 'Clear' button.

The main area is divided into two panels. The left panel, titled 'Tools', contains a list of available tools: 'echo' (Echoes back the input), 'add' (Adds two numbers), 'printEnv' (Prints all environment variables, helpful for debugging MCP server configuration), 'longRunningOperation' (Demonstrates a long running operation with progress updates), 'sampleLLM' (Samples from an LLM using MCP's sampling feature), and 'getTinyImage' (Returns the MCP TINY IMAGE). The right panel shows the configuration for the 'add' tool. It has two input fields labeled 'a' and 'b', with values '2' and '3' respectively. A 'Run Tool' button is present. Below the inputs, the 'Tool Result' is displayed as 'Success' with the message 'The sum of 2 and 3 is 5.'

MCP Servers

<https://mcprepository.com>
<https://github.com/modelcontextprotocol/servers>



T20. filesystem_agent

- MCP filesystem 도구를 사용하는 Filesystem 에이전트를 만들어 보자.

filesystem_agent/agent.py

```
import os

from google.adk.tools.mcp_tool import MCPToolset, StdioConnectionParams
from mcp.client.stdio import StdioServerParameters

filesystem_toolset = MCPToolset(
    connection_params = StdioConnectionParams(
        server_params = StdioServerParameters(
            command="npx", args=["-y", "@modelcontextprotocol/server-filesystem",
                                os.path.abspath("../")])))
```

LLMaBoK

Multi-agent System

조 용 진

(drajin.cho@bosornd.com)



Single Agent vs. Multi-agent System

■ 복잡성 문제

- **단일 에이전트:** 전체 문제를 하나의 거대 모델로 해결하므로, 개발 및 유지보수가 어려움.
- **다중 에이전트의 보완:** 각 에이전트가 전문적인 역할을 수행하며, 시스템의 복잡성을 낮춤.

■ 확장성 및 견고성 부족

- **단일 에이전트:** 에이전트가 실패하면 시스템 전체가 중단됨 (Single Point of Failure).
- **다중 에이전트의 보완:** 분산 구조로 일부 에이전트가 실패해도 전체 시스템은 계속 작동.
새로운 에이전트 추가가 쉬워 **확장성**이 뛰어남.

■ 비효율적 자원 활용

- **단일 에이전트:** 모든 작업을 순차적으로 처리. 높은 연산 자원을 요구.
- **다중 에이전트의 보완:** 여러 에이전트가 **병렬적으로** 작업을 처리.
작고 효율적인 모델을 사용해 **자원 활용 효율성**을 높임.

MAS – Task Decomposition

- 작업 분해는 다중 에이전트 시스템 설계의 근본적인 원리가 됩니다.
 - **복잡성 감소:** 각 에이전트는 하나의 하위 작업에만 집중할 수 있어 시스템의 복잡성을 크게 줄일 수 있습니다.
 - **모듈성 및 확장성:** 특정 하위 작업을 담당하는 에이전트를 교체하거나 추가하기 용이해져 시스템의 **모듈성**과 **확장성**이 향상됩니다.
 - **병렬 처리 가능:** 분해한 작업들을 여러 에이전트에게 동시에 할당하여 **병렬 처리**할 수 있습니다. 이는 문제 해결 시간을 단축하고 시스템의 효율을 극대화합니다.
 - **역할 부여 및 전문화:** 각 에이전트에게 특정 역할을 부여하여 **전문화**를 가능하게 합니다.

MAS – Hierarchical Organization

- Worker Agents: 특정 하위 작업을 수행하는 데 초점을 맞춘 에이전트
- Manager Agents: Worker 에이전트들의 행동을 조율하고, 전체 시스템의 목표를 관리하는 상위 수준의 역할을 담당하는 에이전트
 - Planner: 전체 문제 해결을 위한 전략과 실행 계획을 수립합니다.
 - Coordinator: 에이전트 간의 상호작용을 조율하여 충돌을 방지하고 협력을 촉진합니다.
 - Delegator: 상위 작업을 하위 작업으로 분해하여 적절한 워커에게 할당합니다.
 - Broker: 에이전트 간의 통신을 중개하고, 필요한 에이전트나 자원을 연결해줍니다.

MAS – Self-Reflection

- Monitor Agent: 시스템에서 발생하는 이벤트를 추적하고,
예상치 못한 상황이나 오류를 감지하는 에이전트.
- Evaluator Agent: 시스템의 성능을 분석하고 판단하는 에이전트.
'개선이 필요한 부분', '실패 요인' 등의 피드백 제공.
- Refiner Agent: 시스템을 동작을 개선하는 에이전트.

MAS – Self-Organization

- Architect Agent: 전체적인 구조와 조직을 설계하고 동적으로 관리하는 최상위 계층의 존재입니다. 시스템의 목표를 달성하기 위한 가장 효율적인 '팀'을 구성하는 데 중점을 둡니다.



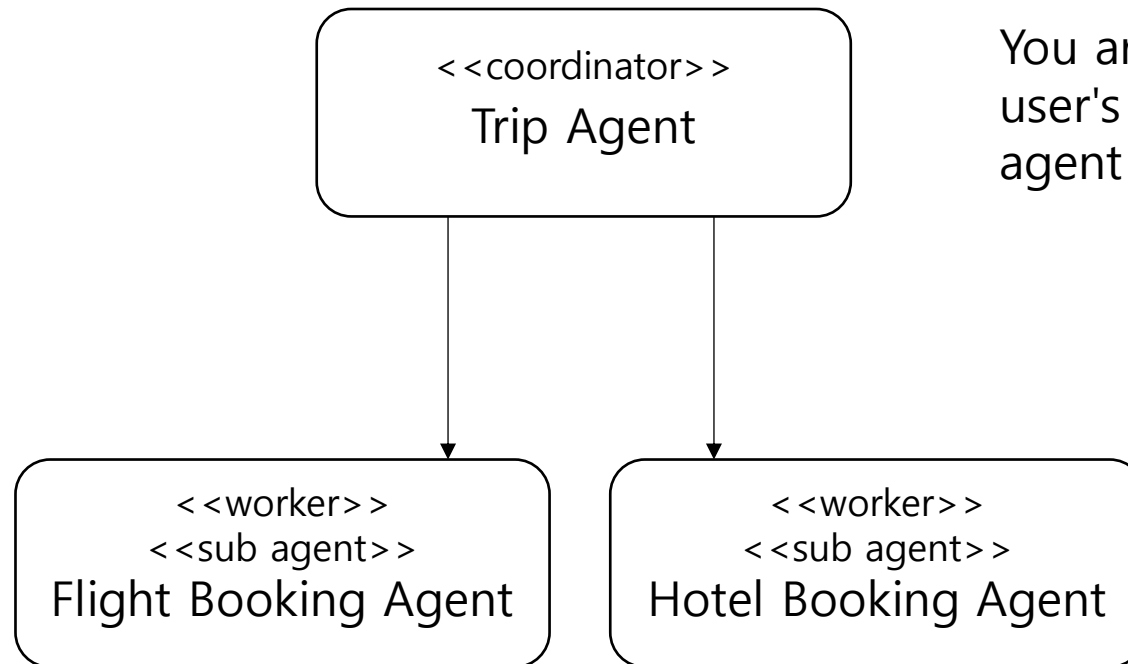
머스크 "AI로만 운영되는 소프트웨어 회사 '매크로하드' 설립할 것"

2025년 8월 22일

- ▲ 인간의 음성과 텍스트를 AI로 생성하는 다운로드 가능한 컴퓨터 소프트웨어
- ▲ AI를 사용해 비디오 게임을 디자인, 코딩, 실행, 플레이하는 다운로드 가능한 컴퓨터 소프트웨어 등

MAS patterns

- Coordinator / Dispatcher Pattern



You are a helpful assistant in planning and organizing user's travel itinerary. Assign tasks to the appropriate agent based on the user's request.

MAS patterns

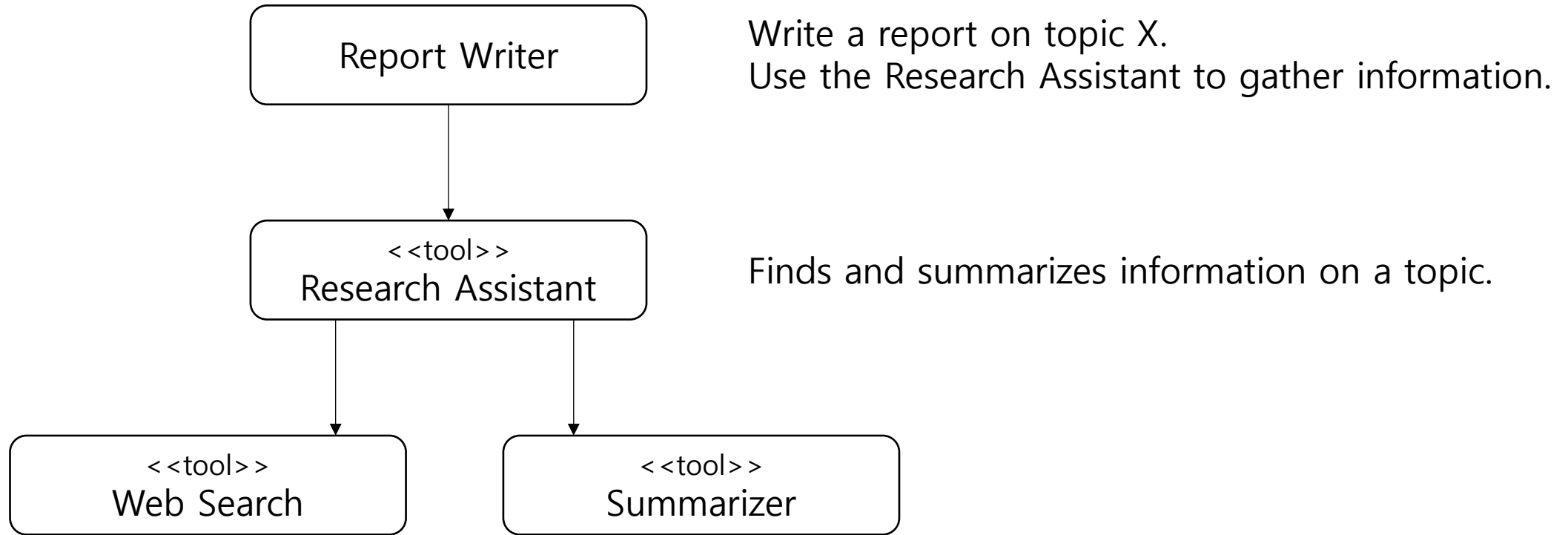
- Coordinator / Dispatcher Pattern

```
from google.adk.agents import Agent

trip_agent = Agent(
    name="TripAgent",
    model="gemini-2.0-flash",
    instruction="""You are a helpful assistant in planning and organizing
                    user's travel itinerary. Assign tasks to the appropriate
                    agent based on the user's request."""
    sub_agents=[flight_booking_agent, hotel_booking_agent]
)
```

MAS patterns

- Hierarchical Task Decomposition



MAS patterns

- Hierarchical Task Decomposition

```
web_searcher = Agent(name="WebSearch", description="Search web for facts.")
summarizer = Agent(name="Summarizer", description="Summarizes text.")

research_assistant = Agent(name="ResearchAssistant",
    instruction="Finds and summarizes information on a topic.",
    tools=[AgentTool(agent=web_searcher), AgentTool(agent=summarizer)])

report_writer = Agent(name="ReportWriter",
    instruction="""Write a report on topic X.

        Use the ResearchAssistant to gather information.""",
    tools=[AgentTool(agent=research_assistant)])
```

MAS patterns

- Parallel Fan-Out / Gather Pattern

```
fetch1 = Agent(name="API1Fetcher", instruction="...", output_key="data1")
fetch2 = Agent(name="API2Fetcher", instruction="...", output_key="data2")

gather = ParallelAgent(name="ConcurrentFetch", sub_agents=[fetch1, fetch2])

synthesizer = Agent(name="Synthesizer",
                    instruction="Combine results from {data1} and {data2}.")

overall_workflow = SequentialAgent(name="FetchAndSynthesize",
                                   sub_agents=[gather, synthesizer])
```

MAS patterns

- Review / Critique Pattern (Generator-Critic)

```
from google.adk.agents import Agent, SequentialAgent

generator = Agent(name="DraftWriter", output_key="draft_text",
                  instruction="Write a short paragraph about subject X.")

reviewer = Agent(name="FactChecker", output_key="review_status",
                 instruction="""Review the text in {draft_text} for factual accuracy.
                               Output 'valid' or 'invalid' with reasons.""")

review_pipeline = SequentialAgent(name="WriteAndReview",
                                  sub_agents=[generator, reviewer])
```

MAS patterns

- Iterative Refinement Pattern

```
class CheckStatusAndEscalate(BaseAgent):
    async def _run_async_impl(self, ctx: InvocationContext)
        -> AsyncGenerator[Event, None]:

        status = ctx.session.state.get("quality", "fail")
        should_stop = (status == "pass")
        yield Event(author=self.name,
                    actions=EventActions(escalate=should_stop))

refinement_loop = LoopAgent(name="CodeRefinementLoop", max_iterations=5,
                             sub_agents=[code_refiner, quality_checker,
                                           CheckStatusAndEscalate(name="StopChecker")])
```

MAS patterns

- Human-in-the-Loop Pattern

```
def ask_for_approval(content: str) -> str:
    response = input(f"Approve '{content}'? (yes/no): ")
    return "approved" if response.lower() == "yes" else "rejected"

request_approval = Agent(name="RequestHumanApproval",
    instruction="Use the 'ask_for_approval' function for user consent.",
    tools=[ask_for_approval])
```

LLMaBoK

Project #1. Algorithm Problem Solver

조 용 진

(drajin.cho@bosornd.com)

알고리즘 문제 해결기

■ 과제 목표

- 주어진 알고리즘 문제를 해결하는 프로그램을 개발하는 에이전트
- 작성된 프로그램을 실행해서 결과를 검증한다.
- 작성된 프로그램의 설계 및 코드 품질을 높인다.
- 작성된 프로그램을 설명하는 문서를 생성한다.

종을 한번 칠때마다 암탉은 알을 낳고, 알은 병아리가 되고, 병아리는 암탉이 된다.
처음에 암탉만 1마리 있을 때, 종을 10번 치면 암탉, 병아리, 알이 몇 개가 되는지를
출력하는 프로그램을 작성해라.

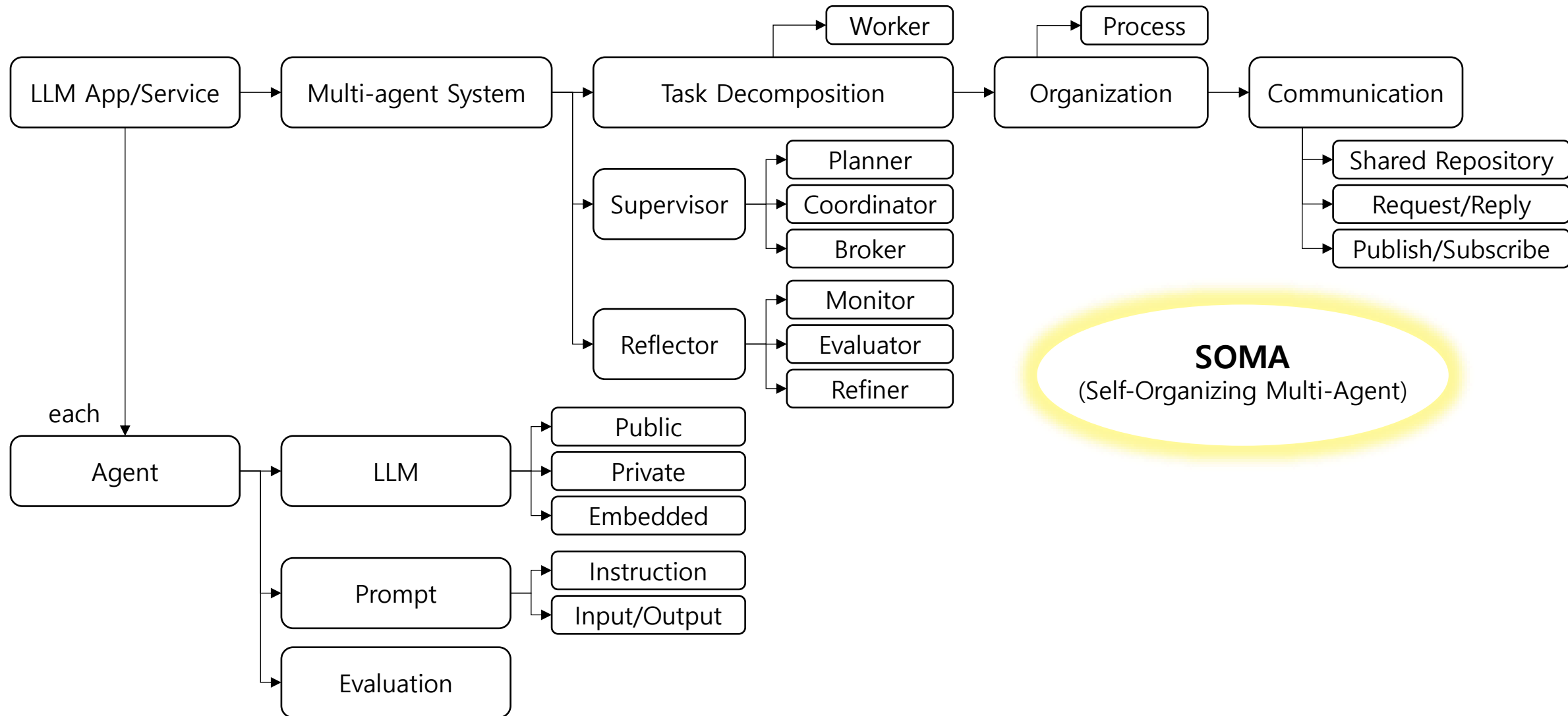
실습 1. 단일 에이전트

- 주어진 알고리즘 문제를 해결하는 단일 에이전트를 개발한다.
 - 프로그램을 개발하고 실행 결과를 확인하도록 요청한다.
 - 작성된 프로그램의 설계 및 코드 품질이 높도록 요청한다.
 - 작성된 프로그램에 대한 설명을 생성하도록 요청한다.

실습 2. MAS

- 주어진 알고리즘 문제를 해결하는 MAS를 설계하고 개발한다.
 - 기능별 역할을 세분화 한다.
 - . 사용자의 요청에 대한 처리 계획을 수립한다.
 - . 계획에 따라 업무를 배정한다.
 - . 업무를 수행한다.
 - . 업무 결과를 점검한다.
 - . 업무 결과를 통합한다.

LLM App/Service Design Guide



실습 3. 공유하기

- 설계 및 개발 내용을 공유(발표) 한다.
 - Monolithic Agent와 MAS를 비교한다.
 - 추가 개선 사항을 도출해 본다.

LLMaBoK

RAG (Retrieval Augmented Generation)

조 용 진

(drajin.cho@bosornd.com)

Why, Hallucination?

<https://openai.com/index/why-language-models-hallucinate>

■ 환각이란?

- 언어 모델이 불확실한 상황에서 사실과 다른, 그럴듯한 답변을 지어내는 현상
- 마치 시험 문제를 모르는 학생이 정답 대신 그럴싸한 오답을 찍는 것과 유사

■ 환각이 발생할 수 밖에 없는 이유는?

- **훈련 과정의 통계적 압력:** 모델은 방대한 데이터 속에서 유효한 문장과 유효하지 않은 문장을 구분하도록 훈련됨. 이 과정에서 훈련 데이터에 드물게 등장하는 사실(Long-tail facts)에 대해서는 통계적 오류를 일으킬 가능성이 높아짐, 이는 환각으로 이어짐.
- **잘못된 평가 시스템:** 현재의 평가 방식은 '**정답**' 또는 '**오답**'으로만 채점하는 이진 분류 방식을 사용함. 모델이 '모르겠다'고 답하거나 답변을 회피할 때 점수를 주지 않음. 따라서 모델은 불확실한 상황에서도 추측하여 답변하는 것이 더 높은 점수를 얻는 데 유리하도록 최적화되어 환각이 만성화됨.

Context & RAG

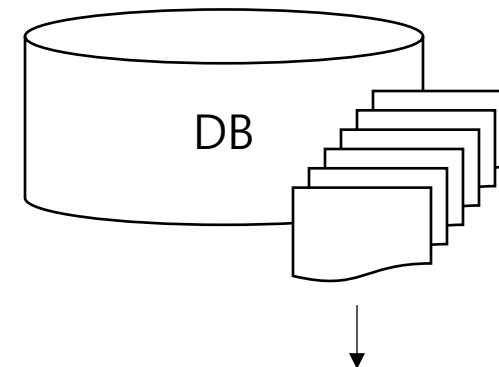
- 대부분 LLM 실패는 모델 문제가 아니라, **컨텍스트 부족** 때문임
→ Context Engineering

다음의 Context를 읽고, 질문에 답해줘.

Context:

질문: 한국의 대통령은?

RAG
(Retrieval Augmented Generation)



RAG(Search) Tool

T3. search_agent using google_search

- 구글 검색 도구를 사용하는 RAG 에이전트를 만들어 보자.

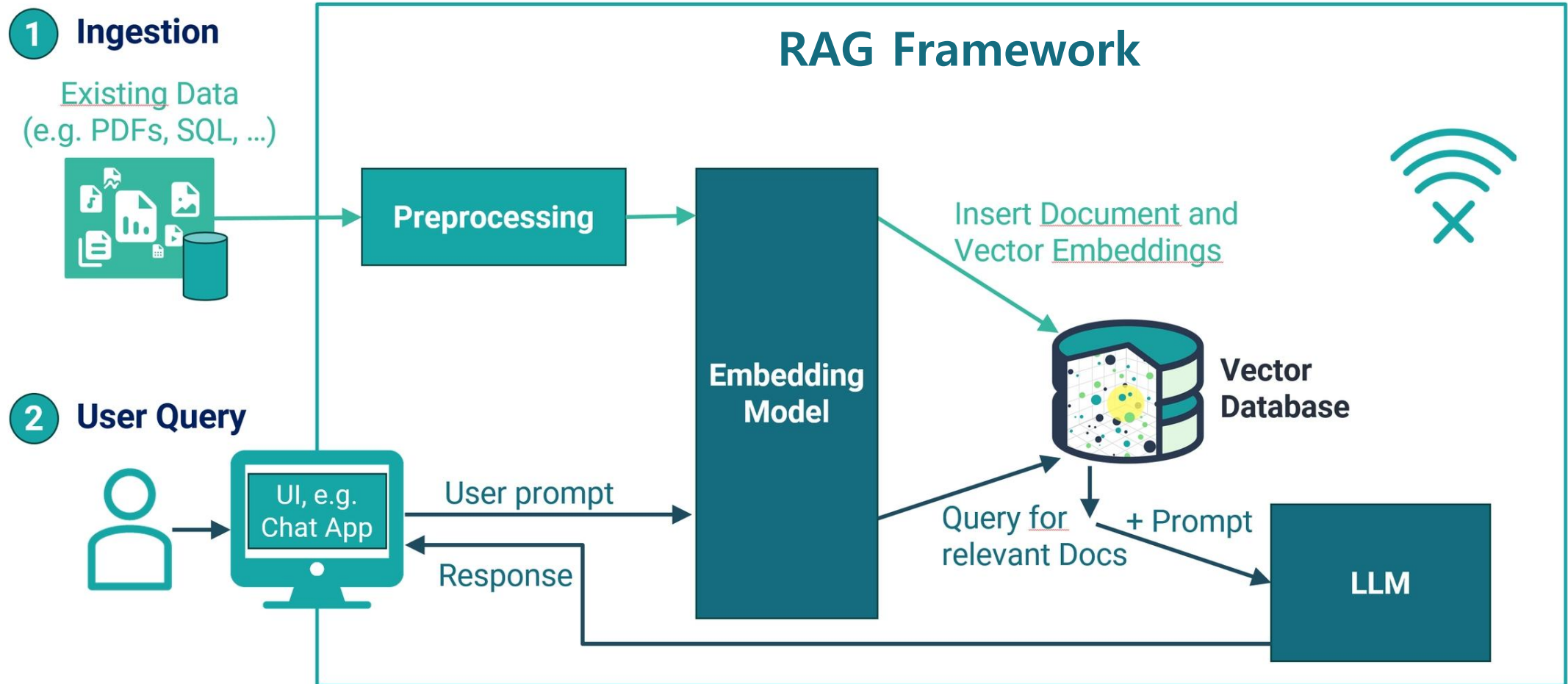
search_agent/agent.py

```
from google.adk.agents import Agent
from google.adk.tools import google_search

root_agent = Agent(
    name="search_agent",
    model="gemini-2.0-flash",
    instruction="You are an expert researcher. You always stick to the facts.",
    tools=[google_search],
)
```

Vector DB

https://en.wikipedia.org/wiki/Vector_database



Vector DB

https://en.wikipedia.org/wiki/Vector_database



<https://www.trychroma.com>



<https://qdrant.tech>



<https://milvus.io>



OBJECTBOX

<https://objectbox.io>



ORACLE

Qdrant

<https://qdrant.tech>

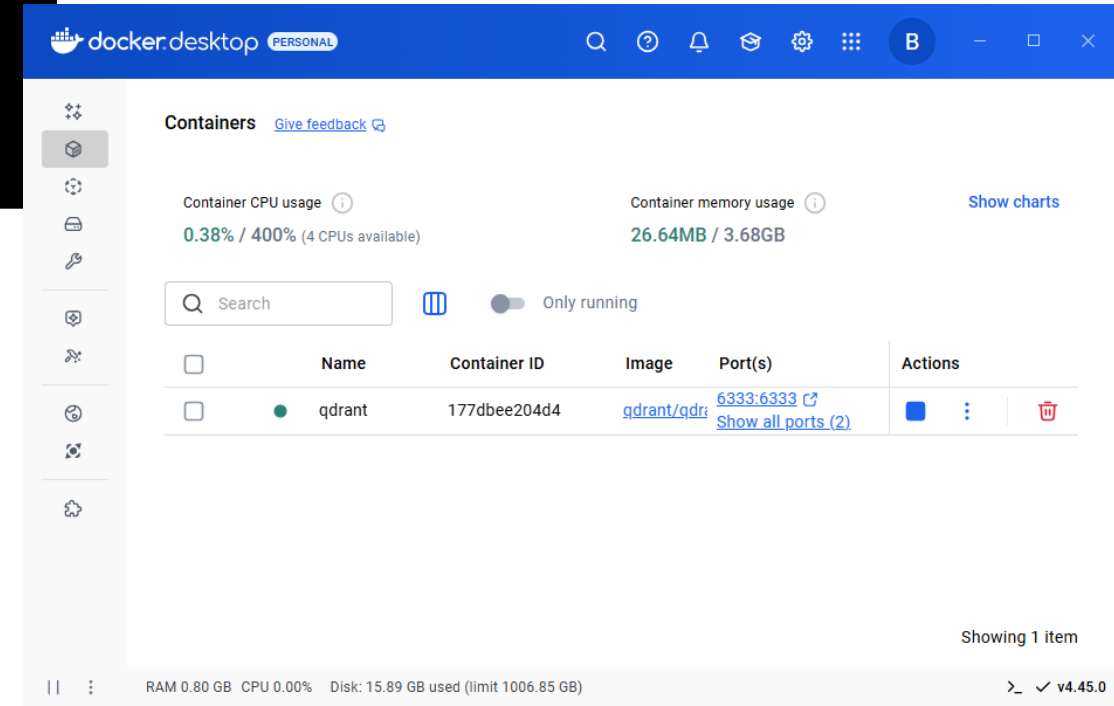
- Local Quickstart

```
C:\W> docker pull qdrant/qdrant
```

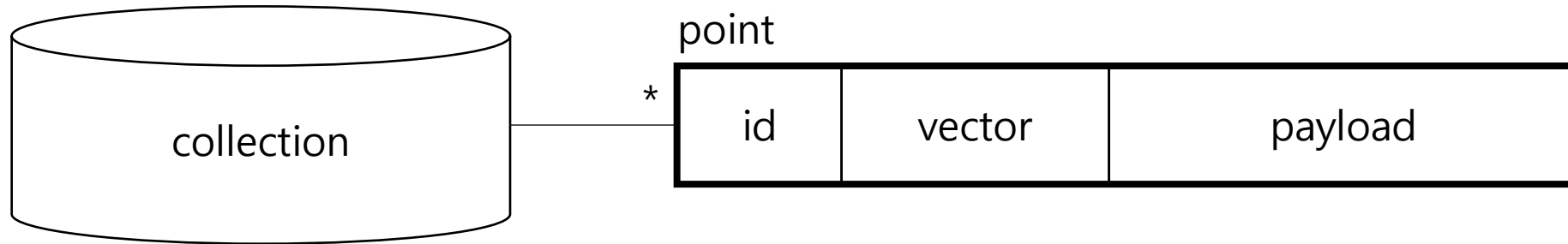
```
C:\W> docker run -p 6333:6333 -p 6334:6334 \
-v ./qdrant_storage:/qdrant/storage:z \
--name qdrant \
qdrant/qdrant
```

<https://www.docker.com>

- web: <http://localhost:6333/dashboard>



Qdrant



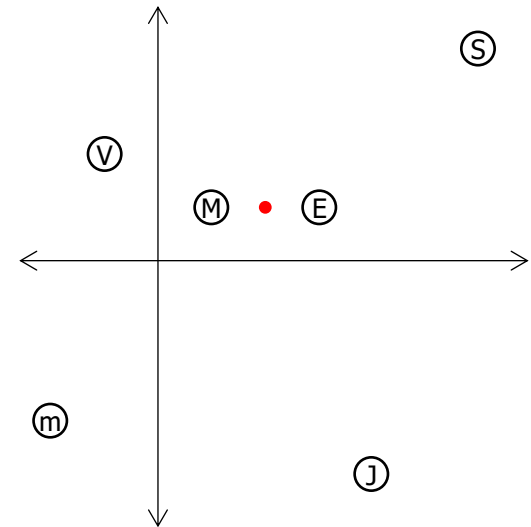
```
{  
  "name": "jacket",  
  "colors": ["red", "blue"],  
  "count": 10,  
  "price": 11.99,  
  "locations": [  
    {  
      "lon": 52.5200,  
      "lat": 13.4050  
    }  
  ]  
}
```

T21. star collection

- Star collection을 생성하고 (2, 1)에서 가까운 point를 검색해 본다.

qdrant/star.py

```
from qdrant_client.models import PointStruct
points=[
    PointStruct(id=0, vector=[1, 1], payload={"planet": "Mercury"}),
    PointStruct(id=1, vector=[-1, 2], payload={"planet": "Venus"}),
    PointStruct(id=2, vector=[3, 1], payload={"planet": "Earth"}),
    PointStruct(id=3, vector=[-2, -3], payload={"planet": "Mars"}),
    PointStruct(id=4, vector=[4, -4], payload={"planet": "Jupiter"}),
    PointStruct(id=5, vector=[6, 4], payload={"planet": "Saturn"}),
]
```



T21. star collection

qdrant/star.py

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams

client = QdrantClient(url="http://localhost:6333")

client.create_collection(
    collection_name="star",
    vectors_config=VectorParams(size=2, distance=Distance.EUCLID),
                                     # Distance.COSINE, Distance.DOT
)
```

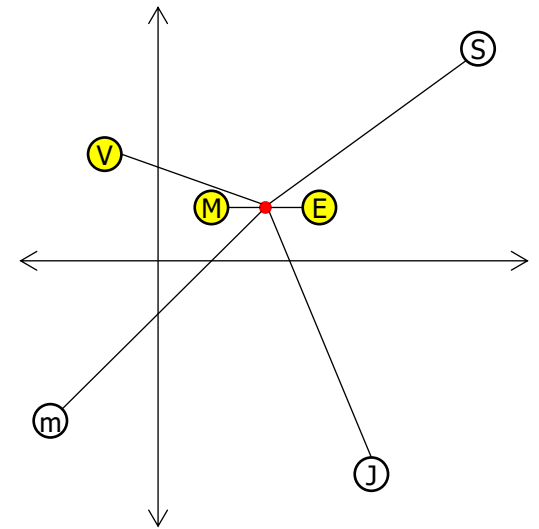
T21. star collection

qdrant/star.py

```
client.upsert(collection_name="star", points=points)

near_points = client.query_points(
    collection_name="star",
    query=[2, 1],
    limit=3
)

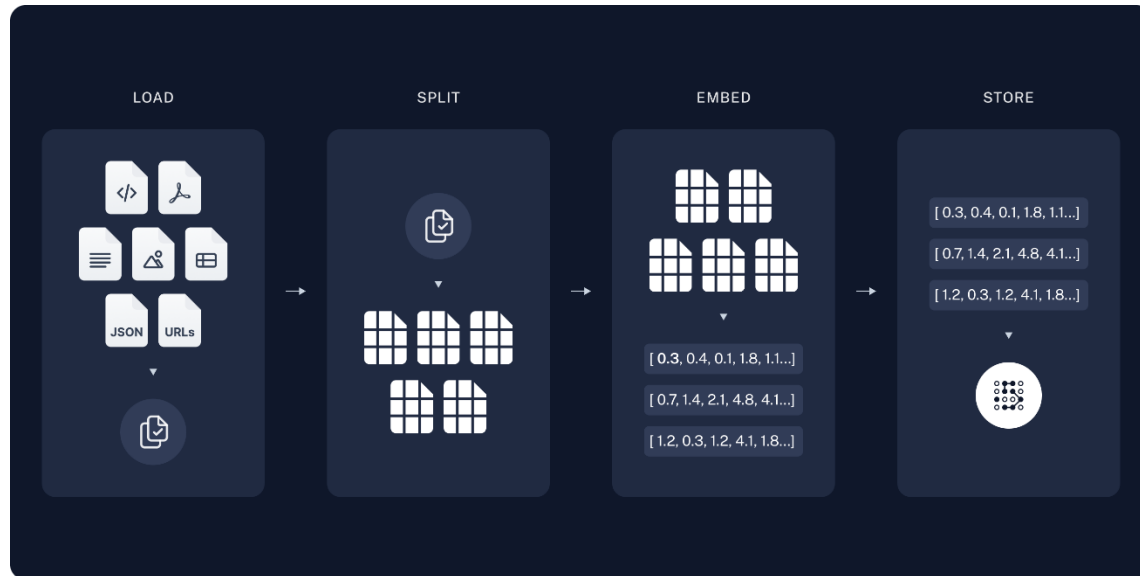
print(near_points)
```



$$\text{Euclidean distance} = | \vec{a} - \vec{b} |$$

RAG

- A typical RAG application has two main components:
 - **Indexing**: a pipeline for ingesting data from a source and indexing it.



- Step 1. Load Document
- Step 2. Split Document
- Step 3. Embed Vector
- Step 4. Store at Vector DB

- **Retrieval and generation**: the actual RAG chain, which takes the user query at run time and retrieves the relevant data from the index, then passes that to the model.

T22. book_agent

- 베스트셀러 도서에 대해 답하는 에이전트를 만들어 보자.

book_agent/agent.py

```
from google.adk.agents import Agent
root_agent = Agent(
    name="book_agent",
    model="gemini-2.0-flash",
    instruction="""사용자의 베스트셀러에 관한 질문에 답하세요.
    'get_best_sellers' 도구를 사용해서 베스트셀러를 검색할 수 있습니다.
    """,
    tools=[get_best_sellers]
)
```


T22. book_agent

book_agent/qdrant.py

```
import json

with open("best-seller-books.json", "r", encoding="utf-8") as f:
    books = json.load(f)

# pip install sentence-transformers hf_xet
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("Qwen/Qwen3-Embedding-0.6B")

from qdrant_client.models import PointStruct
points = [PointStruct(id=idx+1, vector=model.encode(book["description"]).tolist(),
                    payload=book) for idx, book in enumerate(books)]
```

Embedding

<https://huggingface.co/spaces/mteb/leaderboard>

■ MTEB: Massive Text Embedding Benchmark

Rank (Bo...	Model	Zero-shot	Memory U...	Number of P...	Embedding D...	Max Tokens	Mean (T...	Mean (TaskT...
1	gemini-embedding-001	99%	Unknown	Unknown	3072	2048	68.37	59.59
2	Qwen3-Embedding-8B	99%	28866	7B	4096	32768	70.58	61.69
3	Qwen3-Embedding-4B	99%	15341	4B	2560	32768	69.45	60.86
4	Qwen3-Embedding-0.6B	99%	2272	595M	1024	32768	64.34	56.01
5	Linq-Embed-Mistral	99%	13563	7B	4096	32768	61.47	54.14

T22. book_agent

book_agent/qdrant.py

```
from qdrant_client import QdrantClient
client = QdrantClient() # default: localhost:6333

from qdrant_client.models import VectorParams, Distance
collection_name="books"
client.recreate_collection(collection_name=collection_name,
                           vectors_config=VectorParams(size=model.get_sentence_embedding_dimension(),
                                                         distance=Distance.COSINE)
                           )

client.upsert(collection_name=collection_name, points=points)
```

T22. book_agent

book_agent/qdrant.py

```
near_points = client.query_points(  
    collection_name="books",  
    query=model.encode("""역사와 인간의 본질을 다룬 충격적이고 도발적이라고 평가된  
        한강의 소설에 대해 알려줘""").tolist(),  
    limit=3  
)  
  
for point in near_points.points:  
    print(point.payload['title'], point.payload['author'], point.score)
```

T22. book_agent

book_agent/agent.py

```
def search_books(query: str, top_k: int = 3):
```

```
    """
```

주어진 쿼리에 대해 벡터 유사도 검색을 통해 책을 찾아서 반환합니다.

Args:

query (str): 검색할 쿼리 문자열

top_k (int): 반환할 상위 k개의 책 수 (기본값: 3)

Returns:

List[dict]: 검색된 책의 리스트, 각 책은 딕셔너리 형태로 반환됩니다.

```
    """
```

T22. book_agent

- Filter 기능을 이용해 보자.

book_agent/qdrant.py

```
near_points = client.query_points(  
    collection_name="books",  
    query=model.encode("""역사와 인간의 본질을 다룬 소설에 대해 알려줘""").tolist(),  
    query_filter=Filter(  
        must=[FieldCondition(key="author", match=MatchValue(value="한강"))]  
    ),  
    limit=3  
)
```

T22. book_agent

book_agent/agent.py

```
def search_books(query: str, filter: dict = None, top_k: int = 3):  
    """주어진 쿼리에 대해 벡터 유사도 검색을 통해 책을 찾아서 반환합니다.  
    필터 조건이 주어지면 해당 조건에 맞는 책만 검색합니다.
```

Args:

query (str): 검색할 쿼리 문자열

filter (dict): 필터 조건 (예: {"author": "한강"})

top_k (int): 반환할 상위 k개의 책 수 (기본값: 3)

Returns:

List[dict]: 검색된 책의 리스트, 각 책은 딕셔너리 형태로 반환됩니다.

"""

T22. book_agent

book_agent/agent.py

```
def _parse_filter(filter: dict):  
    """  
    filter dict를 Qdrant Filter 객체로 변환.  
    지원 operator: eq, ne, gt, lt, gte, lte  
  
    Args:  
        filter (dict): 필터 조건 (예: {"price": {"gt": 10000, "lt": 20000}})  
  
    Returns:  
        Filter: Qdrant의 Filter 객체  
    """
```

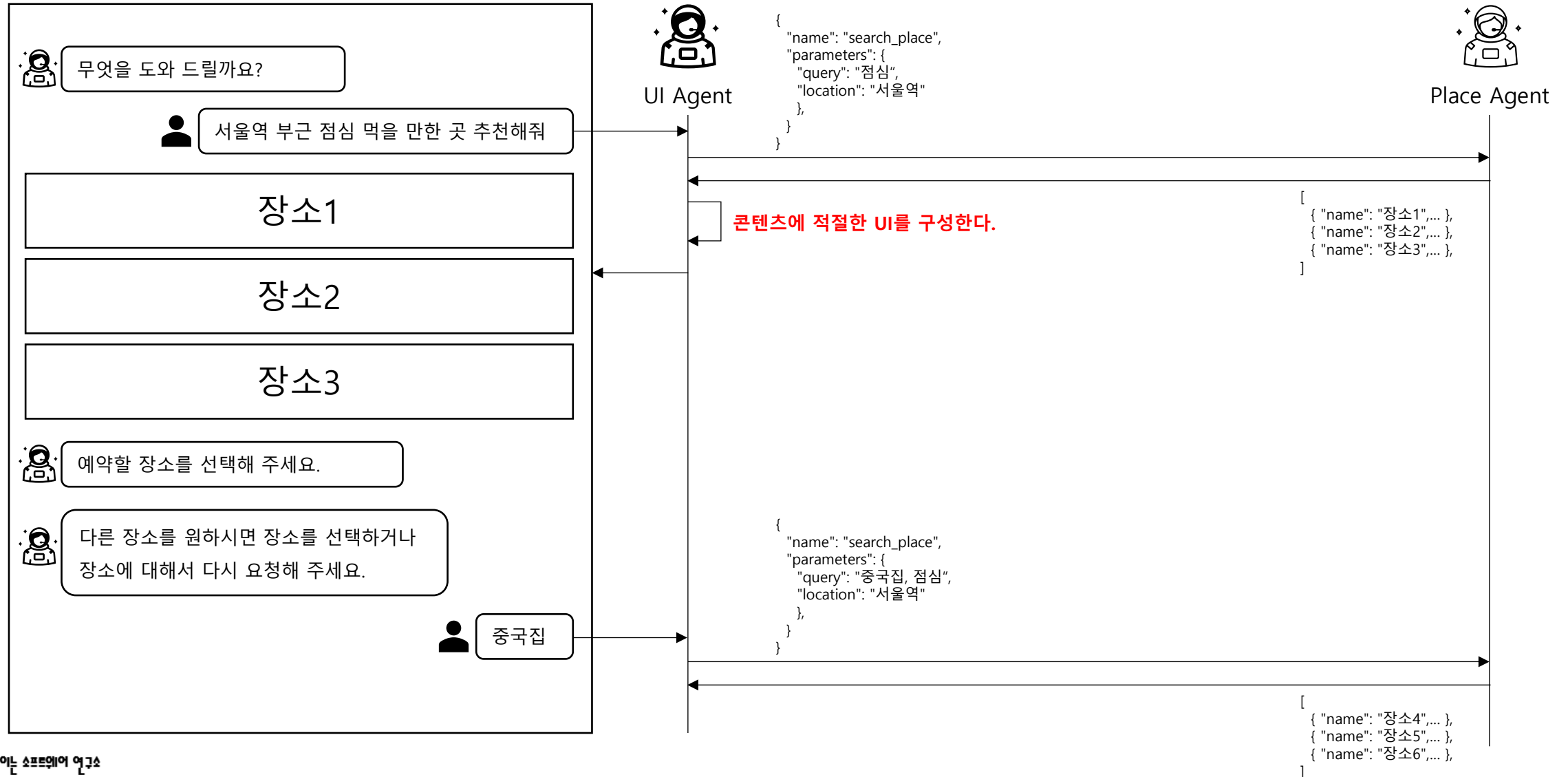

LLMaBoK

Project #2. Restaurant Booking Agent

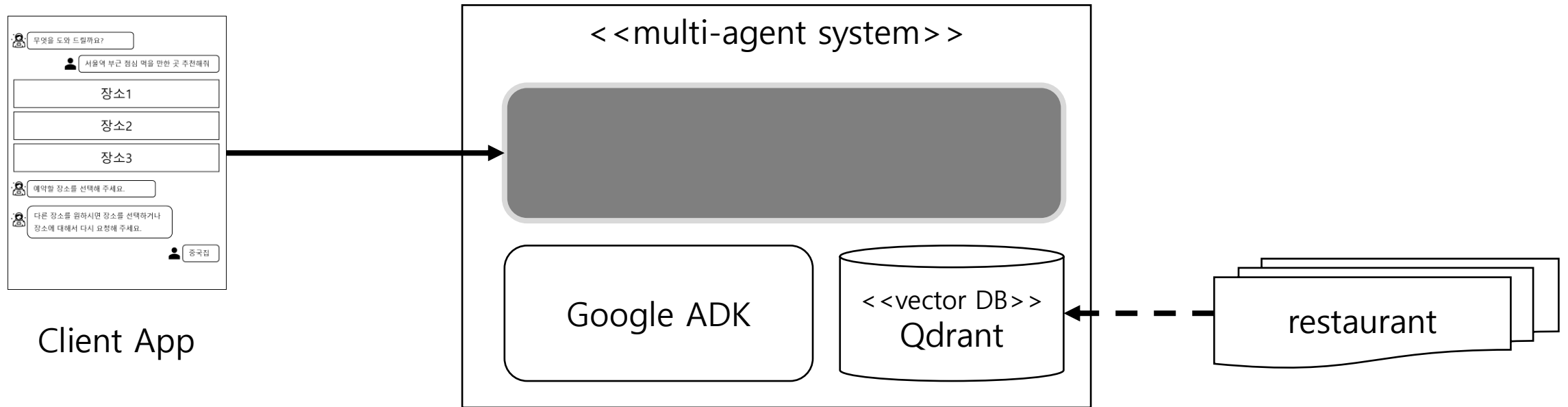
조 용 진

(drajin.cho@bosornd.com)

LLM 앱 - 식당 예약 서비스



LLM 앱 - 식당 예약 서비스



Restaurant Data

<https://github.com/bosornd/book-restaurant-agent>

yelp  Yelp Open Dataset (<https://business.yelp.com/data/resources/open-dataset>)



6,990,280

Reviews



150,346

Businesses



11

Metropolitan areas



200,100

Pictures

Santa Barbara 지역의 Restaurant 759개 추출 ➔ restaurants.json

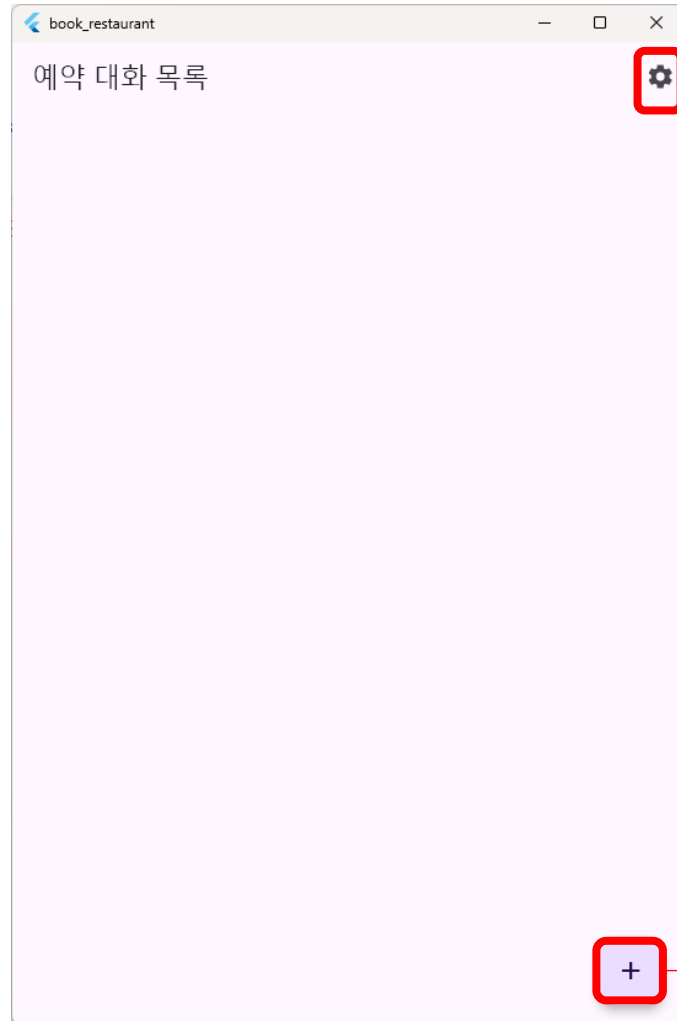
Restaurant Data

<https://github.com/bosornd/book-restaurant-agent>

- **id**: Unique restaurant ID
- **name**: Restaurant name
- **address, city, state, postal_code, location**: Address & location information
- **categories**: Restaurant categories (e.g., 'Italian', 'Mexican', 'Sushi Bars', etc.)
- **attributes**: Various additional properties
 - ambiances: ['casual', 'classy', ...] (atmosphere)
 - good_for_meals: ['breakfast', 'brunch', ...] (meal times)
 - parkings: ['garage', 'lot', ...] (parking options)
 - good_for_kids, corkage, drive_thru, has_tv, dogs_allowed, wifi, good_for_dancing, happy_hour, smoking (true/false)
- **stars**: Average rating (up to 5 stars)
- **review_count**: Total number of reviews
- **reviews**: User-submitted reviews and ratings
- **tips**: User-submitted tips
- **photos**: List of related photos (photo_id matches the filename in the photos folder)

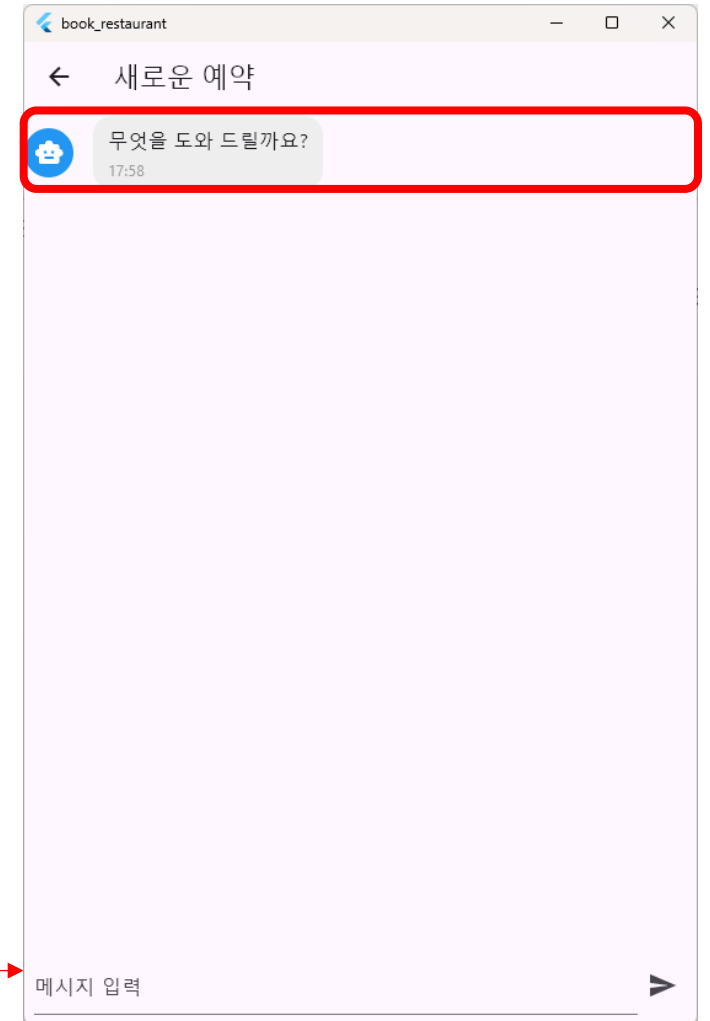
Client App

<https://github.com/bosornd/book-restaurant-agent>



→ 서버 설정

→ 새로운 대화 생성

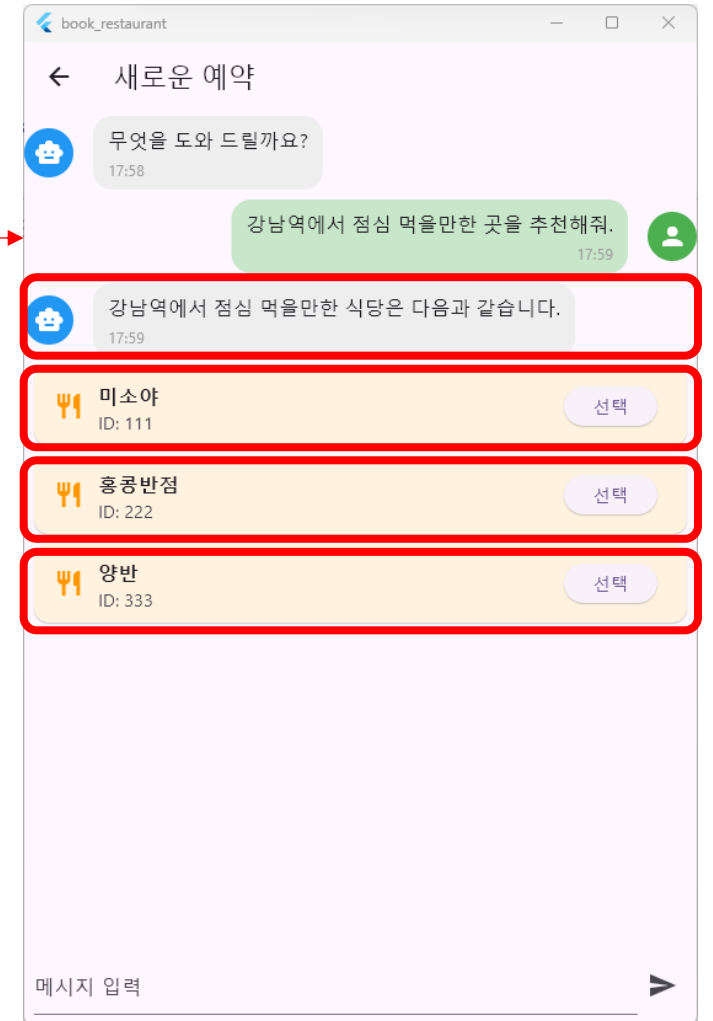
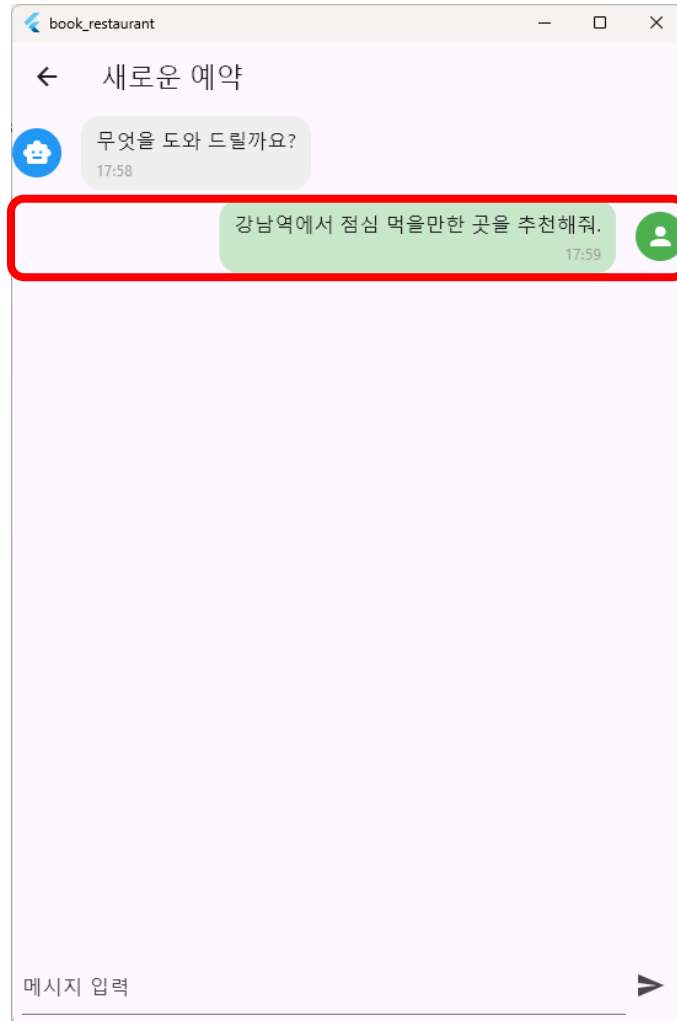


메시지 입력



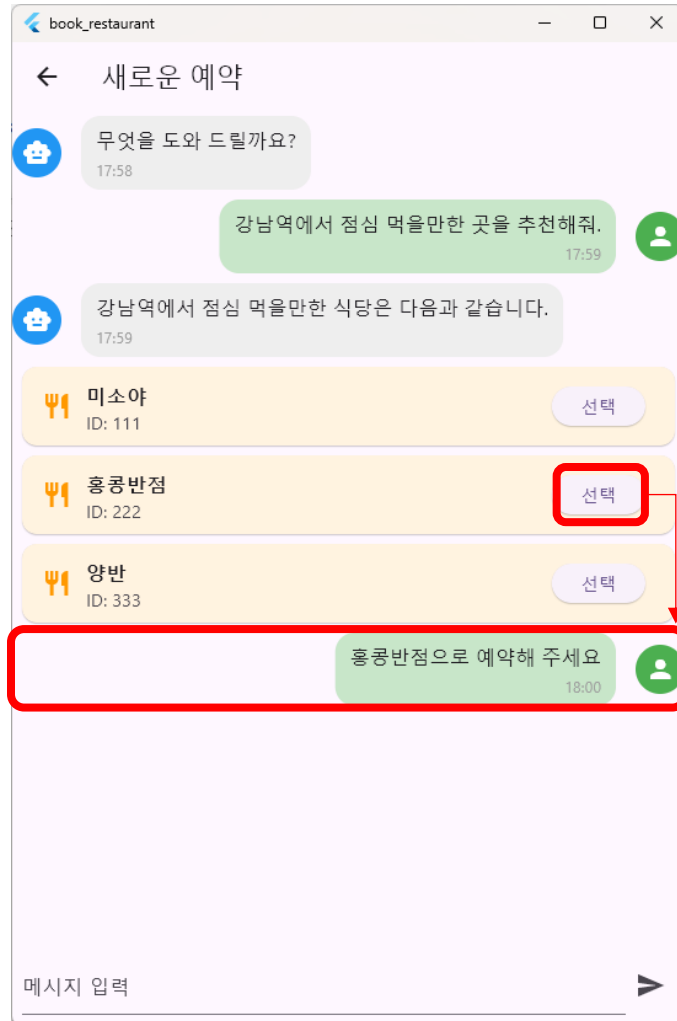
Client App

<https://github.com/bosornd/book-restaurant-agent>



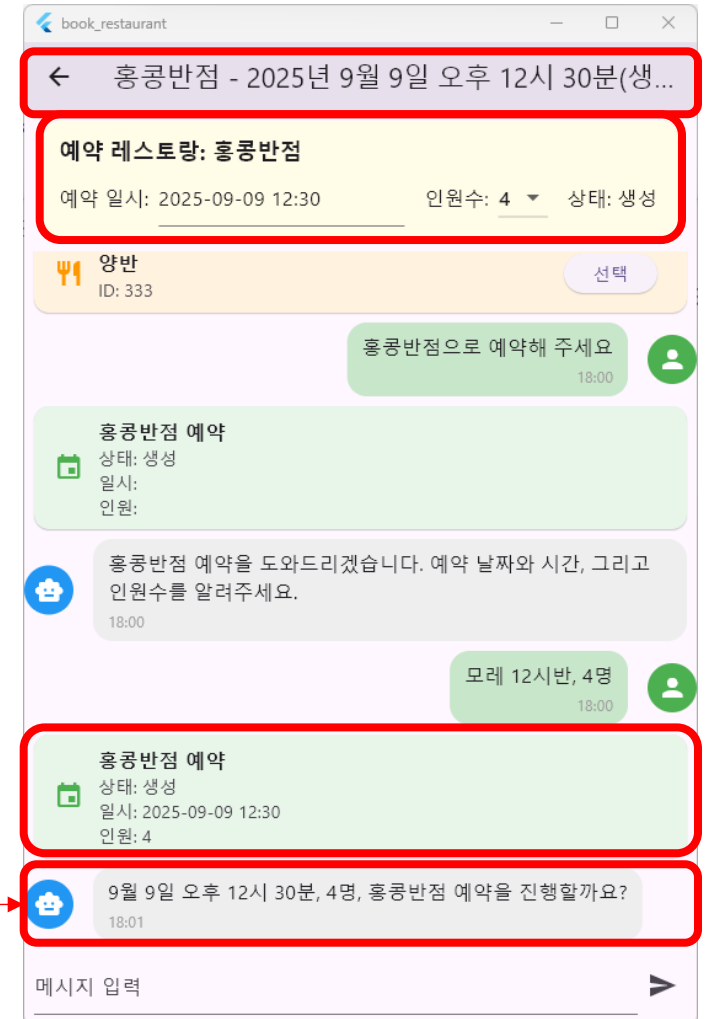
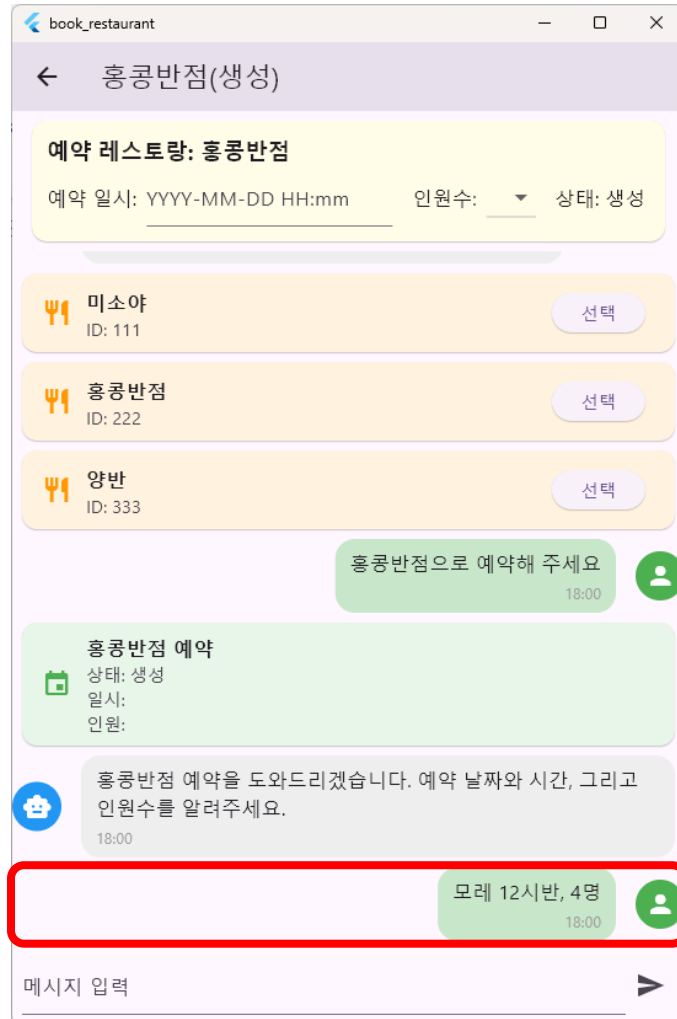
Client App

<https://github.com/bosornd/book-restaurant-agent>



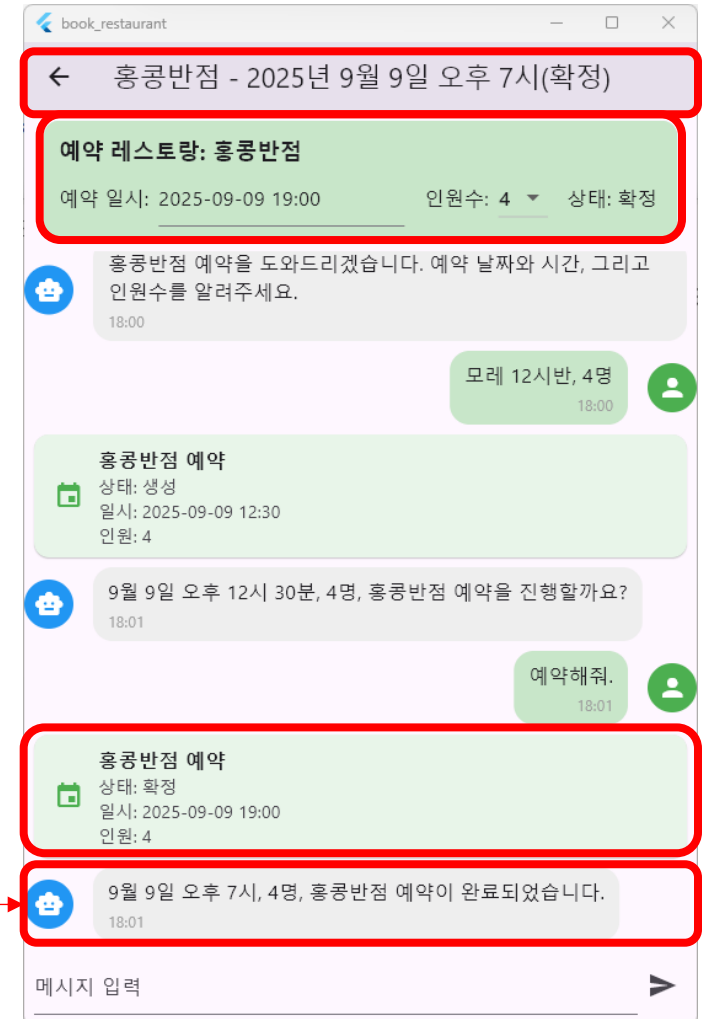
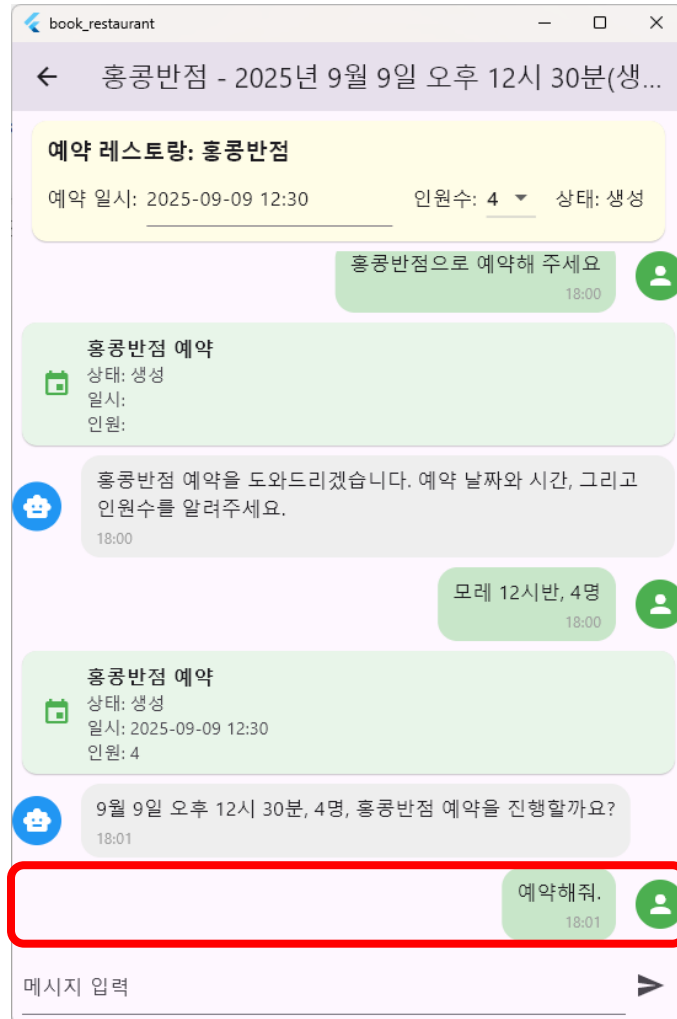
Client App

<https://github.com/bosornd/book-restaurant-agent>



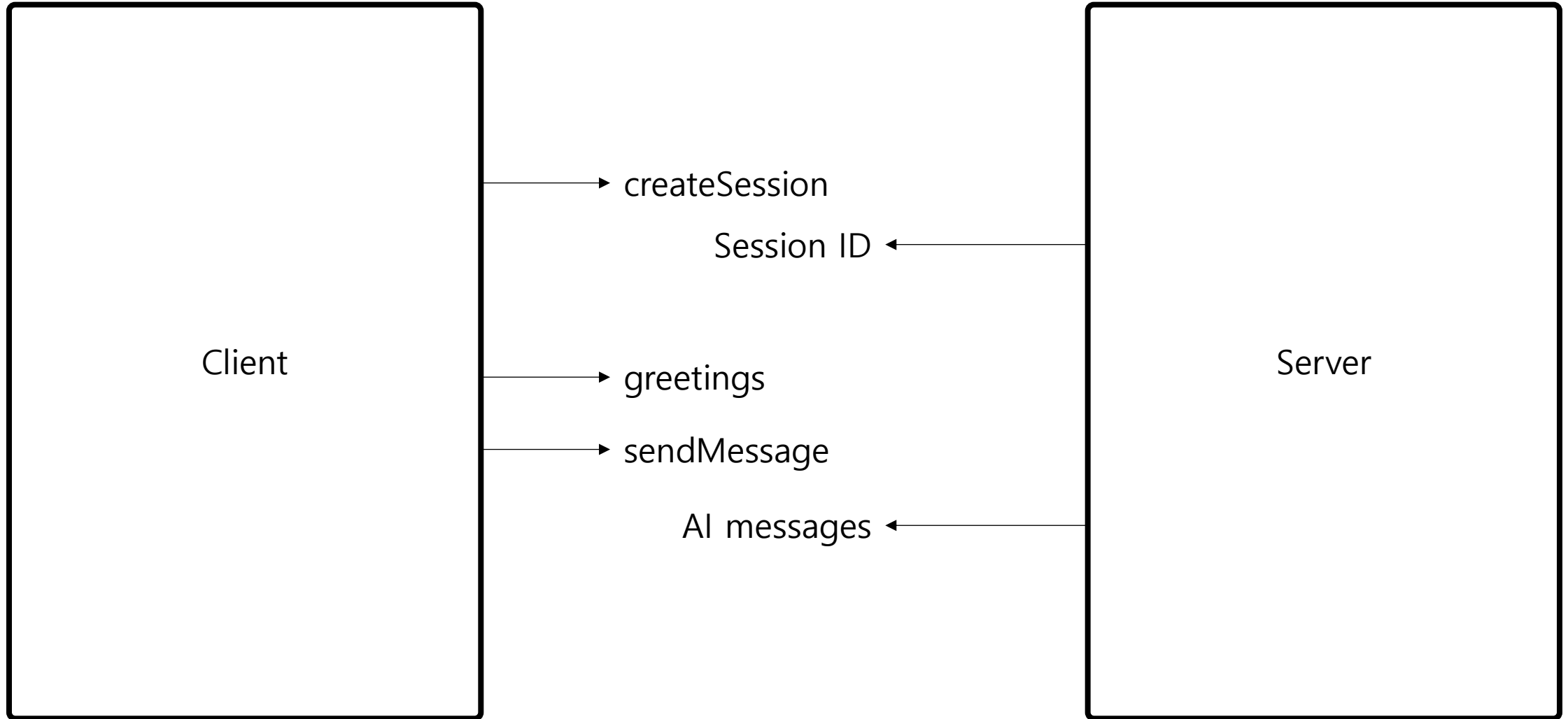
Client App

<https://github.com/bosornd/book-restaurant-agent>



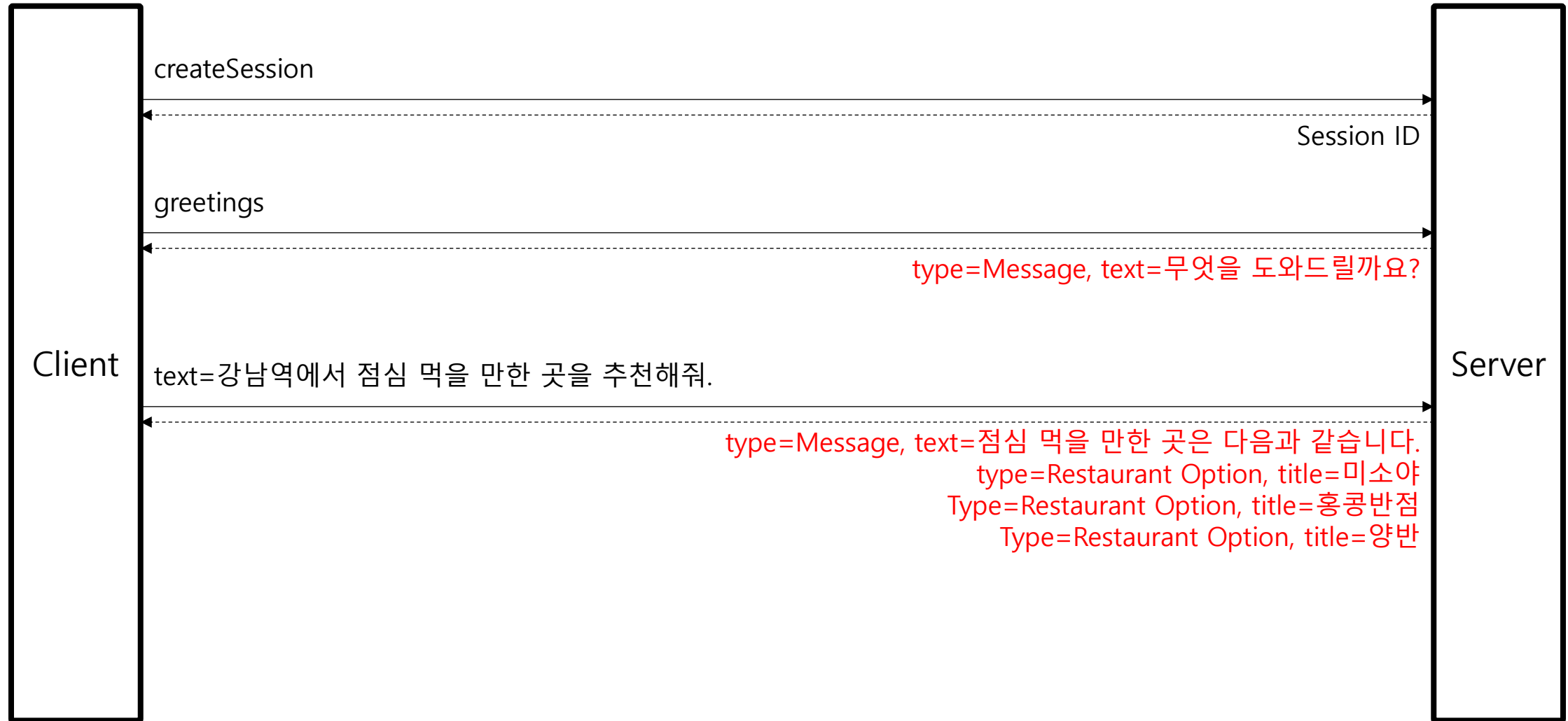
Client App

<https://github.com/bosornd/book-restaurant-agent>



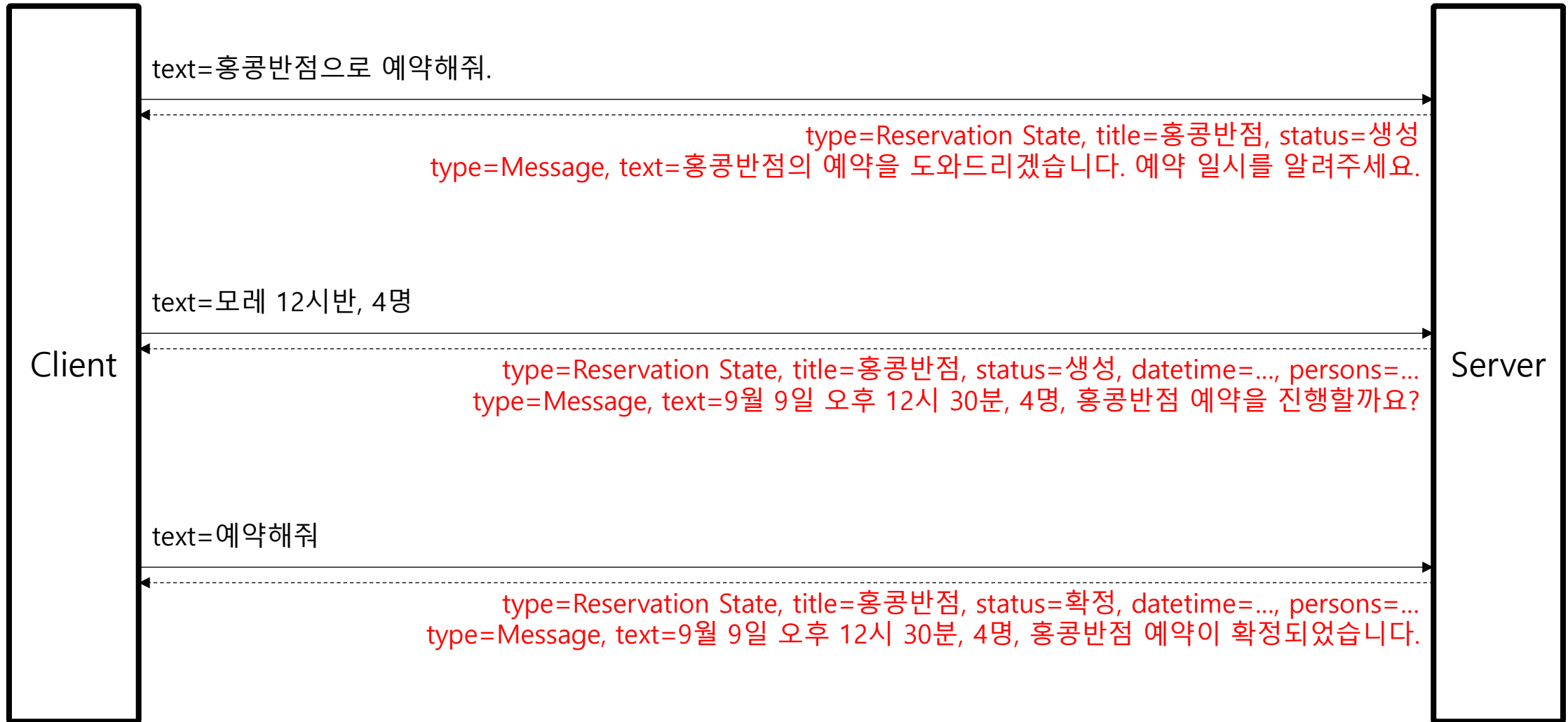
Client App

<https://github.com/bosornd/book-restaurant-agent>



Client App

<https://github.com/bosornd/book-restaurant-agent>



Test Server

<https://github.com/bosornd/book-restaurant-agent>

server/test_server.py

```
# pip install fastapi uvicorn
from fastapi import FastAPI, Request, HTTPException
import uuid

app = FastAPI()

@app.post("/session")          # localhost:5000/session
async def create_session():
    session_id = str(uuid.uuid4())
    return {"session_id": session_id}
```

Test Server

<https://github.com/bosornd/book-restaurant-agent>

server/test_server.py

```
@app.post("/greetings")          # localhost:5000/greetings
async def greetings(request: Request):
    data = await request.json()
    session_id = data.get("session_id")

    greet_msg = "무엇을 도와 드릴까요?"

    return {"text": greet_msg, "session_id": session_id}
```

Test Server

<https://github.com/bosornd/book-restaurant-agent>

server/test_server.py

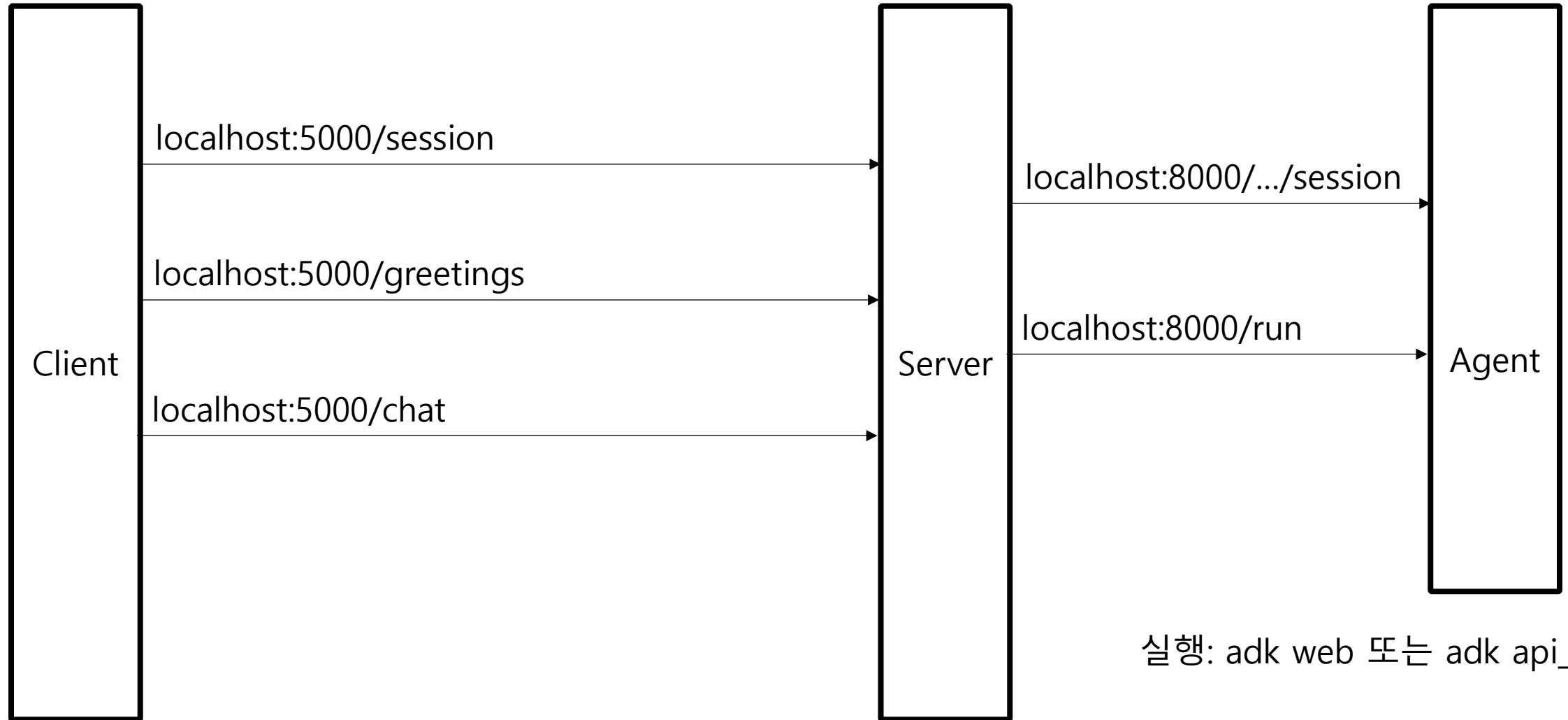
```
@app.post("/chat")                # localhost:5000/chat
async def chat(request: Request):
    data = await request.json()
    session_id = data.get("session_id")
    user_message = data.get("text", "")

    print(f"[{session_id}] User: {user_message}")
    reply = input(f"[{session_id}] Agent: ")

    return {"text": reply, "session_id": session_id}
```


Client → Server → Agent

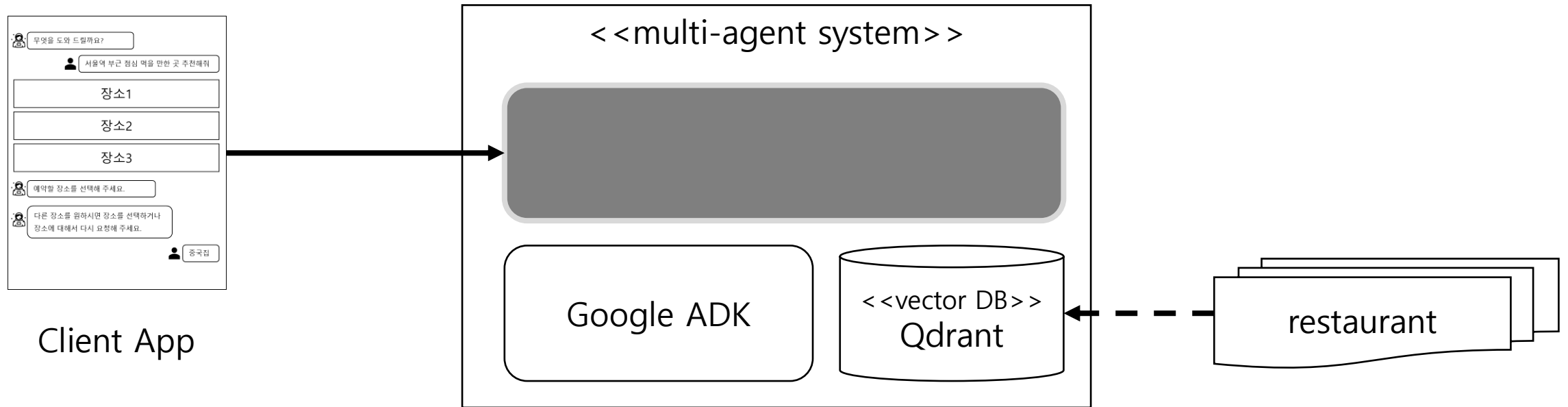
<https://github.com/bosornd/book-restaurant-agent>



실행: `adk web` 또는 `adk api_server`

실행: `uvicorn server:app --host 0.0.0.0 --port 5000`

LLM 앱 - 식당 예약 서비스



실습 1. Qdrant DB 구축하기

- restaurants.json 파일로부터 Qdrant DB를 구축한다.
 - 유사도 검색을 위한 description을 생성한다.
 - Hugging Face embeddings 모델을 이용해서 collection을 생성한다.
 - restaurants를 검색하고 추천하는 agent를 개발한다.

실습 2. UI 연결하기

- 레스토랑 검색/추천 및 예약 서비스를 제공하는 에이전트를 개발한다.
 - 각각 요청에 대해 해야 할 일을 구분해서 제공한다.
 - . Greetings
 - . Restaurant Recommendation
 - . Restaurant Reservation
 - UI/Server 컴포넌트와 연결해서 동작을 확인한다.

실습 3. MAS 개발하기

- 레스토랑 검색/추천 및 예약 서비스를 MAS로 설계하고 개발한다.
 - 기능별 역할을 세분화 한다.
 - . 사용자의 요청에 대한 처리 계획을 수립한다.
 - . 계획에 따라 업무를 배정한다.
 - . 업무를 수행한다.
 - . 업무 결과를 점검한다.
 - . 업무 결과를 통합한다.

실습 3. 공유하기

- 설계 및 개발 내용을 공유(발표) 한다.
 - Monolithic Agent와 MAS를 비교한다.
 - 추가 개선 사항을 도출해 본다.

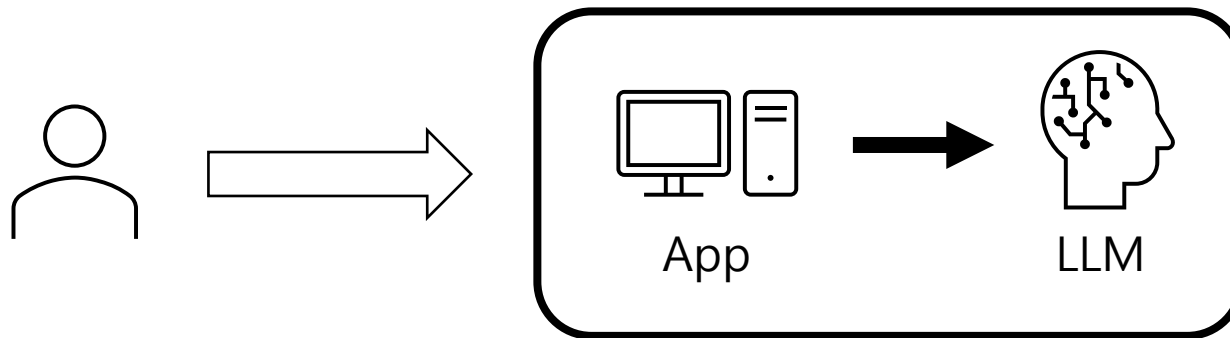
LLMaBoK

Summary

조 용 진

(drajin.cho@bosornd.com)

LLM 앱의 설계 요소



1. LLM 모델

- ◆ 목적에 맞는 성능
- ◆ 비용 및 운영 효율성
- ◆ 인프라 및 보안

2. 프롬프트

- ◆ 명확한 지침 및 역할 부여
- ◆ 출력 가이드 및 파서
- ◆ 프롬프트 엔지니어링

3. 외부 도구 통합

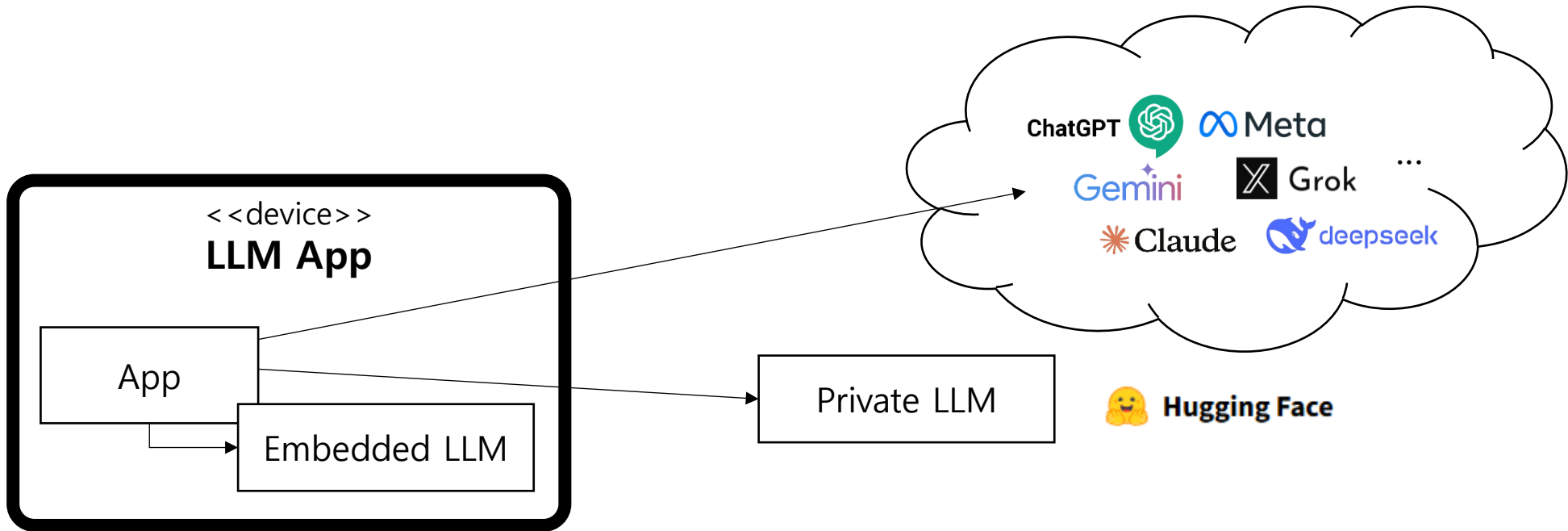
- ◆ RAG
- ◆ Tools & MCP
- ◆ 컨텍스트 엔지니어링

4. 에이전트

- ◆ 복잡한 문제 해결
- ◆ 역할 분담, 협업 및 소통
- ◆ 전문성, 효율성, 유연성

1. LLM App

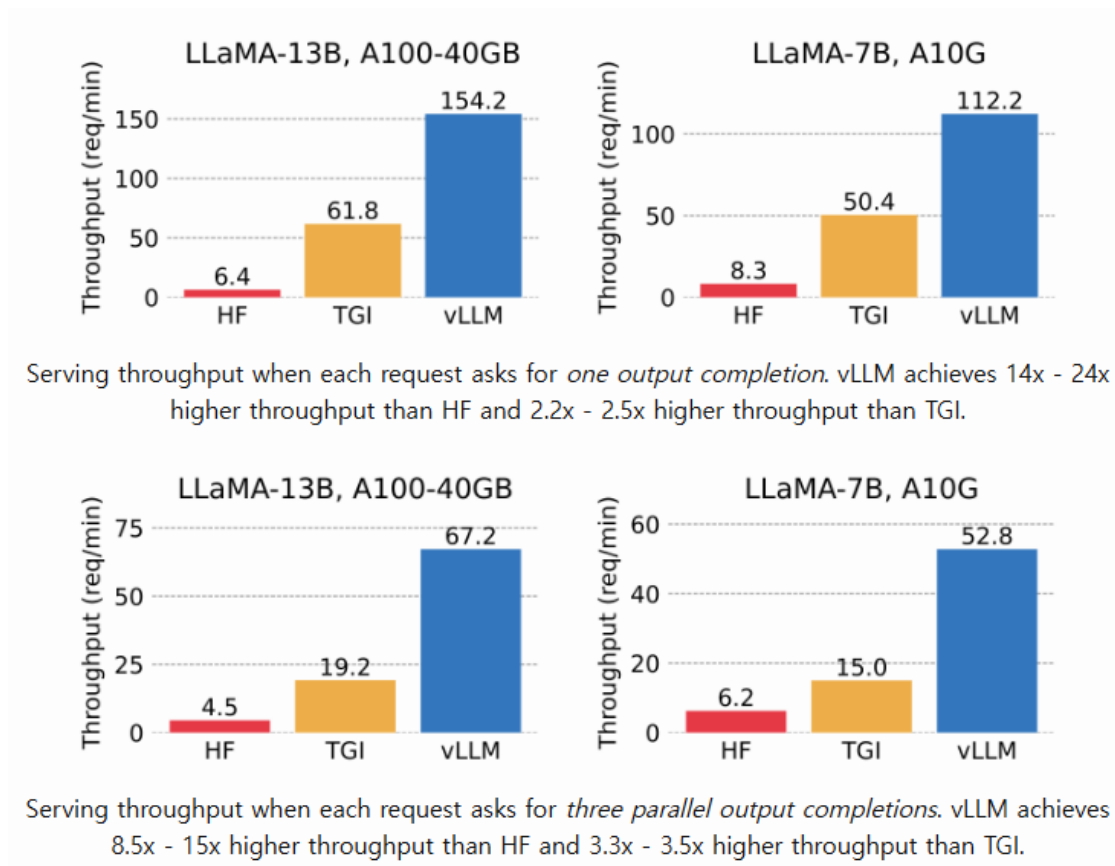
- LLM Deployment
 - Public LLM vs. Private LLM vs. Embedded LLM



1.1 vLLM

<https://docs.vllm.ai>
<https://blog.vllm.ai/2023/06/20/vllm.html>

- vLLM: Easy, Fast, and Cheap LLM Serving with Paged Attention



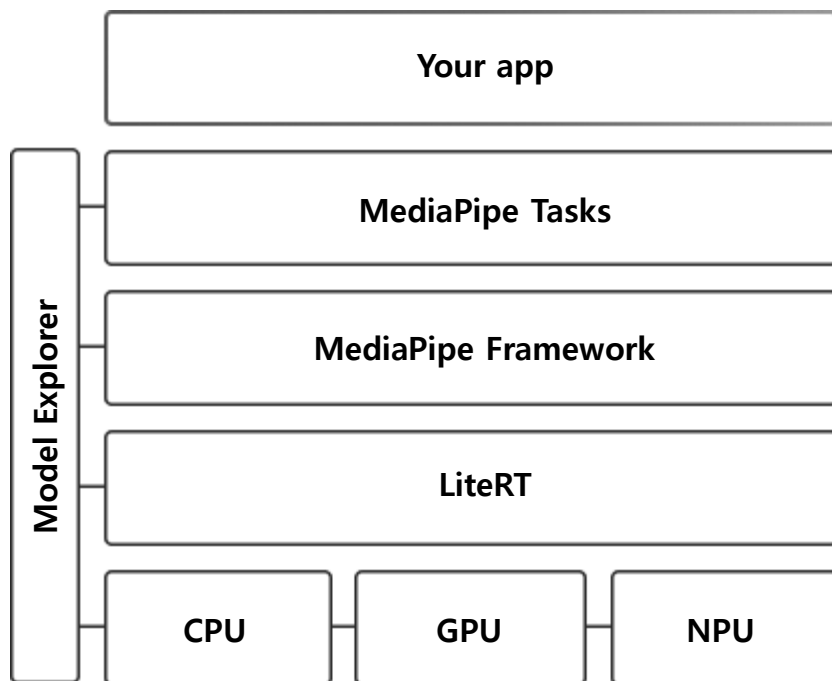
HuggingFace Transformers (HF)

HuggingFace Text Generation Inference (TGI)

1.2 Google AI Edge

<https://ai.google.dev/edge>

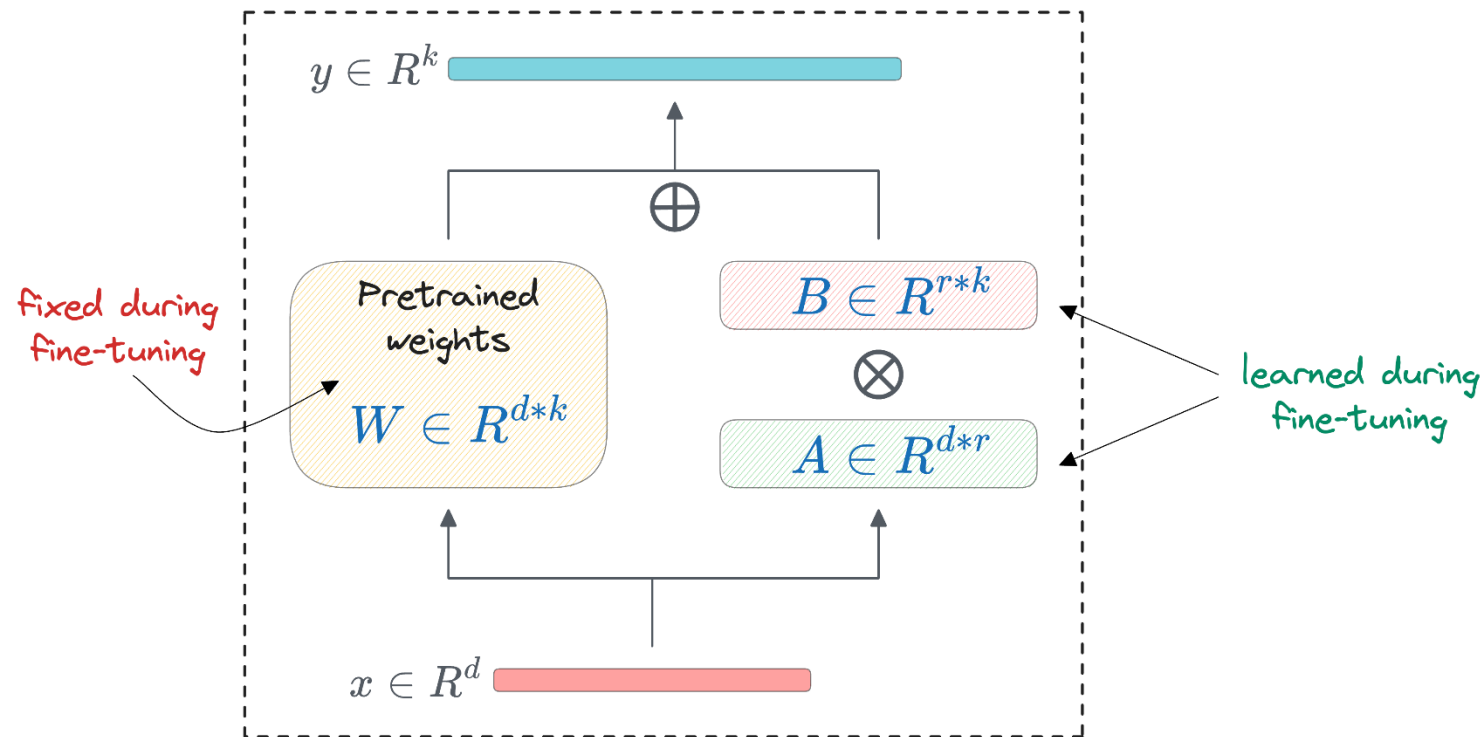
■ LiteRT / MediaPipe Solution



- 인공지능 및 머신러닝 기법을 빠르게 적용할 수 있는 라이브러리 및 도구 모음
- 생성형 AI (LLM 추론, RAG, 도구 등), 오디오/비디오/이미지/텍스트 작업 등
- 효율적인 ODML 파이프라인을 빌드하는 데 사용되는 하위 수준의 구성요소
- 그래프, 노드, I/O 스트림, 패킷
- 지연 시간, 개인 정보 보호, 연결성, 크기, 전원 소모라는 5가지 ODML 제약 조건 해결
- Python, Android용 Java 및 Kotlin, iOS용 Swift, 마이크로 기기용 C++ API 포함

1.3 PEFT: Parameter-Efficient Fine-tuning

- LoRA (Low-Rank Adaptation)
- QLoRA (Quantized LoRA)



2. Prompt Engineering

- Role (역할): 모델의 전문성과 관점 정의
 - 전문 영역 지식의 활성화
 - 응답 톤과 스타일의 설정
- Task (태스크): 수행할 작업의 명확한 정의
 - Instructions (지시사항): 구체적인 수행 방법
 - Constraints (제약사항): 준수해야 할 규칙과 한계
 - Output Format (형식): 출력 구조와 스타일 정의
- Samples (예제): 기대하는 출력의 구체적 예시
- User Input (사용자 입력): 실제 처리할 데이터
 - Contexts (맥락): 작업 수행에 필요한 배경 정보
 - Query (질의): 구체적인 요청 사항

Role	
Instructions	editor
Constraints	and punctuation errors
Output Format	st
Samples	ail text
Context	o schedule meeting
Query	am member
# Query Correct this email for professional communication.	

2.1 Prompt Template

- Prompt의 정형화/일반화 → 재사용

"한국의 수도는? 인구, 면적, 기후 등도 알려줘."

"미국의 수도는? 인구, 면적, 기후 등도 알려줘."

"일본의 수도는? 인구, 면적, 기후 등도 알려줘."

⋮

Prompt Template

"{country}의 수도는? 인구, 면적, 기후 등도 알려줘."

{ "country" : "한국" }

{ "country" : "미국" }

{ "country" : "일본" }

<https://smith.langchain.com/hub>

2.2 Few-shot Prompting

- 몇 가지 예시를 제공해서, 사용자가 원하는 결과의 형식, 스타일, 톤 등을 명확하게 알려주는 프롬프트 엔지니어링 기술

{country}의 수도는?
JSON format으로 출력해줘.

Zero-shot Prompting

{country}의 수도는?
JSON format으로 출력해줘.

예시)

질문: 한국의 수도는?

답변: { "국가": "한국", "수도": "서울" }

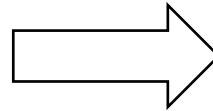
Few-shot Prompting

2.3 CoT (Chain-of-Thinking) Prompt

- 복잡한 문제를 해결하기 위해
사고 과정을 단계별로 보여주도록 유도하는 프롬프트 엔지니어링 기술

사람 한명이 말을 끌고, 사람 2명이 말 위에
있어요. 그럼 다리가 모두 몇 개일까요?

계산 과정을 설명해주세요.



사람이 3명이고, 말이 1마리.

사람 1명당 다리 2개이고,

3명이므로 $3 \times 2 = 6$ 개.

말이 1마리이므로 다리 4개.

$3 \times 2 + 1 \times 4 = 6 + 4 = 10$

그러므로 총 다리 수는 10개입니다.

CoT Prompting

2.4 Structured Output

- 사전에 정의된 특정 형식에 맞춰서 결과를 생성하도록 하는 기능
- LLM을 프로그램의 한 부분(API)처럼 활용할 수 있게 해주는 핵심 기능

{country}의 수도는?
JSON format으로 출력해줘.

예시)

질문: 한국의 수도는?

답변: { "country": "한국", "capital": "서울" }

Structured Output

Output Parser

```
from pydantic import BaseModel, Field
class Capital(BaseModel):
    country: str = Field(
        description="국가")
    capital: str = Field(
        description="수도")
```

3. Context Engineering

Context Poisoning (오염)

Context 내에 잘못된 정보(환각, 오류)가 유입되어 모델이 이를 사실처럼 반복적으로 참조하거나 생성하는 현상

Context Distraction (방해)

Context 내의 특정 정보에 모델이 과도하게 집중하여 학습된 지식이나 더 중요한 정보를 간과하는 현상

Context Confusion (혼란)

Context 내에 불필요하거나 너무 많은 정보가 포함되어 모델이 핵심을 파악하지 못하고 저품질 응답을 생성하는 현상

Context Clash (충돌)

Context 내에 상충하거나 모순되는 정보가 동시에 존재하여 모델이 혼란스러워 하는 현상

3. Context Engineering



RAG (Retrieval Augmented Generation)

LLM의 학습 데이터 외부에 있는 최신, 정확한 정보를 검색하여 Context에 추가하는 방식입니다. Context 오염을 줄이고, 모델이 특정 지식에 집중하도록 도와 Context Distraction을 완화합니다.



Tool Loadout

LLM에 필요한 도구를 사용 시점에 맞춰 동적으로 로드하고, 불필요한 도구는 제거하는 방식입니다. Context Confusion을 줄이고, 모델이 더 효율적으로 올바른 도구를 선택하게 도와줍니다.

3. Context Engineering



Context Quarantine

Context 내의 특정 정보를 독립적인 격리 구역에 보관하여 필요할 때만 LLM에 노출하는 방식입니다.



Context Summarization

긴 Context 내용을 핵심만 추출하여 요약된 형태로 LLM에 제공하는 방식입니다.



Context Pruning

Context 내에서 불필요하거나 중복되는 정보를 제거하고 핵심 정보만 남기는 방식입니다.

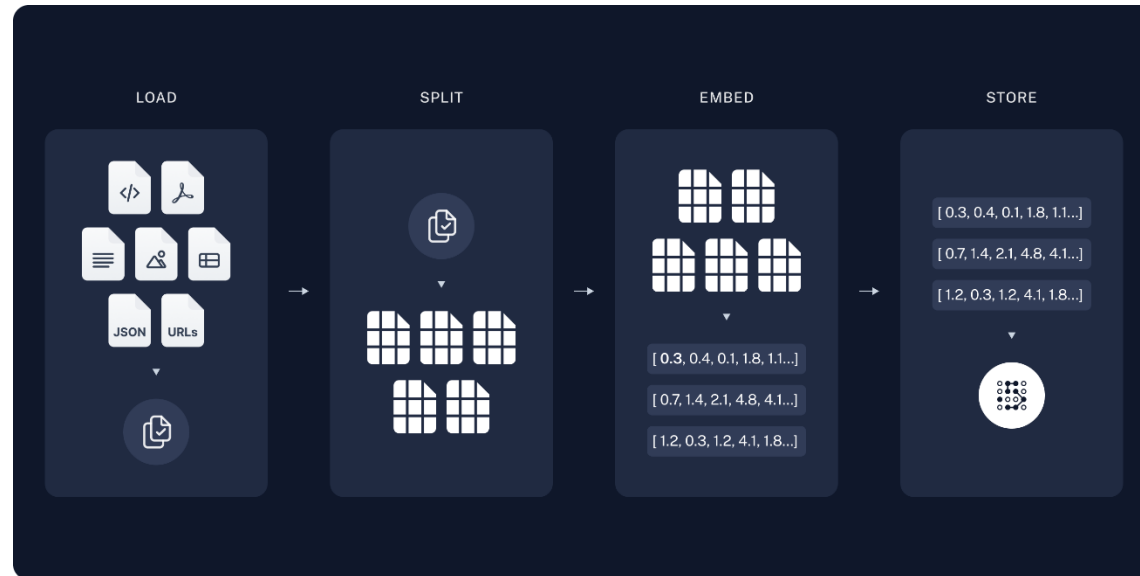


Context Offloading

현재 대화에 필요하지 않은 Context를 외부 저장소로 옮겨두고, 필요할 때 다시 불러오는 방식입니다.

3.1 RAG

- A typical RAG application has two main components:
 - **Indexing**: a pipeline for ingesting data from a source and indexing it.

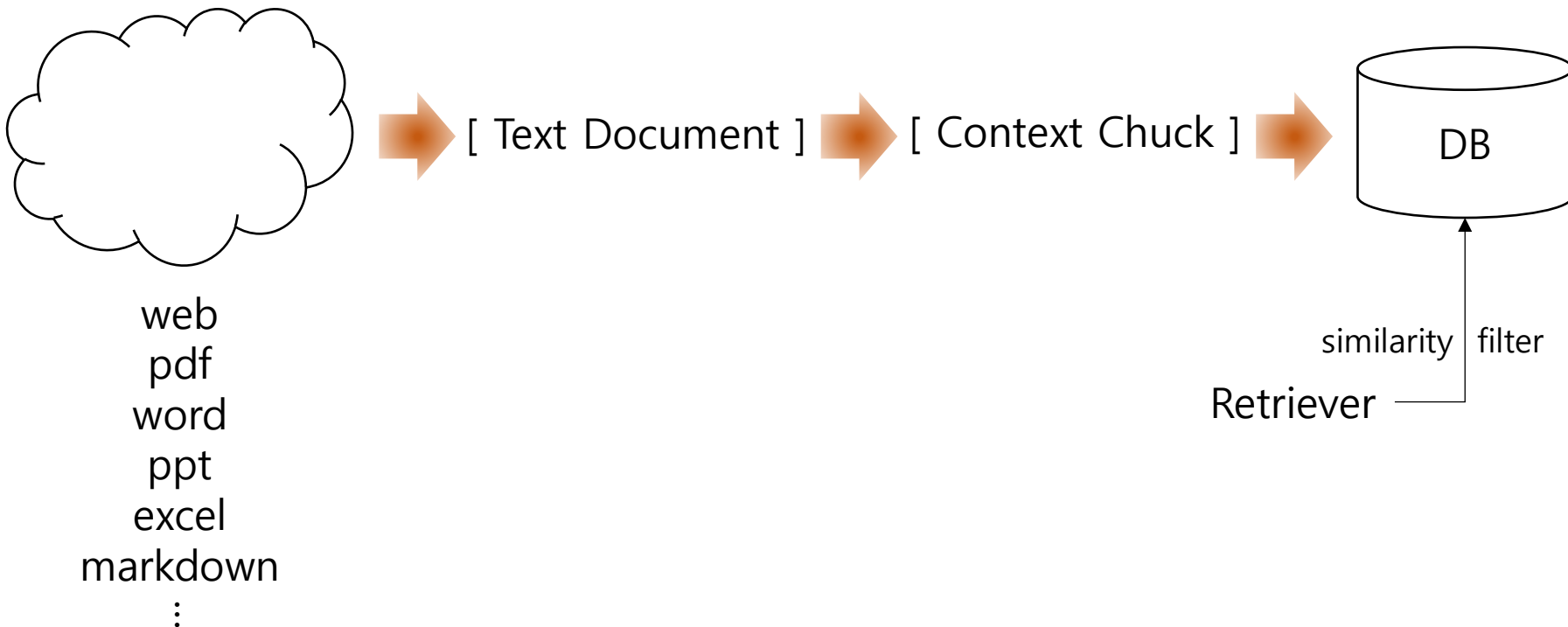


- Step 1. Load Document
- Step 2. Split Document
- Step 3. Embed Vector
- Step 4. Store at Vector DB

- **Retrieval and generation**: the actual RAG chain, which takes the user query at run time and retrieves the relevant data from the index, then passes that to the model.

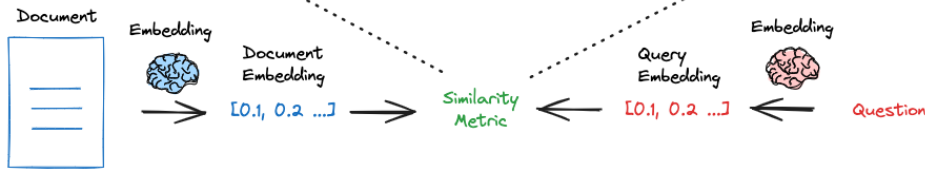
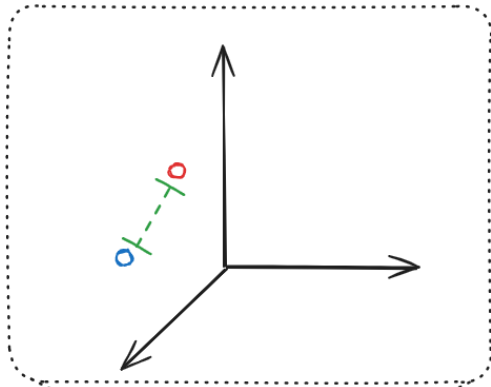
3.1.1 Data Indexing

- Collect the information required for the question and store it in DB.
- Retrieve the information as context from DB and provide it to LLM.



3.1.2 Embeddings

2 Provides a way to measure similarity between texts



1 Embed text as a vector

MTEB: Massive Text Embedding Benchmark

Multilingual	Rank (Bo...	Model
English	1	multilingual-e5-large-instruct
Image	2	jina-embeddings-v3
Domain-Specific	3	snowflake-arctic-embed-l-v2.0
Language-specific	4	gte-multilingual-base
European	5	Solon-embeddings-large-0.1
Indic		
Scandinavian		
Chinese		
German		
French		
Japanese		
Korean		

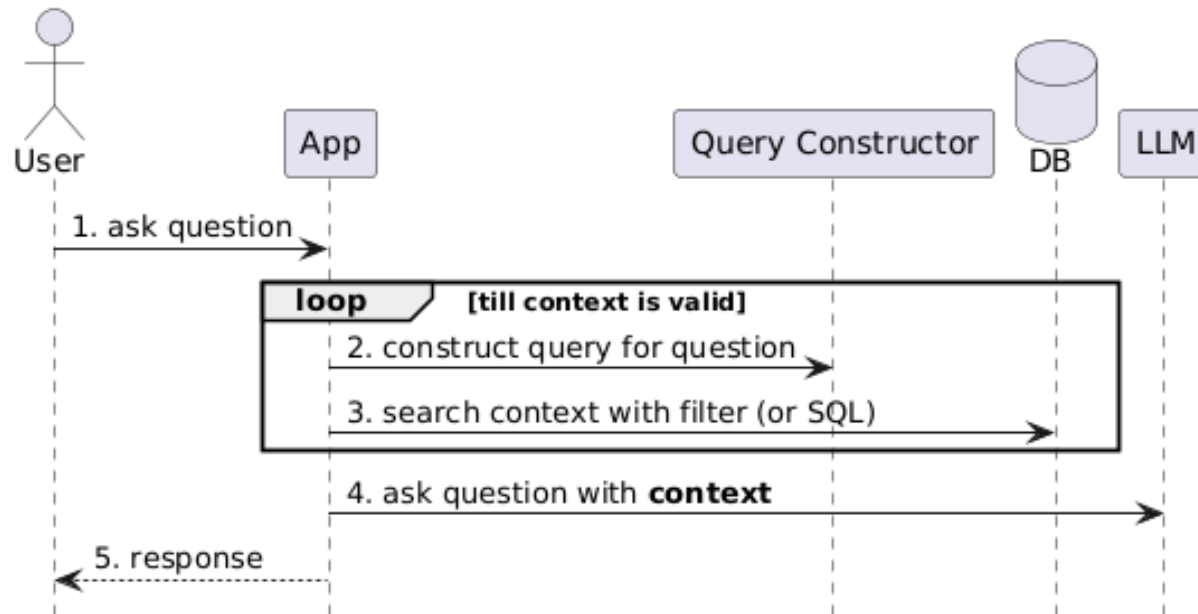
3.1.4 Vector DBs

https://en.wikipedia.org/wiki/Vector_database

- 어떤 Vector DB를 채택할 것인가
 - 빠르게 시작하고 싶고 인프라 운영이 부담스럽다면: Chroma
 - 대규모 데이터를 다루는 엔터프라이즈급 솔루션을 원한다면: Milvus
 - 복잡한 필터링 기능이 중요하고 성능도 놓칠 수 없다면: Qdrant
 - ⋮

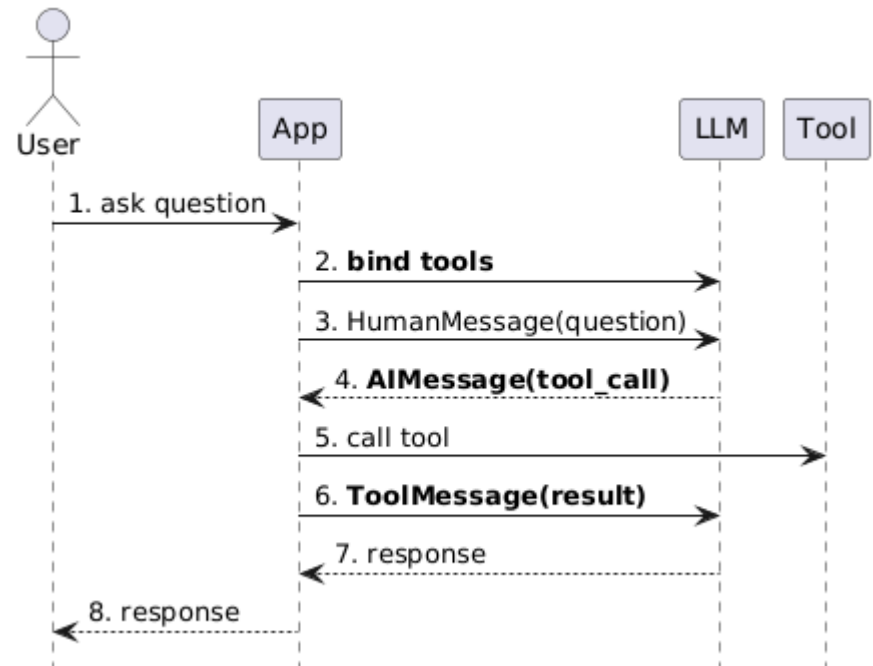
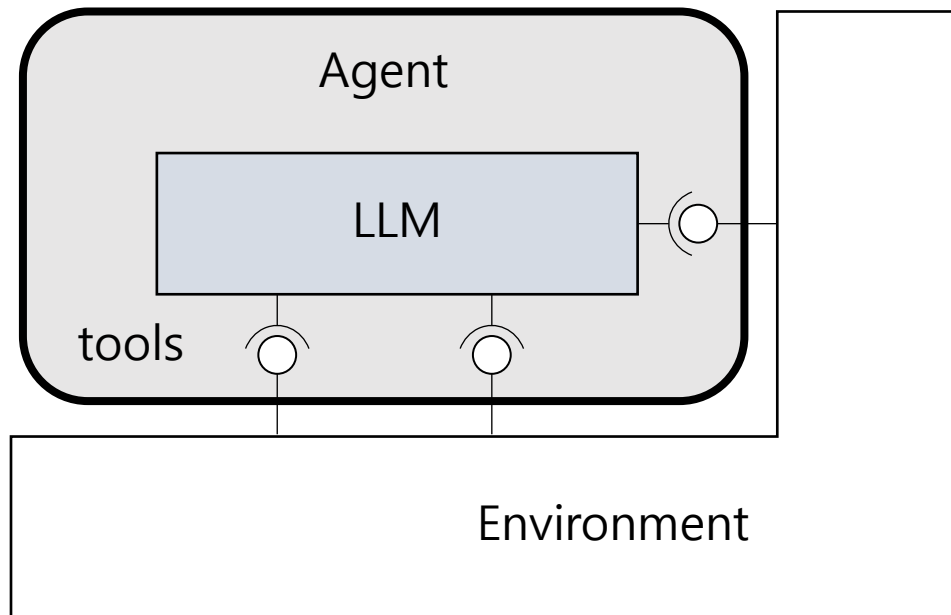
3.1.5 Query Constructor

- Similarity search with filters
- SQL query



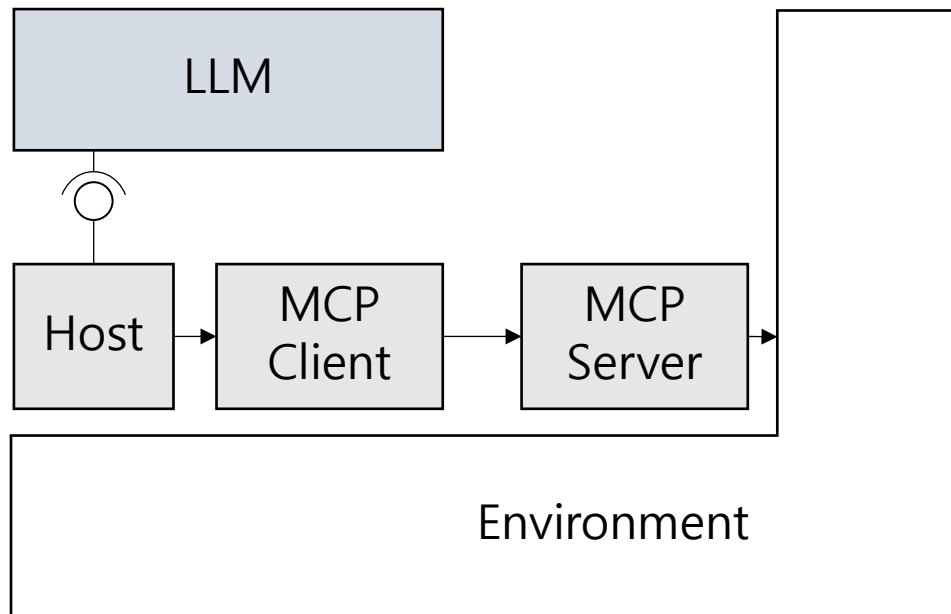
3.2 Tool

- LLM이 환경과 상호 작용할 수 있는 도구를 제공한다.
 - 답변을 위해 필요한 정보를 획득하기 위해 도구를 사용한다.
 - 요청을 처리하기 위해 도구를 사용한다.



3.3 MCP (Model Context Protocol)

- LLM이 환경과 상호 작용할 수 있는 범용 포트(프로토콜) 제공한다.
 - Resource, Prompt, Tool, etc.



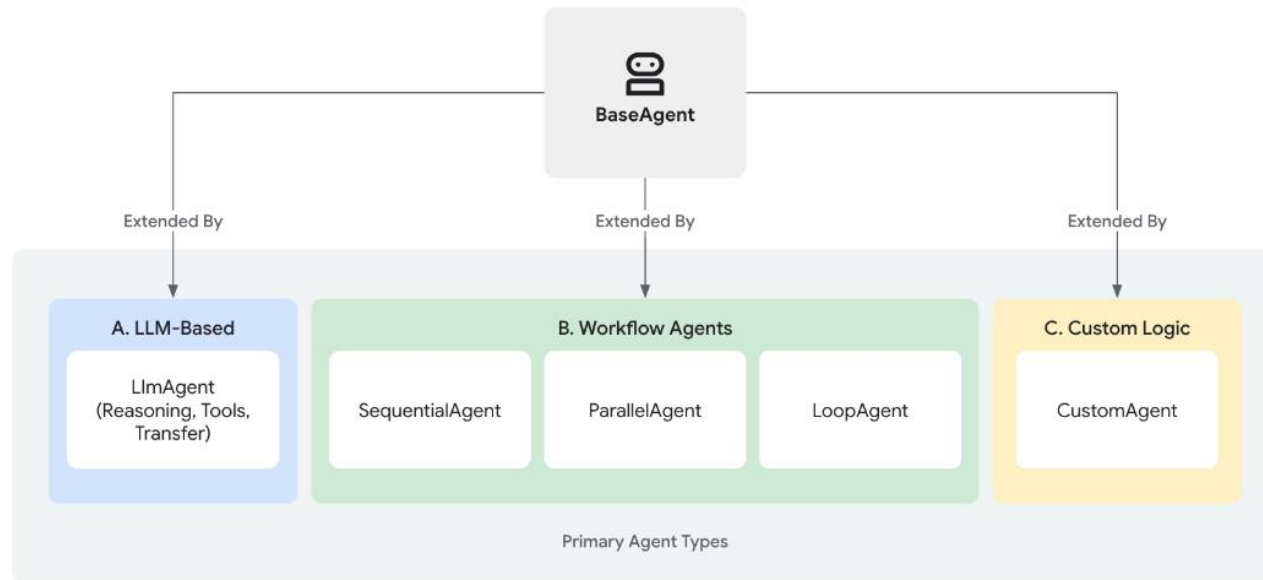
MCP Protocol

Transport: STDIO or Streamable HTTP

Message: JSON-RPC 2.0

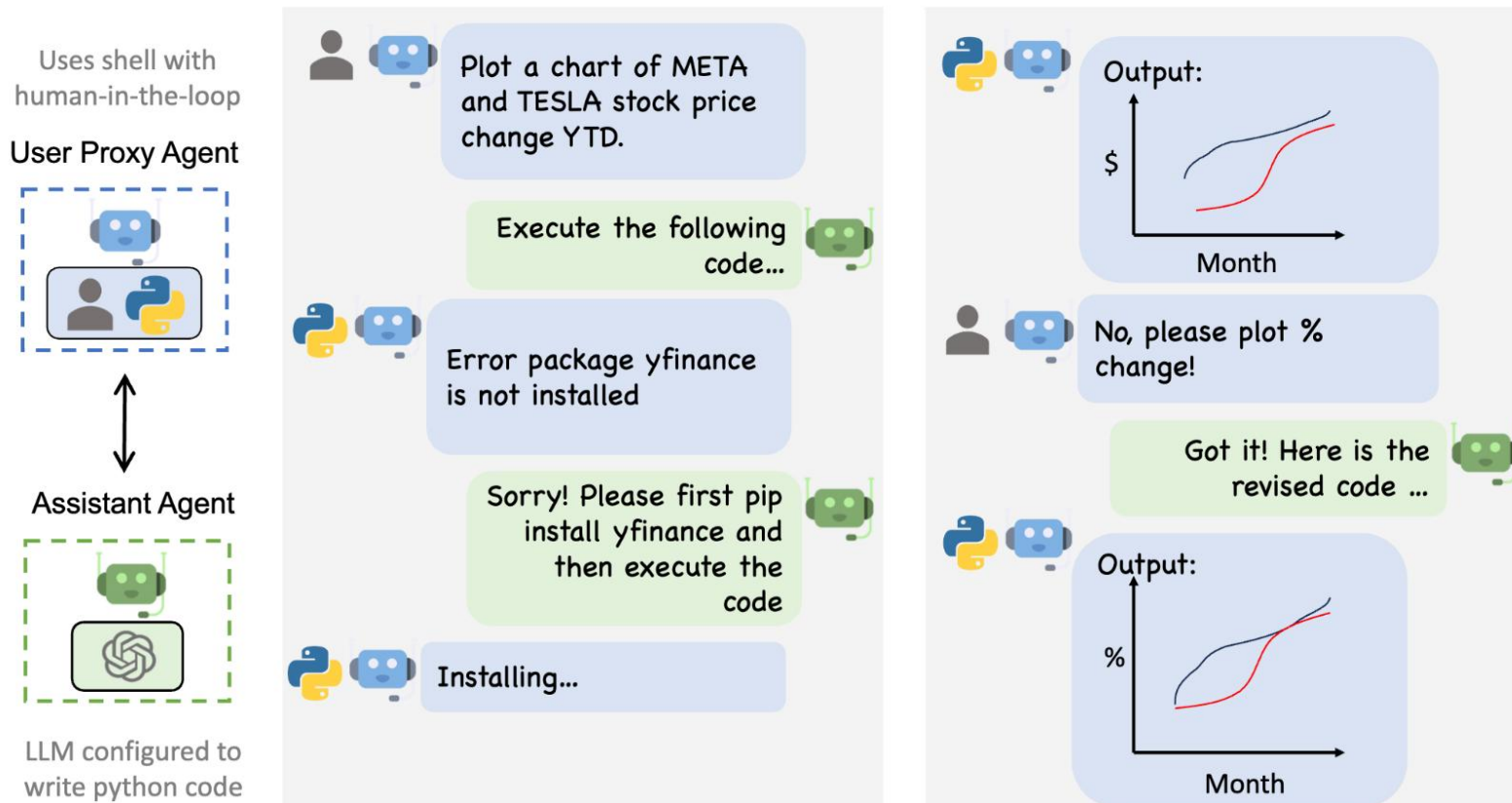
4. 에이전트

- Google ADK – <https://google.github.io/adk-docs>
 - An Agent is a self-contained execution unit designed to act autonomously to achieve specific goals.
 - Agents can perform tasks, interact with users, utilize external tools, and coordinate with other agents.



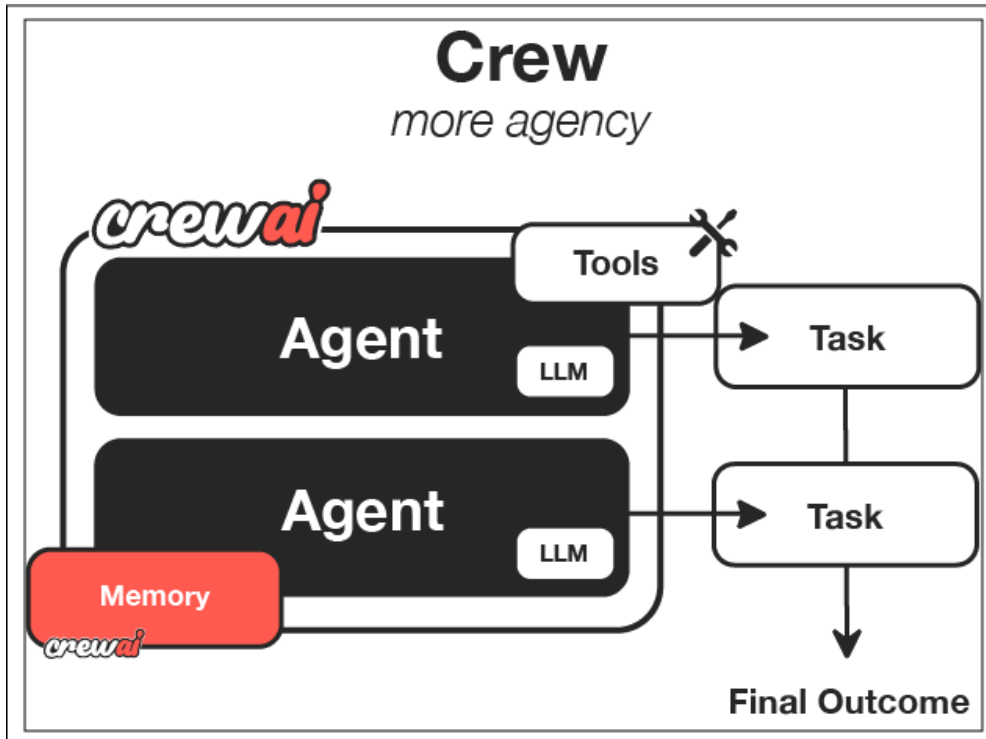
4.1 에이전트 프레임워크

- Google ADK – <https://google.github.io/adk-docs>
- Microsoft AutoGen – Multi-Agent Conversation Framework



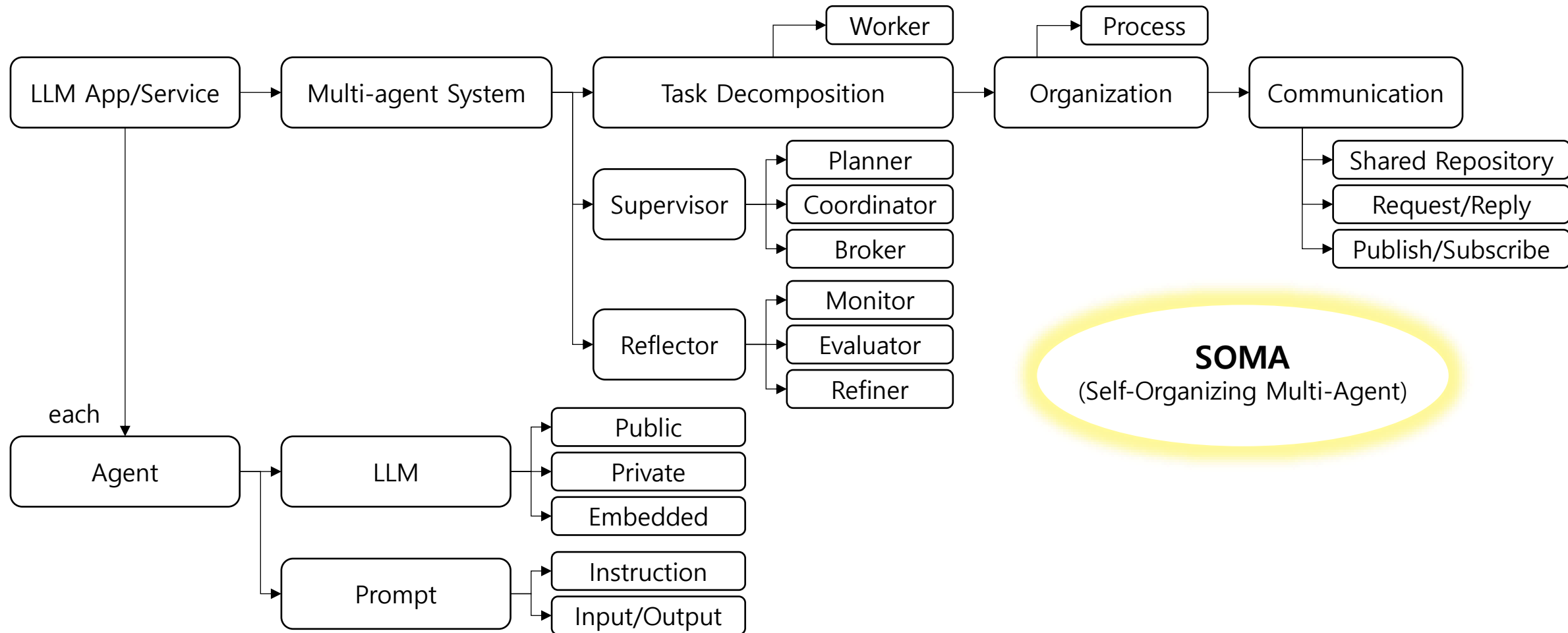
4.1 에이전트 프레임워크

- Google ADK – <https://google.github.io/adk-docs>
- Microsoft AutoGen – Multi-Agent Conversation Framework
- CrewAI – Build AI agent teams that work together to tackle complex tasks

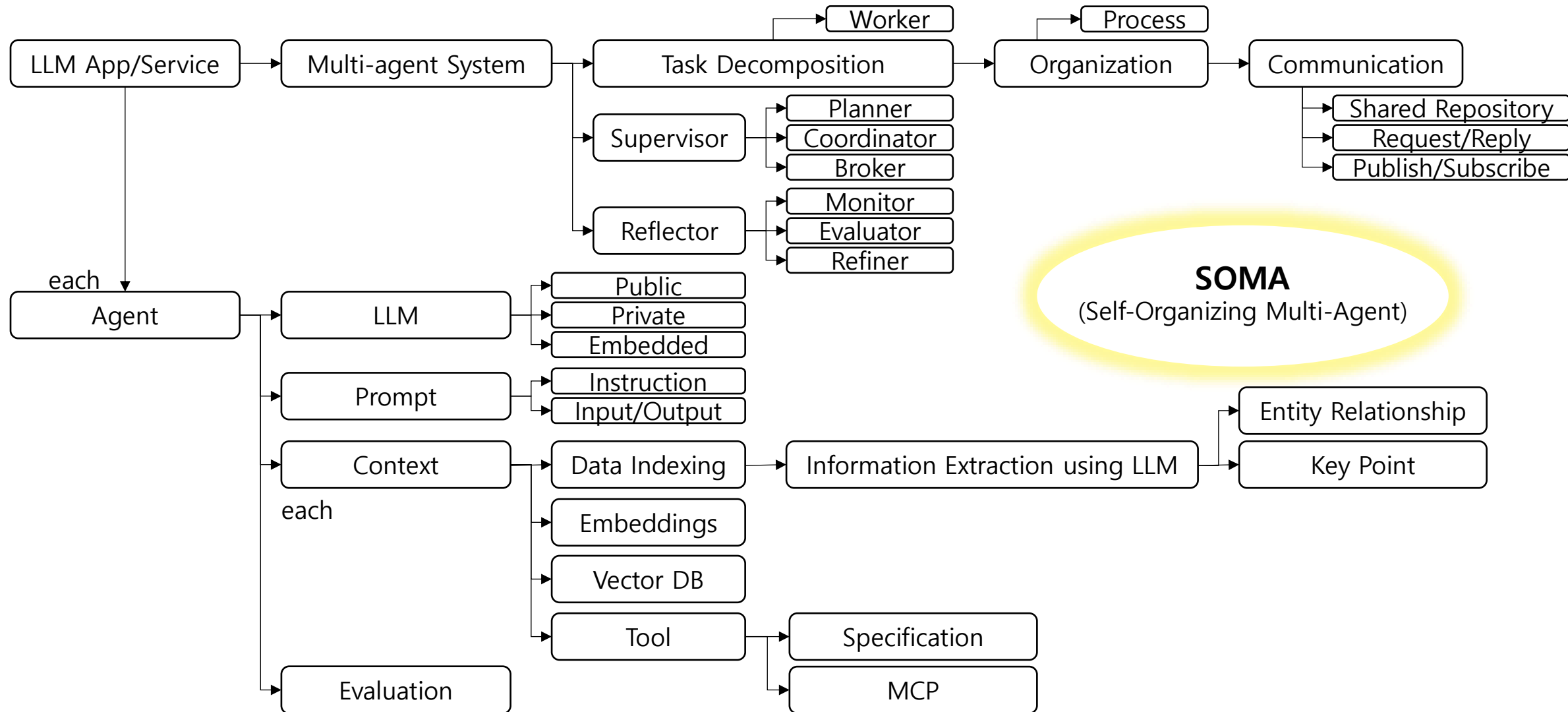


Just like a company has departments (Sales, Engineering, Marketing) working together under leadership to achieve business goals. CrewAI helps you an organization of AI agents with specialized roles collaborating to accomplish complex tasks.

LLM App/Service Design Guide



LLM App/Service Design Guide



SOMA
(Self-Organizing Multi-Agent)

요약

- LLM은 자연어 기반 지능형 서비스를 가능하게 함.
➔ 앱/서비스의 경쟁력 강화
- LLM 앱은 소프트웨어 개발 패러다임을 근본적으로 변화시킴.
- LLM 앱 개발 역량을 강화하는 것은
개인과 조직의 미래 경쟁력 확보에 가장 중요한 투자임.